

设备网络 **SDK** 开发手册

(人脸模块)

(For Windows/Linux)

版本：4.3

声明

非常感谢您购买我公司的产品，如果您有什么疑问或需要请随时联系我们。

- 我们已尽量保证手册内容的完整性与准确性，但也不免出现技术上不准确、与产品功能及操作不相符或印刷错误等情况，如有任何疑问或争议，请以我司最终解释为准。
- 产品和手册将实时进行更新，恕不另行通知。
- 本手册中内容仅为用户提供参考指导作用，请以 SDK 实际内容为准。

目 录

第 1 章 系统简介.....	7
1.1 概述.....	7
1.2 开发包组成.....	7
1.3 注意事项.....	10
第 2 章 版本说明.....	10
2.1 Version 4.3	10
第 3 章 函数调用顺序.....	11
3.1 SDK 接口调用主要流程.....	11
3.2 主动模式（不带目录服务器）	12
3.3 连接视频.....	13
3.4 下载录像.....	14
3.5 录像回放.....	16
3.6 人脸检测.....	17
3.7 人脸识别.....	18
3.8 人脸检索.....	20
3.8.1 检索人脸底图.....	20
3.8.2 检索抓拍图（按图片）	22
3.8.3 检索抓拍图（按事件）	23
3.8.4 检索抓拍图（按特征）	23
第 4 章 函数调用实例.....	24
4.1 主动模式（不带目录服务器）	37
4.2 连接视频.....	38
4.3 下载录像.....	42
4.4 录像回放.....	57
4.5 人脸检测.....	65
4.6 人脸识别.....	75
4.7 人脸检索.....	104
第 5 章 函数说明.....	122
5.1 SDK 初始化.....	122
5.1.1 初始化 SDK.....	122
5.1.2 设置回调函数.....	123
5.1.3 设置响应消息.....	127
5.1.4 释放 SDK.....	128
5.2 主动模式（不带目录服务器）	129
5.2 登录设备.....	129
5.2.1 登录设备.....	129
5.2.2 退出登录.....	131
5.3 连接视频.....	131
5.3.1 开始连接.....	131
5.3.2 停止连接.....	133
5.3.2 设置取原始帧数据的回调函数.....	133

5.3 下载录像.....	134
5.3.1 查询录像文件.....	134
5.3.2 远程下载录像文件.....	135
5.3.3 按时间段下载前端存储的录像文件.....	136
5.4 录像回放.....	137
5.4.1 录像回放.....	137
5.4.2 录像回放控制.....	138
5.4.3 停止录像回放.....	139
5.5 人脸检测.....	140
5.5.1 开启/暂停智能分析.....	140
5.5.2 开启/关闭人脸检测算法.....	142
5.5.3 设置/获取人脸检测算法参数.....	143
5.5.4 开启/停止人脸图片流.....	145
5.6 人脸识别.....	146
5.6.1 人脸库查询.....	146
5.6.2 人脸库添加.....	148
5.6.3 人脸库修改.....	149
5.6.4 人脸库删除.....	150
5.6.5 人脸底图查询.....	151
5.6.6 人脸底图添加.....	152
5.6.7 人脸底图修改.....	153
5.6.8 人脸底图删除.....	154
5.6.9 开启/关闭人脸识别算法.....	154
5.6.10 设置/获取人脸识别参数.....	156
5.6.11 设置/获取人脸布放使能.....	159
5.6.12 设置/获取人脸布放模板.....	160
5.6.13 设置/获取人脸报警参数.....	161
5.7 人脸检索.....	162
5.7.1 按人脸底图检索.....	162
5.7.2 按抓拍图检索.....	164
5.7.3 按事件检索.....	166
5.7.4 按特征检索.....	167
5.8 用户数据.....	167
5.8.1 获得当前播放实时流的用户数据.....	167
5.9 获取错误码.....	168
第 6 章 结构体说明.....	170
6.1 结构体.....	170
6.1.1. 场景信息结构体.....	170
6.1.2 人脸检测参数.....	172
6.1.3 人脸检测图片流上传参数.....	175
6.1.4 图片流相关结构体.....	176
6.1.5 智能分析暂停结果结构体.....	177
6.1.6 暂停智能分析结构体.....	177
6.1.7 人脸报警信息结构体.....	177

6.1.8 人脸属性结构体.....	179
6.1.9 人脸报警入参结构体.....	180
6.1.10 登陆参数结构体.....	181
6.1.11 人脸检测库信息结构体.....	182
6.1.12 人脸库查询结果.....	182
6.1.13 人脸检测结果回复结构体.....	183
6.1.14 人脸检测库删除结构体.....	185
6.1.15 查询人脸信息结构体.....	185
6.1.16 人脸检测底图查询结果.....	186
6.1.17 人脸检测底图信息结构体.....	187
6.1.18 删除人脸结构体.....	189
6.1.19 报警模板使能结构体.....	190
6.1.20 报警联动参数结构体.....	191
6.1.21 时间模板结构体.....	192
6.1.22 人脸相关结构体.....	192
6.1.23 图片信息相关结构体.....	195
6.1.24 时间模板结构体.....	195
6.1.25 连接视频时需要传递给开发包的数据结构.....	196
6.1.26 前端录像文件查询条件结构体.....	197
6.1.27 录像时间属性结构体.....	198
6.1.28 录像文件属性结构体.....	198
6.1.29 录像文件查询结构体.....	199
6.1.30 主动模式登陆参数.....	201
6.1.31 录像文件属性扩展结构体.....	201
6.1.32 回放结构体.....	202
6.1.33 原始流数据结构体.....	203
6.1.34 音视频数据信息.....	204
6.1.35 人脸库查询结构体.....	205
6.1.36 人脸库修改结构体.....	206
6.1.37 人脸底图修改结构体.....	206
6.1.38 抠图结构体.....	207
6.1.39 抠图结果结构体.....	207
6.1.40 抠图结果查询结构体.....	208
6.1.41 搜索条件结构体.....	209
6.1.42 通道结构体.....	210
6.1.43 查询搜索进度结构体.....	210
6.1.44 搜索结果查询结构体.....	210
6.1.45 搜索结果结构体.....	211
6.1.46 智能分析文件属性结构体.....	212
6.1.47 查询智能分析文件结构体.....	213
6.1.48 智能分析文件查询结果结构体.....	214
6.1.49 设备在线状态结构体.....	215
第 7 章 错误代码及说明.....	222
第 8 章 SDK 返回值及说明.....	225

第 1 章 系统简介

1.1 概述

设备网络 SDK（人脸模块）是基于设备私有网络通信协议开发的，为嵌入式网络硬盘录像机，NVR，网络摄像机，网络球机，视频服务器等产品服务的配套模块，用于远程访问和控制设备软件的二次开发，支持人脸系列等所有标准网络设备。

设备网络 SDK 主要功能

视频连接，文件回放和下载，人脸检测，人脸识别，人脸检索，人脸库管理，人脸底图管理功能等。

SDK 支持的系统

Windows 下设备网络 SDK：

（1）32/64 位的 Windows10/Windows8/Windows7/vista/xp/2000 以及 Windows Server 2008/2003。

Linux 下设备网络 SDK：

- （1）32/64 位的 Linux 系统，版本要求 gcc 4.1 或者 4.1 以上。
- （2）测试过的系统：RedHat、Suse、Ubuntu、CentOS、arm Embedded system。

1.2 开发包组成

设备网络 SDK 包含网络通讯库，软解码库，硬解码库等功能组件，我们提供 Windows 和 Linux 两个版本的 SDK。

Windows 下设备网络 SDK：

动态库. dll	说明
NVSSDK. dll	开发包网络核心动态库
PlaySdkM4. dll	播放库动态库
AVDecSDK. dll	全系列解码动态库
AVShowSDK. dll	音视频显示播放动态库
AVFilterSDK. dll	音视频数据提取解析库
OsCore. dll	Os 交互库，所有 SDK 必须依赖库

NetAdmin.dll	搜索器库
AudioDec.dll	音频解码核心库
AudioDecV33.dll	音频解码核心库
VideoDecV33.dll	视频解码核心库
VideoDecV33_SP.dll	视频解码核心库
VideoDecV5.dll	视频解码核心库
VideoMjpegDec.dll	mjpeg 的解码库
AudioDecAMR.dll	AMR 音频解码库
RegServer.dll	注册中心动态库
proxysdk.dll	代理转发库
MultiMedia.dll	ps 流转码库
DOME_Pelco_P.dll	控制协议库

其他文件

类别	文件名
/lib/ 静态库文件	NVSSDK.lib
	PlaySdkM4.lib
	NetAdmin.lib
	RegServer.lib
/include/ 头文件	NetSdkClient.h
	GlobalTypes.h
	NetClientTypes.h
	net_sdk_types.h
	AVPlaySdkTypes.h
	ProductModel.h
	DEFINE_USER_ERROR.h
	RetVal.h
	NetAdmin.h
	ActionControl.h

SDK 精简组合

用途	库文件
网络	NVSSDK.dll

网络+解码+显示	NVSSDK.dll OsCore.dll PlaySdkM4.dll AVDecSDK.dll AVShowSDK.dll AudioDec.dll AudioDecV33.dll CharRes.dll AudioDecAMR.dll VideoDecV33_SP.dll VideoMjpegDec.dll VideoDecV5.dll
PTZ 透明通道控制组码	DOME_PELCO_D.dll DOME_PELCO_P.dll DOME_PLUS.dll PTZ_PELCO_D.dll PTZ_PELCO_P.dll
本地解码+显示	OsCore.dll PlaySdkM4.dll AVDecSDK.dll AVShowSDK.dll AudioDec.dll AudioDecV33.dll CharRes.dll AudioDecAMR.dll VideoDecV33_SP.dll VideoMjpegDec.dll VideoDecV5.dll
注册中心	RegServer.dll
主动模式域名解析查询	nslook.dll
搜索器	NetAdmin.dll

Linux 下设备网络 SDK 库:

文件名	说明
libnvssdk.so	开发包网络核心动态库
libactivelinkserver.so	建立网络连接库
libossdk.so	Os 交互库，所有 SDK 必须依赖库
libstreamproc.so	码流分析库
libregserver.so	目录服务器库
libproxysdk.so	代理库

libplaysdk.so	音视频播放库
libavdecsdk.so	音视频解码库
libavshowsdk.so	音视频显示库
libavfiltersdk.so	音视频提取解析库
libnetadmin.so	网络搜索库

1.3 注意事项

注意事项

1. NVSSDK 采用异步方式实现，登录、连接视频、设置参数操作调用接口成功后还需等待消息或回调后才能执行其他操作。

第 2 章 版本说明

2.1 Version 4.3

1. 本版NVSSDK继承了以前版本的所有功能。本手册主要介绍连接视频、录像查询与下载、人脸检测、人脸识别、人脸检索等功能。

新增同步阻塞接口：

NetClient_SyncLogon

功能：同步阻塞接口，立即返回结果，无需等待异步消息。

NetClient_SyncRealPlay

功能：同步阻塞接口，立即返回结果，无需等待异步消息。

NetClient_StopRealPlay

功能：停止实时预览接口。

NetClient_SyncQuery

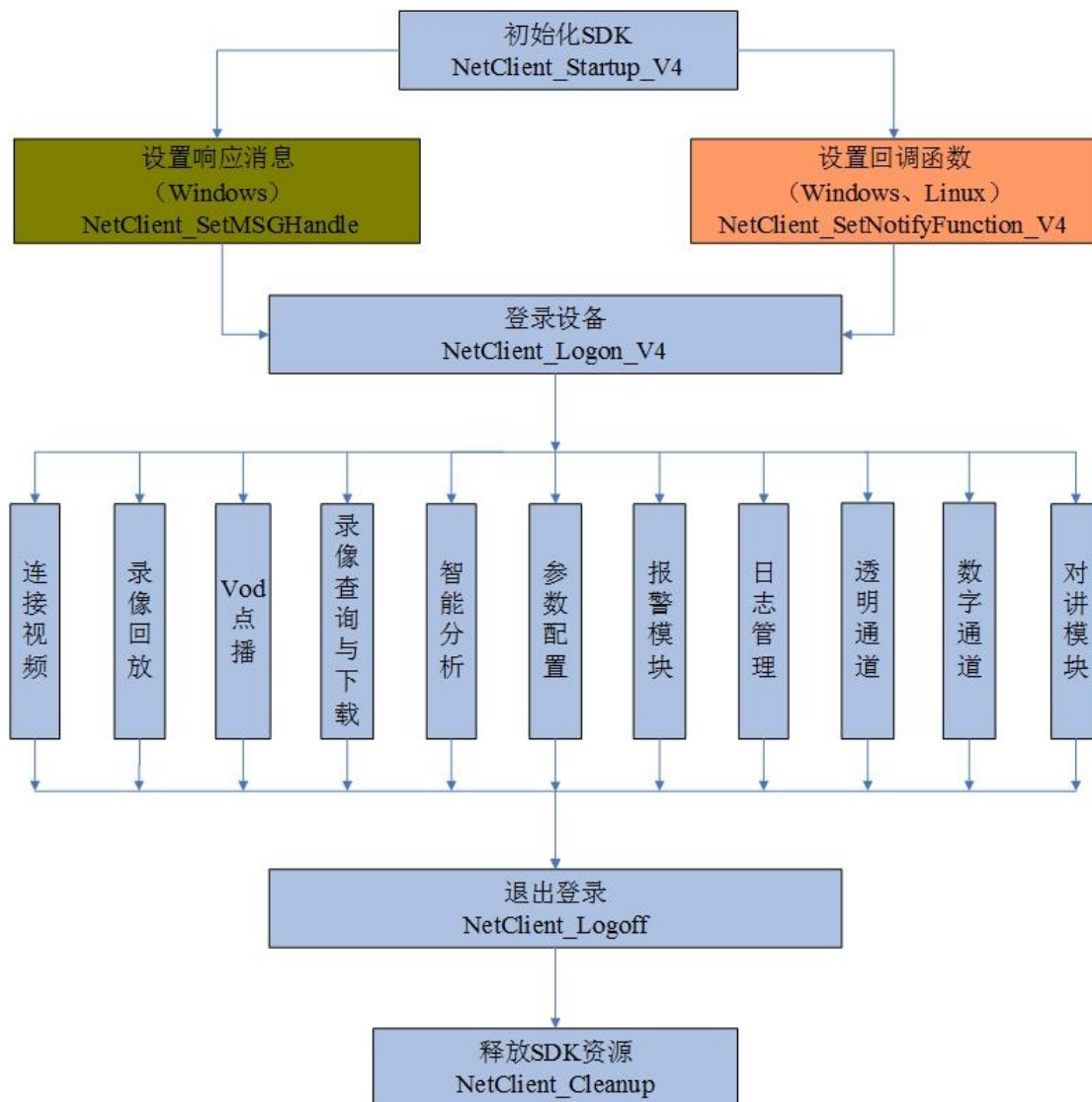
功能：同步阻塞接口，立即返回结果，无需等待异步消息，支持录像文件、设备日志和智能分析文件查询。

NetClient_SyncSetDevCfg

功能：同步阻塞接口，立即返回结果，无需等待异步消息。设置参数，目前仅支持暂停和开启智能分析，无需等待异步消息。

第 3 章 函数调用顺序

3.1 SDK 接口调用主要流程



(1) 初始化 SDK：调用接口 NetClient_Startup|NetClient_Startup_V4，对整个网络 SDK 系统的初始化，内存预分配等操作。

(2) 注册消息 (NetClient_SetMSGHandle) 和回调 (NetClient_SetNotifyFunction_V4)，其中消息机制仅限于 Windows 平台使用，而回调机制可用于 Windows 平台和 Linux 平台。注意：SDK 是异步运行模式，需注册消息回调来处理上层业务逻辑。

(3) 登录设备：调用 NetClient_Logon|NetClient_Logon_V4 完成操作。注意：登录操作成功后并不代表成功登录服务器，需要通过回调或者消息来获得登录状态，登录后会获得一个系统消息（如果设置了消息句柄），可以从消息判断登录是否成功。如果设置了回调函数，也可以在回调函数内处理登录结果，建议使用回调机制。关于 logonID 有效期的解释：在成功调用 NetClient_Logon_V4 至成功调用 NetClient_Logoff 之前的任何时段都是有效的。如果需要彻底销毁 ID，必须调用 NetClient_Logoff，无论这个设备是否真正连接成功。不

然将导致 ID 泄漏，最终无法连接其他设备（IP 无重复）。

（4）连接视频：客户端通过网络连接视频源可进行预览、存储等操作。相关接口按调用顺序如下：NetClient_StartRecv|NetClient_StartRecvEx |NetClient_StartRecv_V4 、等待视频头主消息回调、NetClient_StartPlay、NetClient_StopPlay、NetClient_StopRecv。

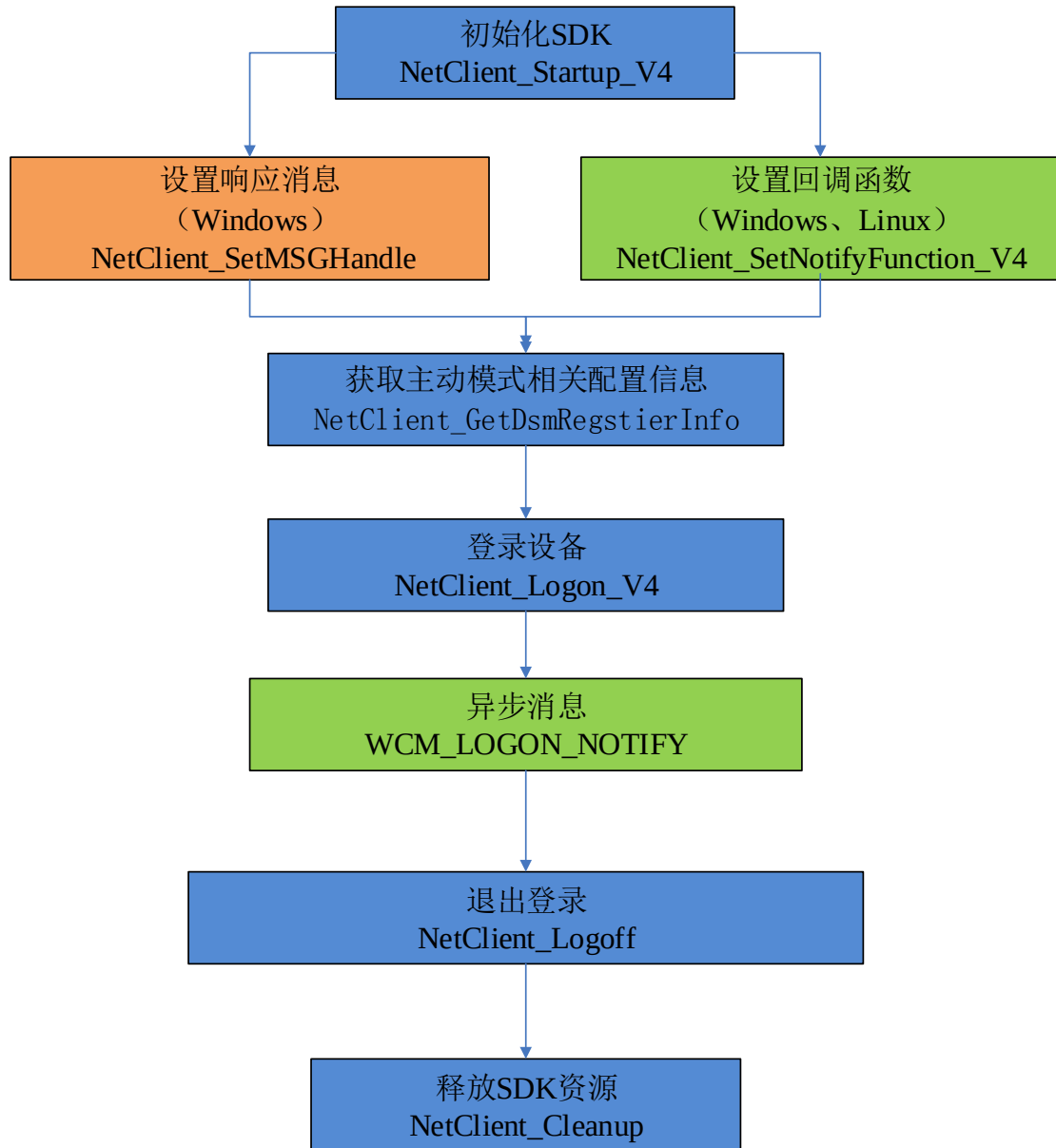
（5）录像查询与下载：远程下载设备（NVR 和 IPC）录像文件，远程文件下载也有两种方式，一种是按录像文件下载，一种按时间段下载。具体流程详见录像查询与下载流程。

（6）人脸检测：设备开启人脸检测算法后，可以进行人脸检测抓拍。人脸检测分为 IPC 检测和 NVR 检测两种，IPC 检测指前端连接抓拍相机，由抓拍相机进行人脸检测抓拍，NVR 检测指由 NVR 进行人脸检测抓拍。**当 NVR 数字通道接入人脸识别机时不能启用 NVR 本地检测。**

（7）人脸识别：设备开启人脸识别算法后，可以进行识别并联动相关报警。人脸识别分为 IPC 识别和 NVR 识别。IPC 识别指前端连接识别相机，由识别相机对抓拍的人脸进行识别比对，NVR 识别指由 NVR 对抓拍的人脸进行识别比对。设备开启人脸识别算法后，可以进行比对报警、陌生人报警，NVR 开启人脸识别后还可以进行频次报警和滞留报警。

（8）人脸检索（以图搜图）：人脸检索分为检索人脸底图和检索抓拍图两种形式，检索抓拍图可分为按图片检索、按事件检索、按特征检索三种形式。

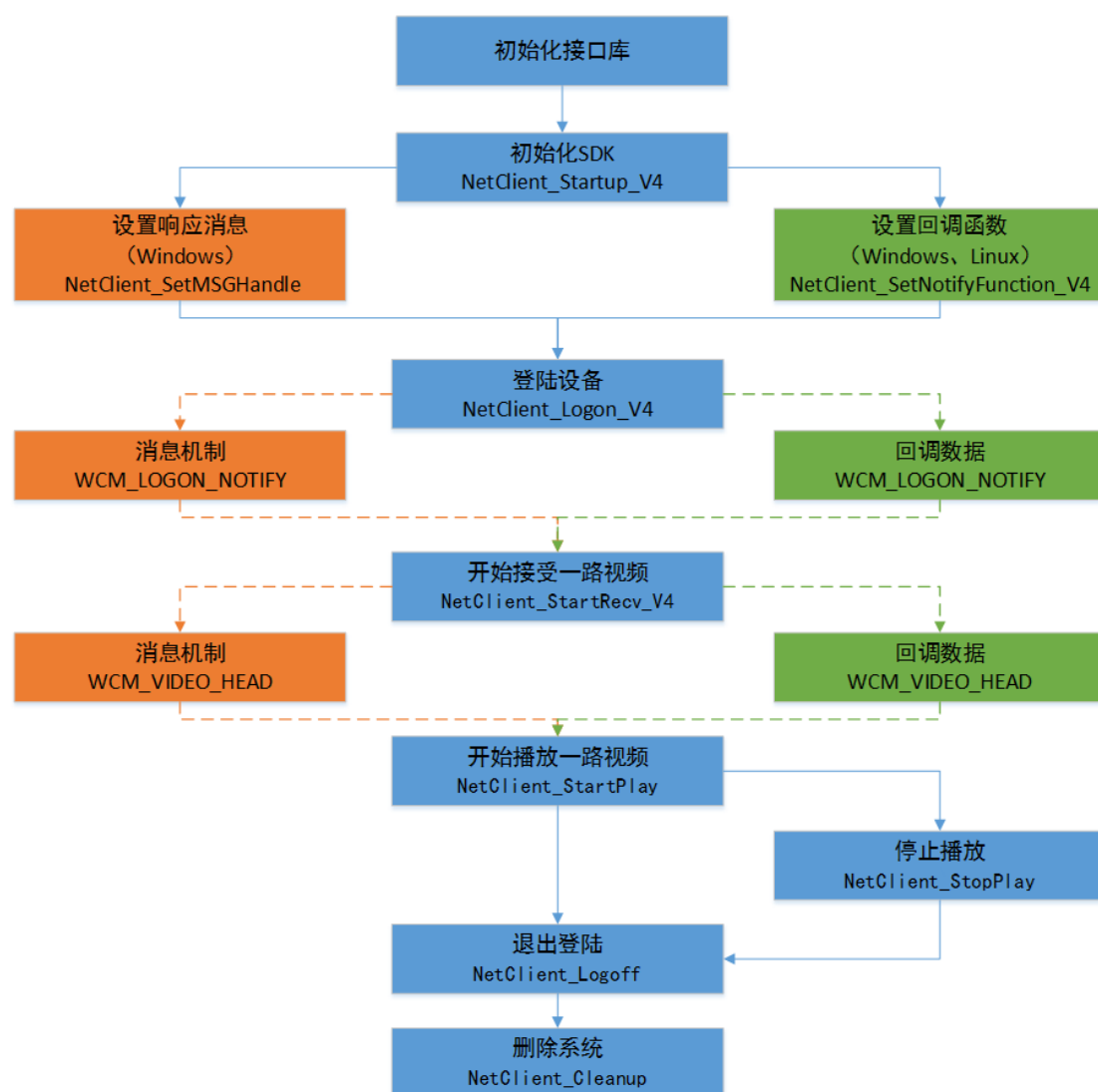
3.2 主动模式（不带目录服务器）



(1) 说明：主动模式是设备以主动连接客户端的方式完成登陆流程。

(2) 注意：此模式应用在设备处于局域网，服务器处于公网，服务器不能通过IP直连设备的情况。如果服务器能通过IP直连设备时，可跳过此节。

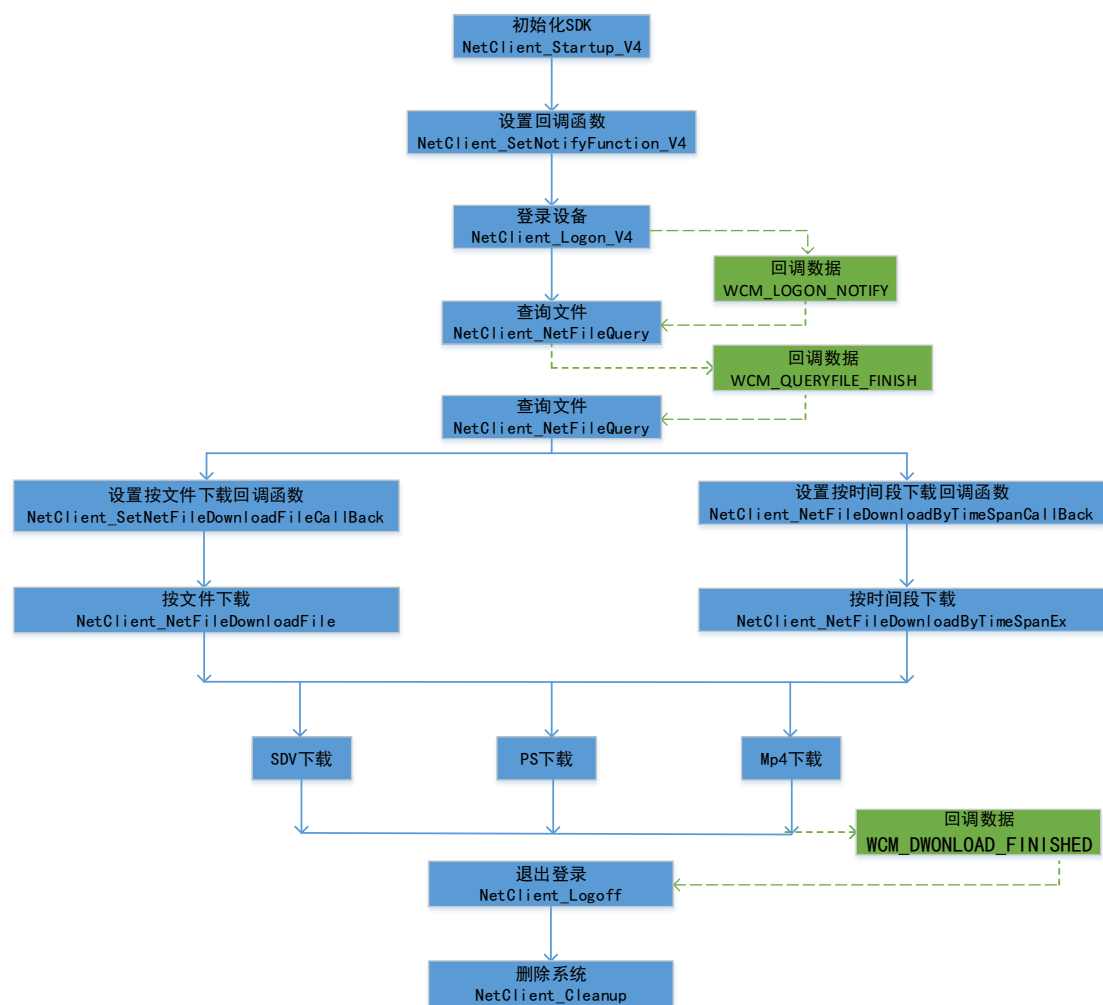
3.3 连接视频



(1) 说明：连接视频支持 tcp 、 udp 、 muc 网络传输模式。

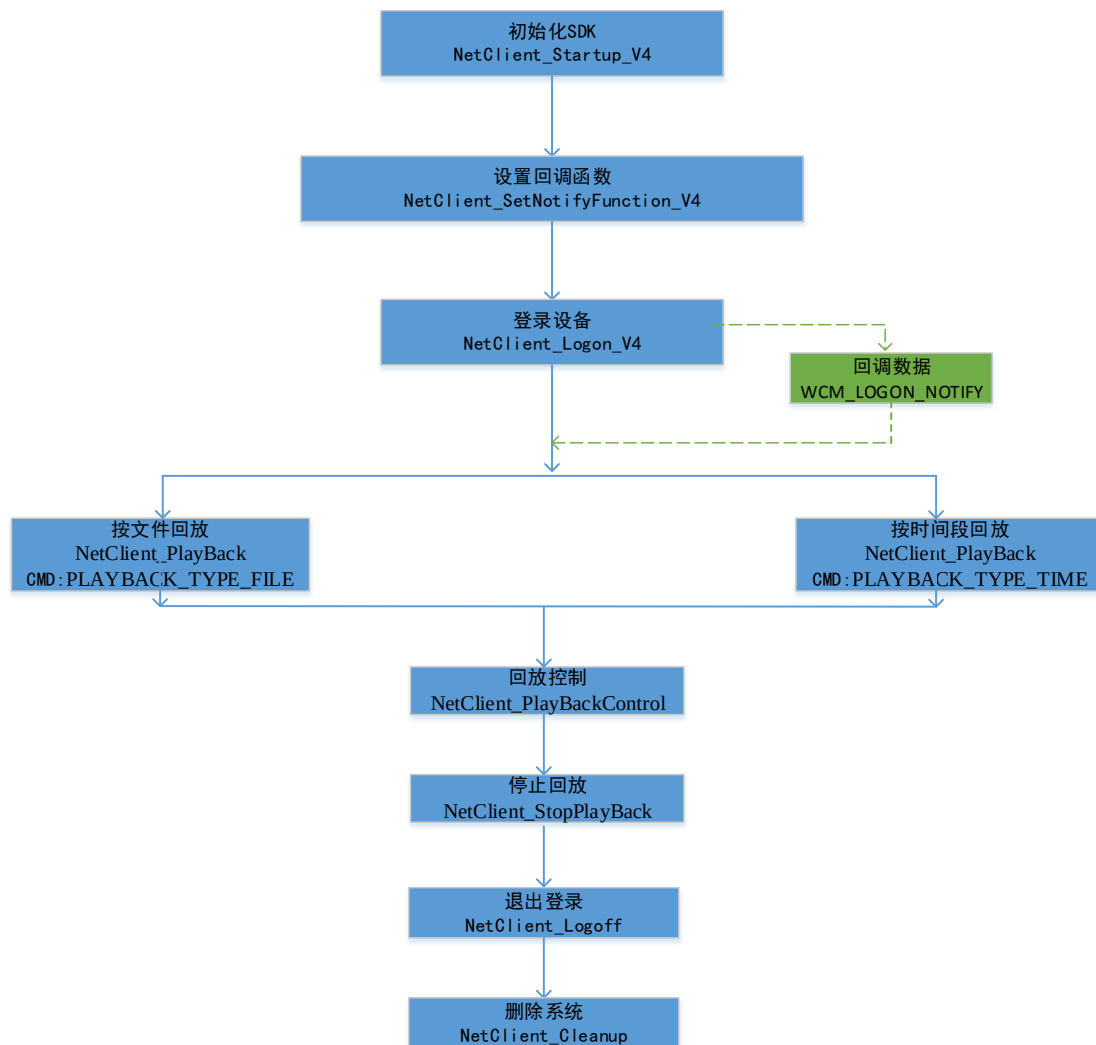
(2) 注意：收到登陆成功的消息后进行连接视频的操作，收到视频头的消息后进行播放视频的操作。

3.4 下载录像



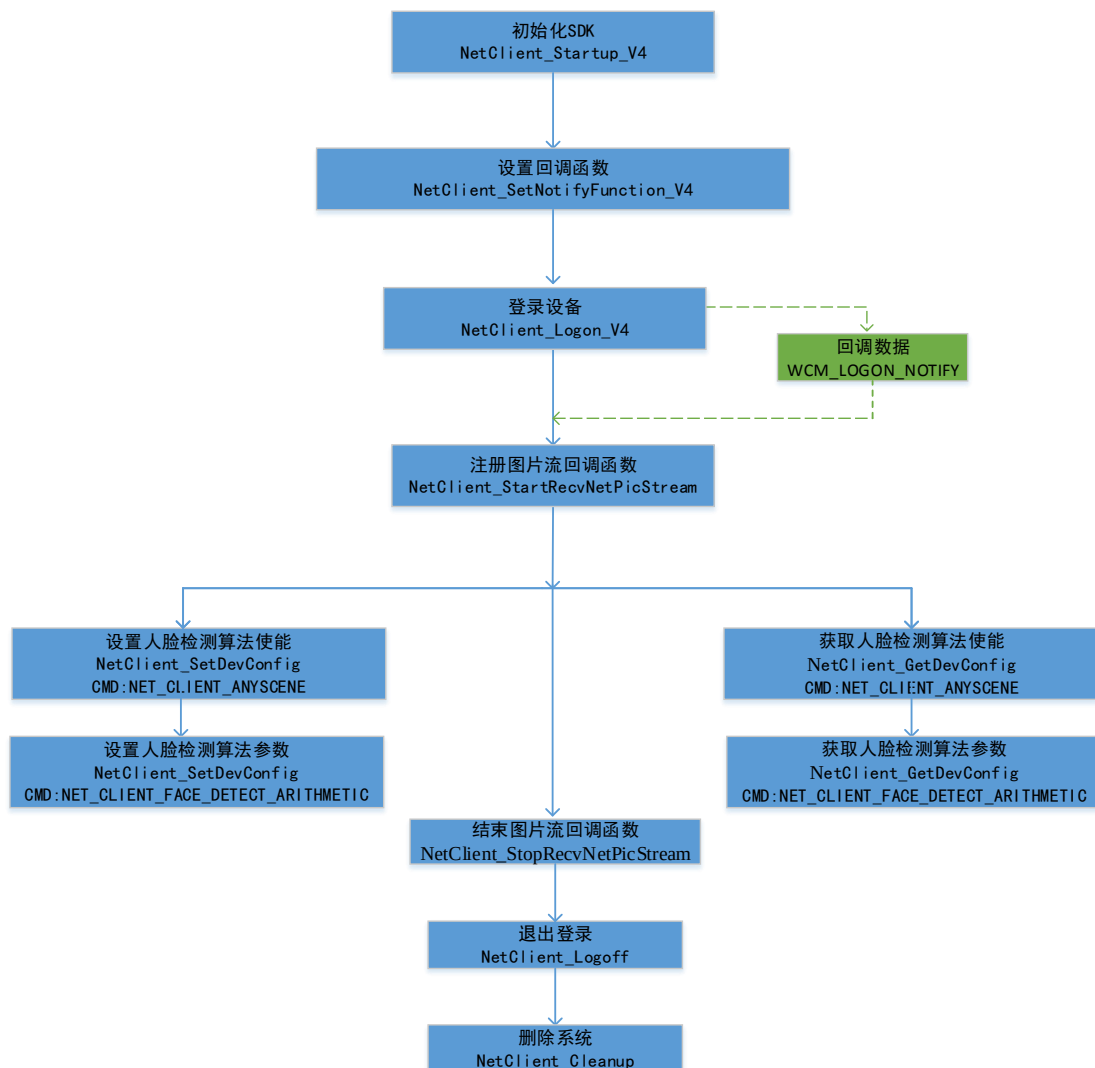
说明：本流程用于远程下载前端设备录像文件，远程文件下载有两种方式，一种是按录像文件下载，一种按时间段下载。

3.5 录像回放



说明：远程回放前端录像，分为按时间段回放和按文件回放。

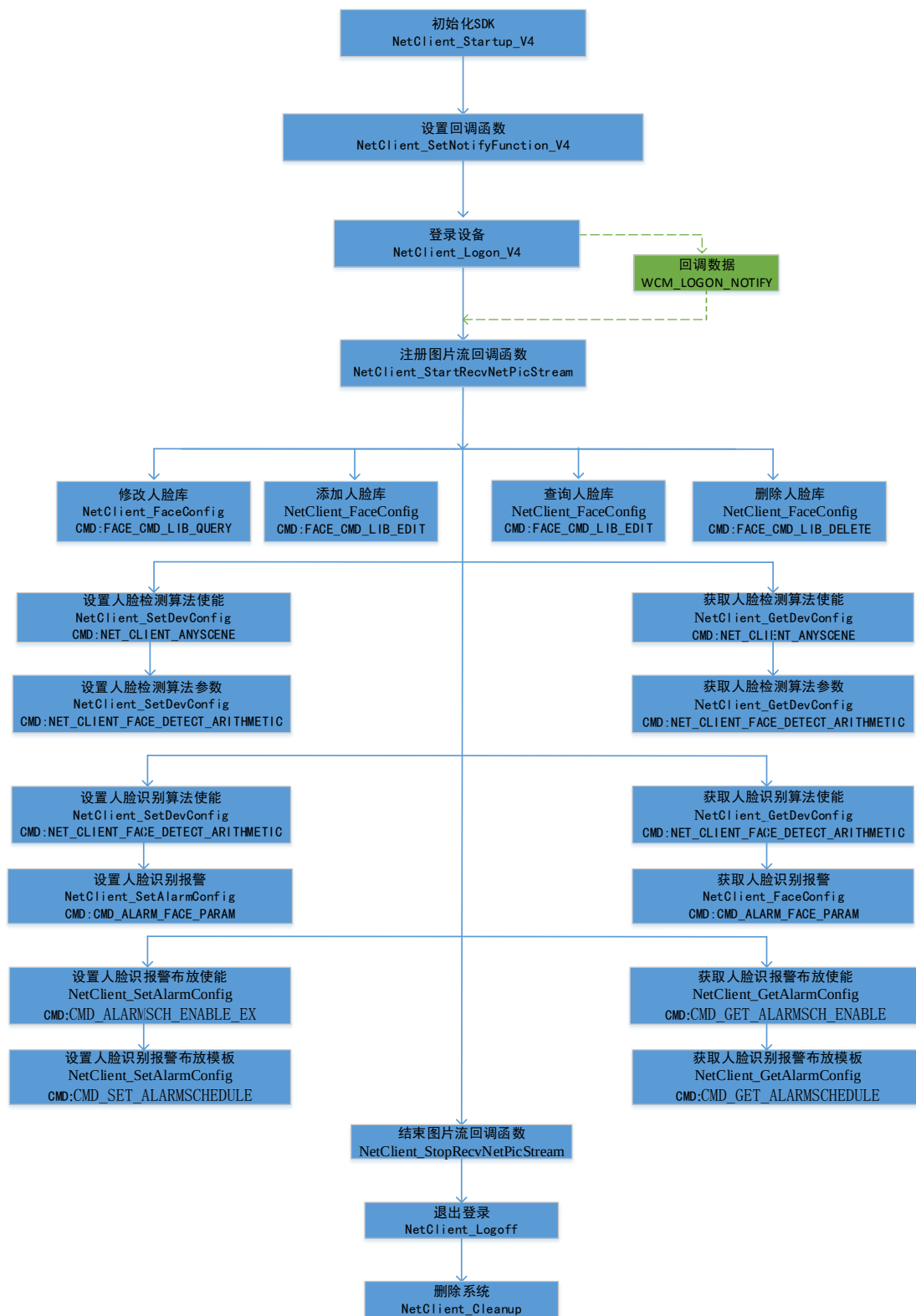
3.6 人脸检测



说明：人脸模块，通过摄像机进行人脸侦测并抓拍可得到人脸图片流。

- (1) 接收图片流：等待主回调登录设备成功的消息，之后调用接口 `NetClient_StartRecvNetPicStream` 开始接收人脸图片流，可得到接收图片流 ID（用于停止接收图片流）。其中回调数据 `NET_PICSTREAM_NOTIFY` 中的参数 `_lCommand` 为 `[NET_PICSTREAM_CMD_FACE]`，接收到的为人脸图片流（可以参照示例程序 `FacePicStreamDemo`）；
- (2) 数据处理：此处用户可以对通过回调得到的人脸图片进行一些自定义处理；
- (3) 启用人脸检测算法：调用接口 `NetClient_SetDevConfig` 开启人脸检测算法，其中接口对应的命令宏为 `[NET_CLIENT_ANYSCENE]`；
- (4) 设置人脸检测算法参数：启用完人脸检测算法之后，可以更改相应的参数，调用接口 `NetClient_SetDevConfig` 可以设置人脸检测算法的参数，其中接口对应的命令宏为 `[NET_CLIENT_FACE_DETECT_ARITHMETIC]`；
- (5) 停止接收图片流：调用接口 `NetClient_StopRecvNetPicStream` 停止接收人脸图片流，只需传入接收图片流 ID 即可；

3.7 人脸识别



说明：人脸识别模块，本模块对人脸库和人脸底图进行增、删、改、查，并由设备完成识别，通过 SDK 回调方式返回识别结果及图片。

（1）接收图片流：等待主回调登录设备成功的消息，之后调用接口

`NetClient_StartRecvNetPicStream` 开始接收人脸图片流，可得到接收图片流 ID（用于停止接收图片流）。其中回调数据 `NET_PICSTREAM_NOTIFY` 中的参数 `_lCommand` 为 `[NET_PICSTREAM_CMD_FACE]`，接收到的为人脸图片流（可以参照示例程序 `FacePicStreamDemo`）；

（2）数据处理：此处用户可以对通过回调得到的人脸图片进行一些自定义处理；

（3）人脸库、人脸底图操作，调用接口 `NetClient_FaceConfig` 根据不同的 `_iCmdId` 参数实现增、删、改、查等功能；

（4）启用人脸识别算法：调用接口 `NetClient_SetDevConfig` 开启人脸识别算法，其中接口对应的命令宏为 `[NET_CLIENT_ANYSCENE]`；

（5）设置人脸识别算法参数：启用完人脸识别算法之后，可以更改相应的参数，调用接口 `NetClient_SetDevConfig` 可以设置人脸识别算法的参数，其中接口对应的命令宏为 `[NET_CLIENT_FACE_DETECT_ARITHMETIC]`；

（6）设置人脸识别报警参数（只有 NVR 设备支持此参数设置）：调用接口 `NetClient_SetAlarmConfig` 设置人脸识别报警，其中接口对应的命令宏为 `[CMD_ALARM_FACE_PARAM]`，接口参数 `_iAlarmType` 宏值为 `[ALARM_TYPE_NVR_VCA]`，人脸识别的子报警类型为：1 比对报警，2 陌生人报警，3 频次报警，4 滞留报警，详细参数参考结构体 `FaceAlarmParam`；

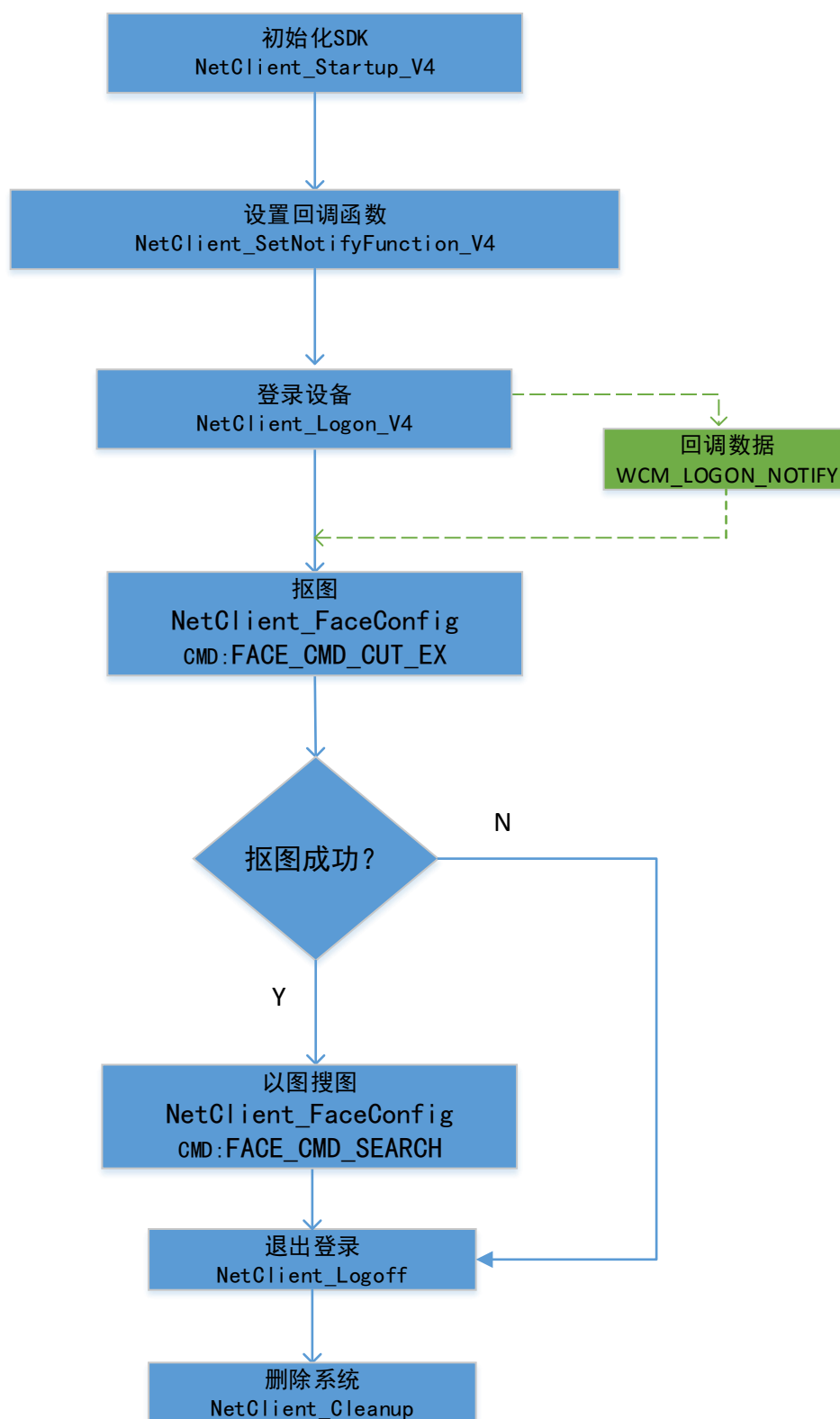
（7）设置人脸识别布防使能：调用接口 `NetClient_SetAlarmConfig` 人脸识别布防使能，其中接口对应的命令宏为 `[CMD_ALARMSCH_ENABLE_EX]`，当设备类型为 IPC 时，接口参数 `_iAlarmType` 宏值为 `[ALARM_TYPE_FACE_IDENT]`，当设备类型为 NVR 时，接口参数 `_iAlarmType` 宏值为 `[ALARM_TYPE_NVR_VCA]`，结构体详见 `TAlarmScheEnableParam`；

（8）设置人脸识别布防模板：调用接口 `NetClient_SetAlarmConfig` 人脸识别布防模板，其中接口对应的命令宏为 `[CMD_SET_ALARMSCHEDULE]`，当设备类型为 IPC 时，接口参数 `_iAlarmType` 宏值为 `[ALARM_TYPE_FACE_IDENT]`，当设备类型为 NVR 时，接口参数 `_iAlarmType` 宏值为 `[ALARM_TYPE_NVR_VCA]`，结构体详见 `TAlarmScheduleParam`；

（9）停止接收图片流：调用接口 `NetClient_StopRecvNetPicStream` 停止接收人脸图片流，只需传入接收图片流 ID 即可；

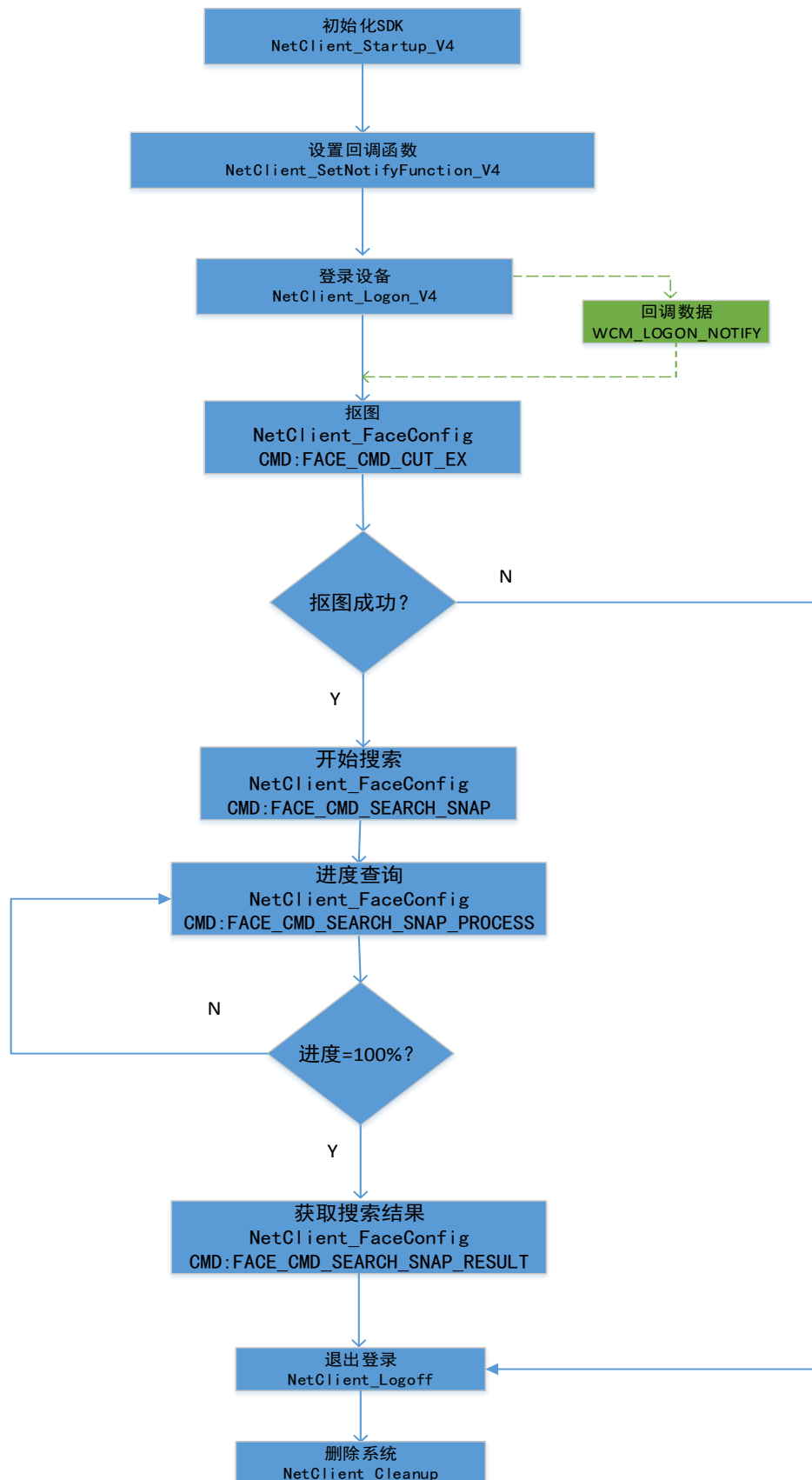
3.8 人脸检索

3.8.1 检索人脸底图



- 说明：本功能为从人脸底库中查询特定人脸图片（NVR 支持），具体步骤如下
- （1）抠图：调用 `NetClient_FaceConfig` 接口，使用宏定义 `FACE_CMD_CUT_EX`，进行抠图，抠图成功方可进行后续步骤；
 - （2）以图搜图：在抠图成功的条件下，调用 `NetClient_FaceConfig` 接口，使用宏定义 `FACE_CMD_SEARCH`，对抠图结果进行检索；

3.8.2 检索抓拍图（按图片）



-
- 说明：本功能为从人脸底库中查询特定人脸图片，具体步骤如下
- (1) 抠图：调用 NetClient_FaceConfig 接口，使用宏定义 FACE_CMD_CUT_EX，进行人脸特征值提取，抠图成功方可进行后续步骤；
 - (2) 开始搜索：在抠图成功的条件下，调用 NetClient_FaceConfig 接口，使用宏定义 FACE_CMD_SEARCH_SNAP，进行搜索；
 - (3) 进度查询：调用 NetClient_FaceConfig 接口，使用宏定义 FACE_CMD_SEARCH_SNAP_PROCESS，查询搜索进度，当进度为 100%时，可获取搜索结果；
 - (4) 获取结果：当搜索进度为 100%时，调用 NetClient_FaceConfig 接口，使用宏定义 FACE_CMD_SEARCH_SNAP_RESULT，获取搜索结果；

3.8.3 检索抓拍图（按事件）

说明：本功能为按事件查询特定人脸图片，事件类型为，1-人脸检测 2-人脸比对 3-陌生人 4-频次 5-时长。具体步骤如下

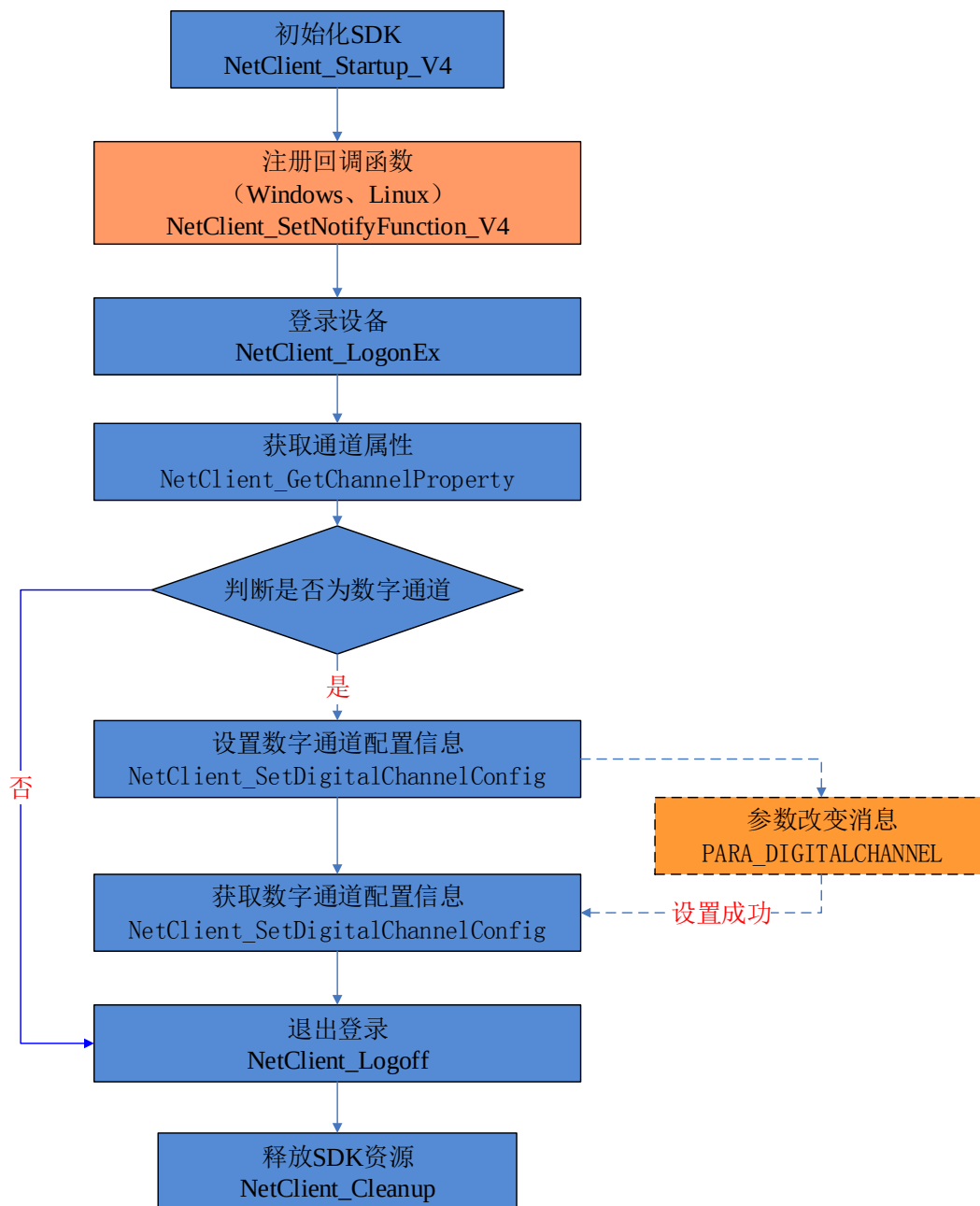
- (1) 检索：调用 NetClient_Query_V5 接口，使用宏定义 CMD_NETFILE_QUERY_VCA，进行查询；

3.8.4 检索抓拍图（按特征）

说明：本功能为按特征查询特定人脸图片，特征类型为，年龄，性别，民族，戴眼镜，戴口罩。具体步骤如下

- (1) 检索：调用 NetClient_Query_V5 接口，使用宏定义 CMD_NETFILE_QUERY_VCA，进行查询；

3.9 数字通道



说明：混合型 DVR/NVR 等设备支持 IPC 接入，通道称为数字通道，配置相关参数时需调用 IP 接入配置参数来进行资源的获取和重新分配。

第 4 章 函数调用实例

注意：函数调用实例默认已经导入了动态库头文件和动态库源文件，下面是动态库文件内容：

头文件：NetSDK.h

```
#ifndef NETSDK_H
#define NETSDK_H

#include "NetClientTypes.h"

#define USER_ERROR 0x10000000
#define ERROR_ALREADY_INTERTALK (USER_ERROR+0x32) // 正在进行双向对讲

//NVSSDK 初始化
typedef int (__stdcall *pfNetClient_Startup_V4)(int _iServerPort, int _iClientPort, int _iWnd);
extern pfNetClient_Startup_V4 NetClient_Startup_V4;
typedef int (__stdcall *pfNetClient_Cleanup)();
extern pfNetClient_Cleanup NetClient_Cleanup;
typedef int (__stdcall *pfNetClient_SetNotifyFunction_V4)(MAIN_NOTIFY_V4 _MainNotify,
ALARM_NOTIFY_V4 _AlarmNotify, PARACHANGE_NOTIFY_V4 _ParaNotify,
COMRECV_NOTIFY_V4 _ComNotify, PROXY_NOTIFY_ProxyNotify);
extern pfNetClient_SetNotifyFunction_V4 NetClient_SetNotifyFunction_V4;

//登录
typedef int (__stdcall *pfNetClient_Logon)(char* _cProxy, char* _cIP, char* _cUserName, char*
_cPassword, char* _pcProID, int _iPort);
extern pfNetClient_Logon NetClient_Logon;
typedef int (__stdcall *pfNetClient_Logoff)(int _iLogonID);
extern pfNetClient_Logoff NetClient_Logoff;
typedef int (__stdcall *pfNetClient_GetChannelNum)(int _iLogonID, int* _iChannelNum);
extern pfNetClient_GetChannelNum NetClient_GetChannelNum;
typedef int (__stdcall *pfNetClient_Logon_V4)(int _iLogonType, void* _pvPara, int
_iInBufferSize);
extern pfNetClient_Logon_V4 NetClient_Logon_V4;

//视频预览
typedef int (__stdcall *pfNetClient_StartRecv)(unsigned int *_ulConID, CLIENTINFO* _cltInfo,
RECVDATA_NOTIFY_cbkDataArrive);
extern pfNetClient_StartRecv NetClient_StartRecv;
typedef int (__stdcall *pfNetClient_StartRecv_V4)(unsigned int* _uiRecvID, CLIENTINFO*
_cltInfo, NVSDATA_NOTIFY_cbkDataArrive,void* _iUserData);
extern pfNetClient_StartRecv_V4 NetClient_StartRecv_V4;
typedef int (__stdcall *pfNetClient_StartPlay)(unsigned int _ulID, HWND _hWnd, RECT
_rcShow, unsigned int _iDecflag);
extern pfNetClient_StartPlay NetClient_StartPlay;
typedef int (__stdcall *pfNetClient_StopPlay)(unsigned int _ulID);
extern pfNetClient_StopPlay NetClient_StopPlay;
```

```

typedef int (__stdcall *pfNetClient_AudioStart)(unsigned long _ulConID);
extern pfNetClient_AudioStart NetClient_AudioStart;
typedef int (__stdcall *pfNetClient_StartCaptureData)(unsigned long _ulID);
extern pfNetClient_StartCaptureData NetClient_StartCaptureData;
typedef int (__stdcall *pfNetClient_StopCaptureData)(unsigned long _ulID);
extern pfNetClient_StopCaptureData NetClient_StopCaptureData;
typedef int (__stdcall *pfNetClient_SetRawFrameCallBack)(unsigned int _ulConID,
RAWFRAME_NOTIFY_cbkGetFrame, void* _pContext);
extern pfNetClient_SetRawFrameCallBack NetClient_SetRawFrameCallBack;
typedef int (__stdcall *pfNetClient_SetDataPackCallBack)(unsigned int _ulConID, int
_iCbctype, void* _pvCallBack, void* _pvUserData);
extern pfNetClient_SetDataPackCallBack NetClient_SetDataPackCallBack;
typedef int (__stdcall *pfNetClient_StopRecv)(unsigned int _uiRecvID);
extern pfNetClient_StopRecv NetClient_StopRecv;

//远程回放
typedef int (__stdcall *pfNetClient_PlayBack)(unsigned int* _ulConID, int _iCmd, PlayerParam*
_ptPlayerParam, void* _hWnd);
extern pfNetClient_PlayBack NetClient_PlayBack;
typedef int (__stdcall *pfNetClient_StopPlayBack)(unsigned long _ulConID);
extern pfNetClient_StopPlayBack NetClient_StopPlayBack;

//文件下载
typedef int (__stdcall* pfNetClient_NetFileQuery)(int _iLogonID, PNVS_FILE_QUERY
_fileQuery);
extern pfNetClient_NetFileQuery NetClient_NetFileQuery;
typedef int (__stdcall *pfNetClient_NetFileGetFileCount)(int _iLogonID, int *_iTotalCount, int
*_iCurrentCount);
extern pfNetClient_NetFileGetFileCount NetClient_NetFileGetFileCount;
typedef int (__stdcall *pfNetClient_NetFileGetQueryfile)(int _iLogonID, int _iFileIndex,
PNVS_FILE_DATA_fileInfo);
extern pfNetClient_NetFileGetQueryfile NetClient_NetFileGetQueryfile;
typedef int (__stdcall *pfNetClient_NetFileDownloadFile)(unsigned long *_ulConID,int
_iLogonID, char* _cRemoteFilename, char* _cLocalFilename, int _iFlag /*= 0*/, int _iPosition
/*= -1*/, int _Speed /*= 1*/);
extern pfNetClient_NetFileDownloadFile NetClient_NetFileDownloadFile;
typedef int (__stdcall *pfNetClient_SetNetFileDownloadFileCallBack)(unsigned long _ulConID,
RECV_DOWNLOADDATA_NOTIFY_cbkDataNotify, void* _lpUserData);
extern pfNetClient_SetNetFileDownloadFileCallBack
NetClient_SetNetFileDownloadFileCallBack;
typedef int (__stdcall *pfNetClient_NetFileDownloadByTimeSpanEx)( unsigned int* _ulConID,
int _iLogonID, char* _pcLocalFile, int _iChannelNO, NVS_FILE_TIME *_pTimeBegin,
NVS_FILE_TIME *_pTimeEnd, int _iFlag/*= 0*/, int _iPosition/*= -1*/,int _iSpeed/*= 1*/);
extern
pfNetClient_NetFileDownloadByTimeSpanEx

```

```
NetClient_NetFileDownloadByTimeSpanEx;
```

```
//智能分析
```

```
typedef int (__stdcall *pfNetClient_VCASetConfig)(int _iLogonID, int _iVCACmdID, int  
_iChannel, void* _lpCmdBuf, int _iCmdBufLen);
```

```
extern pfNetClient_VCASetConfig NetClient_VCASetConfig;
```

```
typedef int (__stdcall *pfNetClient_VCAGetConfig)(int _iLogonID, int _iVCACmdID, int  
_iChannel, void* _lpCmdBuf, int _iCmdBufLen);
```

```
extern pfNetClient_VCAGetConfig NetClient_VCAGetConfig;
```

```
typedef int (__stdcall *pfNetClient_VCARestartEx)(int _iLogonID, int _iChannelNO);
```

```
extern pfNetClient_VCARestartEx NetClient_VCARestartEx;
```

```
typedef int (__stdcall *pfNetClient_VCAGetAlarmInfo)(int _iLogonID, int _iAlarmIndex, void*  
_lpBuf, int _iBufSize);
```

```
extern pfNetClient_VCAGetAlarmInfo NetClient_VCAGetAlarmInfo;
```

```
//数字通道
```

```
typedef int (__stdcall *pfNetClient_GetDigitalChannelConfig)(int _iLogonID, int _iChannel, int  
_iCmd, void* _lpCmdBuf, int _iBufSize);
```

```
extern pfNetClient_GetDigitalChannelConfig NetClient_GetDigitalChannelConfig;
```

```
typedef int (__stdcall *pfNetClient_SetDigitalChannelConfig)(int _iLogonID, int _iChannel, int  
_iCmd, void* _lpCmdBuf, int _iBufSize);
```

```
extern pfNetClient_SetDigitalChannelConfig NetClient_SetDigitalChannelConfig;
```

```
typedef int (__stdcall *pfNetClient_GetChannelProperty)(int _iLogonID, int _iChannel, int  
_iCmd, void* _lpBuf, int _iBufSize);
```

```
extern pfNetClient_GetChannelProperty NetClient_GetChannelProperty;
```

```
//对讲
```

```
typedef int (__stdcall *pfNetClient_TalkStart)(int _iLogonID, int _bUser);
```

```
extern pfNetClient_TalkStart NetClient_TalkStart;
```

```
typedef int (__stdcall *pfNetClient_TalkEnd)(int _iLogonID);
```

```
extern pfNetClient_TalkEnd NetClient_TalkEnd;
```

```
typedef int (__stdcall *pfNetClient_InterTalkStart)(unsigned int * _uiConnID, int _iLogonID,  
int _iUserData);
```

```
extern pfNetClient_InterTalkStart NetClient_InterTalkStart;
```

```
typedef int (__stdcall *pfNetClient_InterTalkEnd)(unsigned int _uiConnID, bool _bStopTalk);
```

```
extern pfNetClient_InterTalkEnd NetClient_InterTalkEnd;
```

```
//PTZ 坐标
```

```
typedef int (__stdcall *pfNetClient_LogonEx)(char* _cProxy, char* _cIP, char* _cUserName, char*  
_cPassword, char* _pcProID, int _iPort, char* _pcCharSet);
```

```
extern pfNetClient_LogonEx NetClient_LogonEx;
```

```
typedef int (__stdcall *pfNetClient_GetDeviceType)(int _iLogonID, int _iChannelNum, int*  
_iComPort, int* _iDevAddress, char* _cDeviceType);
```

```
extern pfNetClient_GetDeviceType NetClient_GetDeviceType;
```

```
typedef int (__stdcall *pfNetClient_ComSend)(int _iLogonID,unsigned char* _ucBuf,int
_iLength,int _iComNo);
extern pfNetClient_ComSend NetClient_ComSend;

//日志管理
typedef int (__stdcall *pfNetClient_NetLogQuery)( int _iLogonID, PNVS_LOG_QUERY
_logQuery);
extern pfNetClient_NetLogQuery NetClient_NetLogQuery;
typedef int (__stdcall *pfNetClient_NetLogGetLogCount)(int _iLogonID, int *_iTotalCount, int
*_iCurrentCount);
extern pfNetClient_NetLogGetLogCount NetClient_NetLogGetLogCount;
typedef int (__stdcall *pfNetClient_NetLogGetLogfile)(int _iLogonID, int _iLogIndex,
PNVS_LOG_DATA _pLogInfo);
extern pfNetClient_NetLogGetLogfile NetClient_NetLogGetLogfile;

//参数配置
typedef int (__stdcall *pNetClient_GetVideoPara)(int _iLogonID, int _iChannelNum,
STR_VideoParam*_strVidoParam);
extern pNetClient_GetVideoPara NetClient_GetVideoPara;
typedef int (__stdcall *pNetClient_SetVideoPara)(int _iLogonID, int _iChannelNum,
STR_VideoParam*_strVidoParam);
extern pNetClient_SetVideoPara NetClient_SetVideoPara;
typedef int (__stdcall *pNetClient_SetDevConfig)(int _iLogonID, int _iCommand, int _iChannel,
void *_lpInBuffer, int _iInBufferSize);
extern pNetClient_SetDevConfig NetClient_SetDevConfig;
typedef int (__stdcall *pNetClient_GetDevConfig)(int _iLogonID, int _iCommand, int _iChannel,
void *_lpOutBuffer, int _iOutBufferSize, int *_lpBytesReturned);
extern pNetClient_GetDevConfig NetClient_GetDevConfig;

//主动模式
typedef int (__stdcall *pNetClient_GetDsmRegstierInfo)(int _iCommand, void* _pvBuf, int
_iBufSize);
extern pNetClient_GetDsmRegstierInfo NetClient_GetDsmRegstierInfo;

//报警模块
//设置联动输出端口
typedef int (__stdcall *pNetClient_SetMotionDecLinkOutport)(int _iLogonID, int _iChannelNum,
int _iOutportArray );
extern pNetClient_SetMotionDecLinkOutport NetClient_SetMotionDecLinkOutport ;
//获得联动输出端口
typedef int (__stdcall *pNetClient_GetMotionDecLinkOutport)(int _iLogonID, int
_iChannelNum, int*_iOutportArray );
extern pNetClient_GetMotionDecLinkOutport NetClient_GetMotionDecLinkOutport ;
//设置移动报警布防时间段
```

```

typedef int (__stdcall *pNetClient_SetMotionDetectSchedule)(int _iLogonID, int
_iChannelNum, int _iWeekday, PNVS_SCHEDTIME
_strScheduleParam[MAX_TIMESEGMENT]);
extern pNetClient_SetMotionDetectSchedule NetClient_SetMotionDetectSchedule ;
//获得移动报警布防时间段
typedef int (__stdcall *pNetClient_GetMotionDetectSchedule)(int _iLogonID, int
_iChannelNum, int _iWeekday, PNVS_SCHEDTIME
_strScheduleParam[MAX_TIMESEGMENT]);
extern pNetClient_GetMotionDetectSchedule NetClient_GetMotionDetectSchedule ;
//设置移动报警灵敏度
typedef int (__stdcall *pNetClient_SetThreshold)(int _iLogonID, int _iChannelNum, int
iThreshold );
extern pNetClient_SetThreshold NetClient_SetThreshold ;
//获得移动抱紧灵敏度
typedef int (__stdcall *pNetClient_GetThreshold)(int _iLogonID, int _iChannelNum, int*
iThreshold );
extern pNetClient_GetThreshold NetClient_GetThreshold ;
//使能移动报警
typedef int (__stdcall *pNetClient_SetMotionDetection)(int _iLogonID, int _iChannelNum, int
_iEnabled);
extern pNetClient_SetMotionDetection NetClient_SetMotionDetection ;

////////////////////////////////////回放库////////////////////////////////////
/*****
*   TC_PutStreamToPlayer 返回值说明
*****/
#define RET_BUFFER_FULL (-11) // 缓冲区满,数据没有放
入缓冲区
#define RET_BUFFER_WILL_BE_FULL (-18) // 即将满,降低送入数据
的频率
#define RET_BUFFER_WILL_BE_EMPTY (-19) // 即将空,提高送入数据
的频率
#define RET_BUFFER_IS_OK (-20) // 正常可以放数
据

/*回放器库消息的 LPARAM 参数取值列表*/
#define LPARAM_FILE_PLAY_END 0 // 播放完毕
#define LPARAM_FILE_SEARCH_END 1 // 寻找到文件末尾
#define LPARAM_NEED_DECRYPT_KEY 2 // 需要解密密码
#define LPARAM_DECRYPT_KEY_FAILED 3 // 解密密码错误
#define LPARAM_DECRYPT_SUCCESS 4 // 解密成功
#define LPARAM_STREAM_SEARCH_END 5 // 流缓冲区中没有数据了
#define LPARAM_VOD_STREAM_END 6 // VOD 文件流播放完毕

```

```

typedef int (__cdecl *pfTC_CreateSystem)(HWND _hWnd);
extern pfTC_CreateSystem TC_CreateSystem;//创建播放系统

typedef int (__cdecl *pfTC_DeleteSystem)();
extern pfTC_DeleteSystem TC_DeleteSystem;//删除播放器系统

typedef int (__cdecl* pfCBCommonEventNotifyEx)(int _iPlayerID, int _iEventMessage, void*
_lpUserData);
typedef int (__cdecl *pfTC_RegisterCommonEventCallBack_V4)(int _iID,
pfCBCommonEventNotifyEx _pf, void* _lpUserData);
extern pfTC_RegisterCommonEventCallBack_V4 TC_RegisterCommonEventCallBack_V4;//注
册回调

typedef int (__cdecl *pfTC_CreatePlayerFromFile)(HWND _hWnd, char* _pcFileName, int
_iDownloadBufSz, int _iFileTrueSz,
int* _piNowSz, int _iLastFrmNo, int*
piCompleteFlag);
extern pfTC_CreatePlayerFromFile TC_CreatePlayerFromFile;//创建播放器实例

typedef int (*pfTC_CreatePlayerFromVoD)(HWND _hWnd, unsigned char* _pucVideoHeadBuf,
int _iHeadSize);
extern pfTC_CreatePlayerFromVoD TC_CreatePlayerFromVoD;

typedef int ( *pfTC_PutStreamToPlayer )(int ID,unsigned char *_ucData,int _iSize);
extern pfTC_PutStreamToPlayer TC_PutStreamToPlayer;

typedef int (*pfTC_GetStreamPlayBufferState)( int _iID, int *_piStreamBufferState );
extern pfTC_GetStreamPlayBufferState TC_GetStreamPlayBufferState;

typedef int (__cdecl *pfTC_DeletePlayer)(int _iID);
extern pfTC_DeletePlayer TC_DeletePlayer;//删除播放器实例

typedef int (__cdecl *pfTC_Play)(int _iID);
extern pfTC_Play TC_Play;//播放
typedef int (__cdecl *pfTC_Stop)(int _iID);
extern pfTC_Stop TC_Stop;//停止播放

int LoadNVSSDK();
void FreeNVSSDK();

int LoadPlaySdk();
void FreePlaySdk();

#endif

```

源文件 NetSDK.cpp

```
#include "NetSDK.h"
```

```
HINSTANCE g_hNVSSDK = NULL;
```

```
HINSTANCE g_hPlaySdk = NULL;
```

```
pfNetClient_Startup_V4 NetClient_Startup_V4 = NULL;
```

```
pfNetClient_Logon NetClient_Logon = NULL;
```

```
pfNetClient_Logoff NetClient_Logoff = NULL;
```

```
pfNetClient_Logon_V4 NetClient_Logon_V4 = NULL;
```

```
pfNetClient_GetChannelNum NetClient_GetChannelNum = NULL;
```

```
pfNetClient_StartRecv NetClient_StartRecv = NULL;
```

```
pfNetClient_StartRecv_V4 NetClient_StartRecv_V4 = NULL;
```

```
pfNetClient_StopRecv NetClient_StopRecv = NULL;
```

```
pfNetClient_StartPlay NetClient_StartPlay = NULL;
```

```
pfNetClient_StopPlay NetClient_StopPlay = NULL;
```

```
pfNetClient_PlayBack NetClient_PlayBack = NULL;
```

```
pfNetClient_StopPlayBack NetClient_StopPlayBack = NULL;
```

```
pfNetClient_SetNotifyFunction_V4 NetClient_SetNotifyFunction_V4 = NULL;
```

```
pfNetClient_Cleanup NetClient_Cleanup = NULL;
```

```
pfNetClient_NetFileQuery NetClient_NetFileQuery = NULL;
```

```
pfNetClient_NetFileGetFileCount NetClient_NetFileGetFileCount = NULL;
```

```
pfNetClient_NetFileGetQueryfile NetClient_NetFileGetQueryfile = NULL;
```

```
pfNetClient_NetFileDownloadFile NetClient_NetFileDownloadFile = NULL;
```

```
pfNetClient_SetNetFileDownloadFileCallBack NetClient_SetNetFileDownloadFileCallBack =  
NULL;
```

```
pfNetClient_VCASetConfig NetClient_VCASetConfig = NULL;
```

```
pfNetClient_VCAGetConfig NetClient_VCAGetConfig = NULL;
```

```
pfNetClient_VCARestartEx NetClient_VCARestartEx = NULL;
```

```
pfNetClient_VCAGetAlarmInfo NetClient_VCAGetAlarmInfo = NULL;
```

```
pfTC_CreateSystem TC_CreateSystem = NULL;
```

```
pfTC_DeleteSystem TC_DeleteSystem = NULL;
```

```
pfTC_RegisterCommonEventCallBack_V4 TC_RegisterCommonEventCallBack_V4 = NULL;
```

```
pfTC_CreatePlayerFromFile TC_CreatePlayerFromFile = NULL;
```

```
pfTC_DeletePlayer TC_DeletePlayer = NULL;
```

```
pfTC_Play TC_Play = NULL;
```

```
pfTC_Stop TC_Stop = NULL;
```

```
pfTC_CreatePlayerFromVoD TC_CreatePlayerFromVoD = NULL;
```

```
pfTC_PutStreamToPlayer TC_PutStreamToPlayer = NULL;
```

```
pfTC_GetStreamPlayBufferState TC_GetStreamPlayBufferState = NULL;
```

```
pfNetClient_NetFileDownloadByTimeSpanEx NetClient_NetFileDownloadByTimeSpanEx =  
NULL;
```

```
pfNetClient_GetDigitalChannelConfig NetClient_GetDigitalChannelConfig = NULL;
```

```
pfNetClient_SetDigitalChannelConfig NetClient_SetDigitalChannelConfig = NULL;
```

```

pfNetClient_GetChannelProperty NetClient_GetChannelProperty = NULL;
pfNetClient_TalkStart NetClient_TalkStart = NULL;
pfNetClient_TalkEnd NetClient_TalkEnd = NULL;
pfNetClient_InterTalkStart NetClient_InterTalkStart = NULL;
pfNetClient_InterTalkEnd NetClient_InterTalkEnd = NULL;
pfNetClient_AudioStart NetClient_AudioStart = NULL;
pfNetClient_StopCaptureData NetClient_StopCaptureData = NULL;
pfNetClient_StartCaptureData NetClient_StartCaptureData = NULL;
pfNetClient_SetRawFrameCallBack NetClient_SetRawFrameCallBack = NULL;
pfNetClient_SetDataPackCallBack NetClient_SetDataPackCallBack = NULL;
pfNetClient_LogonEx NetClient_LogonEx = NULL;
pfNetClient_GetDeviceType NetClient_GetDeviceType = NULL;
pfNetClient_ComSend NetClient_ComSend = NULL;
pfNetClient_NetLogQuery NetClient_NetLogQuery = NULL;
pfNetClient_NetLogGetLogCount NetClient_NetLogGetLogCount = NULL;
pfNetClient_NetLogGetLogfile NetClient_NetLogGetLogfile = NULL;
pNetClient_GetVideoPara NetClient_GetVideoPara = NULL;
pNetClient_SetVideoPara NetClient_SetVideoPara = NULL;
pNetClient_SetDevConfig NetClient_SetDevConfig = NULL;
pNetClient_GetDevConfig NetClient_GetDevConfig = NULL;
pNetClient_SetMotionDecLinkOutport NetClient_SetMotionDecLinkOutport = NULL;
pNetClient_SetMotionDetectSchedule NetClient_SetMotionDetectSchedule = NULL;
pNetClient_SetThreshold NetClient_SetThreshold = NULL;
pNetClient_SetMotionDetection NetClient_SetMotionDetection = NULL;
pNetClient_GetMotionDecLinkOutport NetClient_GetMotionDecLinkOutport = NULL;
pNetClient_GetMotionDetectSchedule NetClient_GetMotionDetectSchedule = NULL;
pNetClient_GetThreshold NetClient_GetThreshold = NULL;
pNetClient_GetDsmRegstierInfo NetClient_GetDsmRegstierInfo = NULL;

int LoadNVSSDK()
{
    if (g_hNVSSDK)
        return -2;
    else
    {
        g_hNVSSDK = LoadLibrary("NVSSDK.dll");
        if(!g_hNVSSDK)
            return -1;
    }

    NetClient_Startup_V4 = (pfNetClient_Startup_V4)GetProcAddress(g_hNVSSDK,
"NetClient_Startup_V4");
    NetClient_Logon = (pfNetClient_Logon)GetProcAddress(g_hNVSSDK,
"NetClient_Logon");

```



```

NetClient_Logon_V4      =      (pfNetClient_Logon_V4)GetProcAddress(g_hNVSSDK,
"NetClient_Logon_V4");
NetClient_Logoff        =      (pfNetClient_Logoff)GetProcAddress(g_hNVSSDK,
"NetClient_Logoff");
NetClient_GetChannelNum = (pfNetClient_GetChannelNum)GetProcAddress(g_hNVSSDK,
"NetClient_GetChannelNum");
NetClient_StartPlay     =      (pfNetClient_StartPlay)GetProcAddress(g_hNVSSDK,
"NetClient_StartPlay");
NetClient_StartRecv     =      (pfNetClient_StartRecv)GetProcAddress(g_hNVSSDK,
"NetClient_StartRecv");
NetClient_StartRecv_V4  =      (pfNetClient_StartRecv_V4)GetProcAddress(g_hNVSSDK,
"NetClient_StartRecv_V4");
NetClient_StopRecv      =      (pfNetClient_StopRecv)GetProcAddress(g_hNVSSDK,
"NetClient_StopRecv");
NetClient_StopPlay      =      (pfNetClient_StopPlay)GetProcAddress(g_hNVSSDK,
"NetClient_StopPlay");
NetClient_PlayBack      =      (pfNetClient_PlayBack)GetProcAddress(g_hNVSSDK,
"NetClient_PlayBack");
NetClient_StopPlayBack  =      (pfNetClient_StopPlayBack)GetProcAddress(g_hNVSSDK,
"NetClient_StopPlayBack");
NetClient_SetNotifyFunction_V4      =
(pfNetClient_SetNotifyFunction_V4)GetProcAddress(g_hNVSSDK,
"NetClient_SetNotifyFunction_V4");
NetClient_Cleanup       =      (pfNetClient_Cleanup)GetProcAddress(g_hNVSSDK,
"NetClient_Cleanup");
NetClient_NetFileQuery  =      (pfNetClient_NetFileQuery)GetProcAddress(g_hNVSSDK,
"NetClient_NetFileQuery");
NetClient_NetFileGetFileCount      =
(pfNetClient_NetFileGetFileCount)GetProcAddress(g_hNVSSDK,
"NetClient_NetFileGetFileCount");
NetClient_NetFileGetQueryfile      =
(pfNetClient_NetFileGetQueryfile)GetProcAddress(g_hNVSSDK,
"NetClient_NetFileGetQueryfile");
NetClient_NetFileDownloadFile      =
(pfNetClient_NetFileDownloadFile)GetProcAddress(g_hNVSSDK,
"NetClient_NetFileDownloadFile");
NetClient_SetNetFileDownloadFileCallBack      =
(pfNetClient_SetNetFileDownloadFileCallBack)GetProcAddress(g_hNVSSDK,
"NetClient_SetNetFileDownloadFileCallBack");
NetClient_VCASetConfig      =
(pfNetClient_VCASetConfig)GetProcAddress(g_hNVSSDK, "NetClient_VCASetConfig");
NetClient_VCAGetConfig      =
(pfNetClient_VCAGetConfig)GetProcAddress(g_hNVSSDK, "NetClient_VCAGetConfig");
NetClient_VCAREstartEx      =

```

```

(pfNetClient_VCARestartEx)GetProcAddress(g_hNVSSDK,"NetClient_VCARestartEx");
    NetClient_VCAGetAlarmInfo =
(pfNetClient_VCAGetAlarmInfo)GetProcAddress(g_hNVSSDK,"NetClient_VCAGetAlarmInfo"
);
    NetClient_NetFileDownloadByTimeSpanEx =
(pfNetClient_NetFileDownloadByTimeSpanEx)GetProcAddress(g_hNVSSDK,"NetClient_NetFil
eDownloadByTimeSpanEx");
    NetClient_GetDigitalChannelConfig =
(pfNetClient_GetDigitalChannelConfig)GetProcAddress(g_hNVSSDK,"NetClient_GetDigitalCha
nnelConfig");
    NetClient_SetDigitalChannelConfig =
(pfNetClient_SetDigitalChannelConfig)GetProcAddress(g_hNVSSDK,"NetClient_SetDigitalCha
nnelConfig");
    NetClient_GetChannelProperty =
(pfNetClient_GetChannelProperty)GetProcAddress(g_hNVSSDK,"NetClient_GetChannelPropert
y");
    NetClient_TalkStart =
(pfNetClient_TalkStart)GetProcAddress(g_hNVSSDK,"NetClient_TalkStart");
    NetClient_TalkEnd =
(pfNetClient_TalkEnd)GetProcAddress(g_hNVSSDK,"NetClient_TalkEnd");
    NetClient_InterTalkStart =
(pfNetClient_InterTalkStart)GetProcAddress(g_hNVSSDK,"NetClient_InterTalkStart");
    NetClient_InterTalkEnd =
(pfNetClient_InterTalkEnd)GetProcAddress(g_hNVSSDK,"NetClient_InterTalkEnd");
    NetClient_AudioStart =
(pfNetClient_AudioStart)GetProcAddress(g_hNVSSDK,"NetClient_AudioStart");
    NetClient_StopCaptureData =
(pfNetClient_StopCaptureData)GetProcAddress(g_hNVSSDK,"NetClient_StopCaptureData");
    NetClient_StartCaptureData =
(pfNetClient_StartCaptureData)GetProcAddress(g_hNVSSDK,"NetClient_StartCaptureData");
    NetClient_SetRawFrameCallBack =
(pfNetClient_SetRawFrameCallBack)GetProcAddress(g_hNVSSDK,"NetClient_SetRawFrameCa
llBack");
    NetClient_SetDataPackCallBack =
(pfNetClient_SetDataPackCallBack)GetProcAddress(g_hNVSSDK,"NetClient_SetDataPackCall
Back");
    NetClient_LogonEx =
(pfNetClient_LogonEx)GetProcAddress(g_hNVSSDK,"NetClient_LogonEx");
    NetClient_ComSend =
(pfNetClient_ComSend)GetProcAddress(g_hNVSSDK,"NetClient_ComSend");
    NetClient_GetDeviceType =
(pfNetClient_GetDeviceType)GetProcAddress(g_hNVSSDK,"NetClient_GetDeviceType");
    NetClient_NetLogQuery = (pfNetClient_NetLogQuery)GetProcAddress(g_hNVSSDK,
"NetClient_NetLogQuery");

```

```

        NetClient_NetLogGetLogCount =
(pfNetClient_NetLogGetLogCount)GetProcAddress(g_hNVSSDK,
"NetClient_NetLogGetLogCount");
        NetClient_NetLogGetLogfile =
(pfNetClient_NetLogGetLogfile)GetProcAddress(g_hNVSSDK, "NetClient_NetLogGetLogfile");
        NetClient_GetVideoPara = (pNetClient_GetVideoPara)GetProcAddress(g_hNVSSDK,
"NetClient_GetVideoPara");
        NetClient_SetVideoPara = (pNetClient_SetVideoPara)GetProcAddress(g_hNVSSDK,
"NetClient_SetVideoPara");
        NetClient_SetDevConfig = (pNetClient_SetDevConfig)GetProcAddress(g_hNVSSDK,
"NetClient_SetDevConfig");
        NetClient_GetDevConfig = (pNetClient_GetDevConfig)GetProcAddress(g_hNVSSDK,
"NetClient_GetDevConfig");
        NetClient_SetMotionDetection =
(pNetClient_SetMotionDetection)GetProcAddress(g_hNVSSDK,
"NetClient_SetMotionDetection");
        NetClient_SetThreshold = (pNetClient_SetThreshold)GetProcAddress(g_hNVSSDK,
"NetClient_SetThreshold");
        NetClient_SetMotionDetectSchedule =
(pNetClient_SetMotionDetectSchedule)GetProcAddress(g_hNVSSDK,
"NetClient_SetMotionDetectSchedule");
        NetClient_SetMotionDecLinkOutport =
(pNetClient_SetMotionDecLinkOutport)GetProcAddress(g_hNVSSDK,
"NetClient_SetMotionDecLinkOutport");
        NetClient_GetThreshold = (pNetClient_GetThreshold)GetProcAddress(g_hNVSSDK,
"NetClient_GetThreshold");
        NetClient_GetMotionDecLinkOutport =
(pNetClient_GetMotionDecLinkOutport)GetProcAddress(g_hNVSSDK,
"NetClient_GetMotionDecLinkOutport");
        NetClient_GetMotionDetectSchedule =
(pNetClient_GetMotionDetectSchedule)GetProcAddress(g_hNVSSDK,
"NetClient_GetMotionDetectSchedule");
        NetClient_GetDsmRegstierInfo =
(pNetClient_GetDsmRegstierInfo)GetProcAddress(g_hNVSSDK,
"NetClient_GetDsmRegstierInfo");
        return 0;
    }

void FreeNVSSDK()
{
    if(g_hNVSSDK)
    {
        ::FreeLibrary(g_hNVSSDK);
        g_hNVSSDK = NULL;
    }
}

```

```

    }
}

int LoadPlaySdk()
{
    if (g_hPlaySdk)
        return -2;
    else
    {
        g_hPlaySdk = LoadLibrary("PlaySdkM4.dll");
        if(!g_hPlaySdk)
            return -1;
    }

    TC_CreateSystem = (pfTC_CreateSystem)GetProcAddress(g_hPlaySdk,
"TC_CreateSystem");
    TC_DeleteSystem = (pfTC_DeleteSystem)GetProcAddress(g_hPlaySdk,
"TC_DeleteSystem");
    TC_RegisterCommonEventCallBack_V4 =
(pfTC_RegisterCommonEventCallBack_V4)GetProcAddress(g_hPlaySdk,
"TC_RegisterCommonEventCallBack_V4");
    TC_CreatePlayerFromFile = (pfTC_CreatePlayerFromFile)GetProcAddress(g_hPlaySdk,
"TC_CreatePlayerFromFile");
    TC_DeletePlayer = (pfTC_DeletePlayer)GetProcAddress(g_hPlaySdk, "TC_DeletePlayer");
    TC_Play = (pfTC_Play)GetProcAddress(g_hPlaySdk, "TC_Play");
    TC_Stop = (pfTC_Stop)GetProcAddress(g_hPlaySdk, "TC_Stop");
    TC_CreatePlayerFromVoD =
(pfTC_CreatePlayerFromVoD)GetProcAddress(g_hPlaySdk, "TC_CreatePlayerFromVoD");
    TC_PutStreamToPlayer =
(pfTC_PutStreamToPlayer)GetProcAddress(g_hPlaySdk, "TC_PutStreamToPlayer");
    TC_GetStreamPlayBufferState =
(pfTC_GetStreamPlayBufferState)GetProcAddress(g_hPlaySdk, "TC_GetStreamPlayBufferState")
;

    return 0;
}

void FreePlaySdk()
{
    if(g_hPlaySdk)
    {
        ::FreeLibrary(g_hPlaySdk);
        g_hPlaySdk = NULL;
    }
}

```

```
}
```

4.1 主动模式（不带目录服务器）

```
#include "NetSDK.h"
#include <stdio.h>

int g_iLogonID = -1;

void Notify_Main(int _iLogonID, long _iWparam, void* _iLParam, void* _iUserData)
{
    int iMsgType = LOWORD(_iWparam);
    switch (iMsgType)
    {
        case WCM_LOGON_NOTIFY:
        {
            printf("WCM_LOGON_NOTIFY\n");
            if(_iLParam == LOGON_SUCCESS)
            {
                printf("Logon success!\n");
            }
            else
            {
                printf("Logon failed!\n");
            }
        }
        break;
        default:
            break;
    }
}

int main(int argc, char *argv[])
{
    LoadNVSSDK(); //初始化接口库
    NetClient_Startup_V4(6004, 0, 0); //初始化 SDK。主动模式通信端口，取值范围(70-65535)
    可随意设置，只要在设备的网络服务-注册中心中设置向该端口注册即可。
    NetClient_SetNotifyFunction_V4(Notify_Main, NULL, NULL, NULL, NULL); //设置回调
    函数
    int iRet = -1;
    LogonActiveServer tLogonActiveServer = {0};
    //根据设备出厂 ID 获取在线状态
    DsmOnline tDsmOnline = {0};
```

```

    tDsmOnline.iSize = sizeof(DsmOnline);
    strcpy_s(tDsmOnline.cProductID, sizeof(tDsmOnline.cProductID), "ID0000801940400180480380");
    iRet = NetClient_GetDsmRegstierInfo(DSM_CMD_GET_ONLINE_STATE, &tDsmOnline,
    sizeof(DsmOnline));
    if (0 == iRet)
    {
        printf("NetClient_GetDsmRegstierInfo success!\n");
    }
    else
    {
        printf("NetClient_GetDsmRegstierInfo failed!\n");
        goto END;
    }

    //登录设备
    tLogonActiveServer.iSize = sizeof(LogonActiveServer);
    strcpy_s(tLogonActiveServer.cUserName, sizeof(tLogonActiveServer.cUserName),
    "admin");
    strcpy_s(tLogonActiveServer.cUserPwd, sizeof(tLogonActiveServer.cUserPwd), "1111");
    strcpy_s(tLogonActiveServer.cProductID, sizeof(tLogonActiveServer.cProductID),
    "ID0000801940400180480380");
    g_iLogonID = NetClient_Logon_V4(SERVER_ACTIVE, &tLogonActiveServer,
    sizeof(LogonActiveServer));
    if(g_iLogonID < 0)
    {
        printf("Logon failed!\n");
    }
    getchar();//等待用户输入
    NetClient_Logoff(g_iLogonID);//注销用户
    NetClient_Cleanup();//释放 SDK 资源
    FreeNVSSDK();//释放接口库
END:
    return iRet;
}

```

4.2 连接视频

(1) 以下程序为控制台程序，VC 环境下编译运行

```

#include "NetSDK.h"
#include <stdio.h>

int g_iLogonID = -1;

```

```
unsigned int g_uConnID = -1;

unsigned int StartRecv(int _iLogonID)
{
    unsigned int uConnID = -1;
    CLIENTINFO clientinfo = {0};
    clientinfo.m_iNetMode = NETMODE_TCP;
    clientinfo.m_iServerID = _iLogonID;
    clientinfo.m_iChannelNo = 0;//预览通道号
    clientinfo.m_iStreamNO = MAIN_STREAM;

    int iRet = NetClient_StartRecv_V4(&uConnID,&clientinfo,NULL, NULL);//建立视频连接
    if (iRet == 0)
    {
        printf("StartRecv success!\n");
    }
    else
    {
        printf("StartRecv failed!\n");
    }

    return uConnID;
}

void StartPlay(unsigned int _uConnID)
{
    RECT rc = {0};
    HWND hWnd = GetConsoleWindow();//获得窗口句柄
    NetClient_StopPlay(_uConnID);//停止播放视频
    int iRet = NetClient_StartPlay(_uConnID, hWnd, rc, H264DEC_DECTWO);//开始播放视频
    if(iRet >= 0)
    {
        printf("StartPlay success!\n");
    }
    else
    {
        printf("StartPlay failed!\n");
    }
}

void Notify_Main(int _iLogonID, long _iWparam, void* _iLParam,void* _iUser)
{
    int iMsgType = LOWORD(_iWparam);
    switch (iMsgType)
```

```

{
case WCM_LOGON_NOTIFY:
    {
        printf("WCM_LOGON_NOTIFY\n");
        if(_iLParam == LOGON_SUCCESS)
        {
            printf("Logon success!\n");
            if(-1 == g_uConnID)
            {
                g_uConnID = StartRecv(_iLogonID);//连接视频
            }
        }
        else
        {
            printf("Logon failed!\n");
        }
    }
    break;
case WCM_VIDEO_HEAD:
    {
        printf("WCM_VIDEO_HEAD\n");
        StartPlay(g_uConnID);//播放视频
    }
    break;
default:
    break;
}
}

int main(int argc,char *argv[])
{
    LoadNVSSDK();//初始化接口库
    NetClient_Startup_V4(0, 0, 0);//初始化 SDK
    NetClient_SetNotifyFunction_V4(Notify_Main, NULL, NULL, NULL, NULL);//设置回调函数
    //登录设备
    LogonPara tInfo = {0};
    tInfo.iSize = sizeof(LogonPara);
    tInfo.iNvsPort = 3000;

    fprintf(stderr, "Please input server IP: ");
    gets_s(tInfo.cNvsIP, 32);
    if (0 == strlen(tInfo.cNvsIP)) {
        strncpy(tInfo.cNvsIP, "192.168.1.2", sizeof(tInfo.cNvsIP));
    }
}

```



```
}

fprintf(stderr, "Please input user name: ");
gets_s(tInfo.cUserName, 16);
if (0 == strlen(tInfo.cUserName)) {
    strncpy(tInfo.cUserName, "Admin", sizeof(tInfo.cUserName));
}
fprintf(stderr, "Please input password: ");
gets_s(tInfo.cUserPwd, 16);
if (0 == strlen(tInfo.cUserPwd)) {
    strncpy(tInfo.cUserPwd, "Admin", sizeof(tInfo.cUserPwd));
}
fprintf(stderr, "Please input port: ");
char cPort[16] = {0};
gets_s(cPort, 16);
tInfo.iNvsPort = atoi(cPort);
if (tInfo.iNvsPort <= 0) {
    tInfo.iNvsPort = 3000;
}
g_iLogonID = NetClient_Logon_V4(SERVER_NORMAL, &tInfo, tInfo.iSize);
if(g_iLogonID < 0)
{
    printf("Logon failed!\n");
}
getchar();//等待用户输入
NetClient_Logoff(g_iLogonID);//注销用户
NetClient_Cleanup();//释放 SDK 资源
FreeNVSSDK();//释放接口库
return 0;
}
```

4.3 下载录像

说明：本流程用于远程下载 DVR 设备录像文件，远程文件下载有两种方式，一种是按录像文件下载，一种按时间段下载。以下控制台程序基于 VC 环境可直接运行

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include "NetSdk.h"

#ifdef __WIN__
#include <process.h>
#else
#include <pthread.h>
#include <unistd.h>
#include <dlfcn.h>
#endif

int g_iLogonID = -1;
unsigned int g_uiConnID = -1;//连接 ID
int g_iPos = 0;
int g_iSize = 0;
int g_iFileNum = -1;
int g_iEndThread = 0;
int g_iCurrentCount = -1;
NVS_FILE_DATA g_NvsFileInfo[100] = {{0}};
DOWNLOAD_FILE g_DownloadFile[100] = {{0}};
int LogonDevice(int _iLogonType);
void PrintFileInfo();
void QueryFile();
void DwonloadByTime();
int DownloadFileByFile( int _iFileNum,int _iFlag, int _iDownloadPos , int _iDownloadSpeed );
void __stdcall RecvRawFramNotify(unsigned int _uiID, unsigned char* _pucData,int _iLen,
RAWFRAME_INFO* _ptRawFrameInfo, void* _pvUser);

int strncpy_ss(char *_pcDest, char *_pcSrc, int _iSize)
{
    if (NULL == _pcDest || NULL == _pcSrc)
    {
        return -1;
    }
}
```

```
    if(_iSize < 0)
    {
        strcpy((char*)_pcDest,_pcSrc);
    }
    else
    {
        if(strlen(_pcSrc) >= (size_t)(_iSize - 1))
        {
            memcpy((char*)_pcDest, _pcSrc, _iSize - 1);
            memset((char*)_pcDest + _iSize - 1, 0, 1);
        }
        else
        {
            strcpy((char*)_pcDest,_pcSrc);
        }
    }
    return 0;
}

#ifdef __WIN__
void WinGetDownloadProcessThread( void * _p)
{
    while(1)
    {
        if( 1 == g_iEndThread )
        {
            return;
        }
        if( -1 != g_uiConnID)
        {
            int iRet = NetClient_NetFileGetDownloadPos(g_uiConnID, &g_iPos, &g_iSize);
            if( 0 == iRet)
            {
                printf("Position = %d, Size = %d \n", g_iPos, g_iSize);
            }
            else
            {
                printf("Err : NetClient_NetFileGetDownloadPos\n");
                return ;
            }
        }
        Sleep(1000);
    }
}
```

```
void usleep(int _iMicroSecond)
{
    if (_iMicroSecond < 1000 && _iMicroSecond != 0)
    {
        _iMicroSecond = 1000;
    }
    Sleep(_iMicroSecond/1000);
}
#else
void *LinuxGetDownloadProcessThread( void * _p)
{
    while(1)
    {
        if( 1 == g_iEndThread )
        {
            return((void*)0);
        }
        if( -1 != (int)g_uiConnID)
        {
            int iRet = NetClient_NetFileGetDownloadPos(g_uiConnID, &g_iPos, &g_iSize);
            if( 0 == iRet)
            {
                printf("Position = %d%%, Size = %d \n", g_iPos, g_iSize);
            }
            else
            {
                printf("Err : NetClient_NetFileGetDownloadPos\n");
                return((void*)0);
            }
        }
        sleep(1);
    }
    return((void*)0);
}

int gets_s(char *_pcStr, int _iCount)
{
    if (_pcStr == NULL)
        return -1;
    char* pcRet = fgets(_pcStr, _iCount, stdin);
    size_t uiLen = strlen(_pcStr);
    if (pcRet == NULL || uiLen == 0)
        return -2;
    if (_iCount - 1 > (int)uiLen || _pcStr[uiLen-1] == '\n')
```

```
{
    _pcStr[uiLen-1] = '\0';
}
//stdin->_IO_read_ptr = stdin->_IO_read_end;
return 0;
}
#endif

void Notify_Main(int _iLogonID, long _lWparam, void* _pvLParam, void* _pvUsr)
{
    int lMsgType = LOWORD(_lWparam);
    long iMsgValue = (long)_pvLParam;
    switch (lMsgType)
    {
    case WCM_LOGON_NOTIFY:
        {
            printf("[Notify_Main]->WCM_LOGON_NOTIFY\n");
            if((LOGON_SUCCESS == iMsgValue) )
            {
                printf("[Notify_Main]->Success : Logon\n");
            }
            else
            {
                printf("[Notify_Main]->Err = %d : Logon, Please Retry\n", iMsgValue);
                return;
            }
        }
        break;
    case WCM_QUERYFILE_FINISHED:
        {
            printf("[Notify_Main]->WCM_QUERYFILE_FINISHED\n");
            int iTotalCount = 0;
            NetClient_NetFileGetFileCount(_iLogonID, &iTotalCount, &g_iCurrentCount);
            printf("[Notify_Main]->TotalCount(%d),CurrentCount(%d)\n",iTotalCount,
                g_iCurrentCount);
            if( (g_iCurrentCount == 0) || (iTotalCount == 0) )
            {
                printf( "[Notify_Main]->Err : Find No File\n");
                break;
            }
            printf("[Notify_Main]-><Press Any Key To Get File Info>\n");
        }
        break;
    case WCM_DWONLOAD_FINISHED:
```

```

        {
            g_iEndThread = 1;
            g_iCurrentCount = -1;
            printf("[Notify_Main]->WCM_DWONLOAD_FINISHED\n");
            printf("[Notify_Main]-><Download Position Is = 100%% >\n");
        }
        break;
case WCM_DWONLOAD_FAULT:
    {
        g_iEndThread = 1;
        g_iCurrentCount = -1;
        printf("[Notify_Main]->WCM_DWONLOAD_FAULT\n");
    }
    break;
case WCM_DOWNLOAD_INTERRUPT:
    {
        g_iEndThread = 1;
        g_iCurrentCount = -1;
        printf("[Notify_Main]->WCM_DOWNLOAD_INTERRUPT\n");
    }
    break;
default:
    break;
}
}

//原始流回调函数
void __stdcall GetRawNotify( unsigned int _ulID,unsigned char* _cData,int _iLen,
RAWFRAME_INFO *_pRawFrameInfo, void* _iUser )
{
    if( -1 == (int)_ulID)
    {
        return;
    }
    if( NULL != _pRawFrameInfo)
    {
        //printf("Log : Raw Stream Type = %d, Length = %d\n", _pRawFrameInfo->nType,
        _iLen);
    }
}

int to_int_def(const char* _pstrFrom, int _iDef)
{
    if(NULL == _pstrFrom)

```

```
{
    return _iDef;
}
for (size_t i=0; i<strlen(_pstrFrom); i++)
{
    if (!isdigit(_pstrFrom[i]))
    {
        if(i == 0 && _pstrFrom[i] == '-' && strlen(_pstrFrom) > 1)
        {
            continue;
        }
        return _iDef;
    }
}
return atoi(_pstrFrom);
}

int main(int argc,char *argv[])
{
    char cCmd[16] = {0};
    int iLogonType = SERVER_NORMAL;
    char cTemp[8] = {0};
    int iRet = LoadNVSSDK();
    if (0 != iRet)
    {
        printf("Err : LoadNVSSDK\n");
        return -1;
    }
    fprintf(stderr, "Please input LogonType: 0----Normal   1----Active\n");
    gets_s(cTemp, 8);
    iLogonType = to_int_def(cTemp, 0);
    if (SERVER_ACTIVE == iLogonType)
    {
        fprintf(stderr, "Please input listening port:");
        gets_s(cTemp, 8);
        int iLlisteningPort = to_int_def(cTemp, 0);
        iRet = NetClient_Startup_V4(iLlisteningPort, 0, 0);//初始化 SDK
    }
    else
    {
        iRet = NetClient_Startup_V4(0, 0, 0);//初始化 SDK
    }
    if (0 == iRet )
    {

```

```
        printf("Success : NetClient_Startup_V4()\n");
    }
    else
    {
        printf("Err : NetClient_Startup_V4()\n");
        return -1;
    }
    NetClient_SetNotifyFunction_V4(Notify_Main, NULL, NULL, NULL, NULL);
    //登录设备
    iRet = LogonDevice(iLogonType);
    if( 0 == iRet )
    {
        printf("Success : LogonDevice()\n");
    }
    else
    {
        printf("Err : LogonDevice()\n");
    }
    while(1)
    {
        if( g_iCurrentCount > 0 )
        {
            if(-1 != g_iLogonID)
            {
                PrintFileInfo();
            }
        }
        printf("<Please Input Download Mode[f=File, t=Time, s=Stop, q=Exit]>:\n");
        gets_s( cCmd, 16);
        //退出
        if( 0 == strcmp(cCmd, "q"))
        {
            if( -1 != g_iLogonID)
            {
                NetClient_Logoff(g_iLogonID);//注销用户
            }
            NetClient_Cleanup();//释放 SDK 资源
            FreeNVSSDK();//释放接口库
            return 0;
        }
        //文件模式
        else if( 0 == strcmp(cCmd, "f"))
        {
            if(-1 != g_iLogonID)
```



```
        {
            QueryFile();
        }
    }
    //时间段模式
    else if( 0 == strcmp(cCmd, "t"))
    {
        if(-1 != g_iLogonID)
        {
            DwonloadByTime();
        }
    }
    else if (0 == strcmp(cCmd, "s"))
    {
        g_iCurrentCount = 0;
        if( -1 != (int)g_uiConnID)
        {
            int iRet = NetClient_NetFileStopDownloadFile( g_uiConnID);
            if( 0 == iRet)
            {
                printf("Success : NetClient_NetFileStopDownloadFile\n");
            }
            else
            {
                g_uiConnID = -1;
                printf("Err : NetClient_NetFileStopDownloadFile\n");
            }
        }
    }
}

getchar();
return 0;
}

int LogonDevice( int _iLogonType )
{
    char cIP[16] = {0};
    char cUserName[16] = {0};
    char cPassword[16] = {0};
    char cProductID[32] = {0};

    fprintf(stderr, "Please input user name: ");
    gets_s(cUserName, 16);
    fprintf(stderr, "Please input password: ");
```

```
gets_s(cPassword, 16);

LogonPara tNormal = {0};
LogonActiveServer tActive = {0};
void* pvPara = NULL;
int iBufLen = 0;
if (SERVER_ACTIVE == _iLogonType)
{
    fprintf(stderr, "Please input ProductID: ");
    gets_s(cProductID, 32);
    tActive.iSize = sizeof(LogonActiveServer);
    strcpy(tActive.cUserName, cUserName);
    strcpy(tActive.cUserPwd, cPassword);
    strcpy(tActive.cProductID, cProductID);
    pvPara = &tActive;
    iBufLen = sizeof(LogonActiveServer);
    DsmOnline tOnline = {0};
    tOnline.iSize = sizeof(DsmOnline);
    strncpy(tOnline.cProductID, cProductID, LEN_32);
    NetClient_GetDsmRegstierInfo(DSM_CMD_GET_ONLINE_STATE, &tOnline,
    sizeof(DsmOnline));
    int iOutTime = 0;
    while (DSM_STATE_ONLINE != tOnline.iOnline)
    {
        if (iOutTime >= 20)
        {
            fprintf(stderr, "Device not register!\n");
            break;
        }
        usleep(1000 * 1000);
        NetClient_GetDsmRegstierInfo(DSM_CMD_GET_ONLINE_STATE, &tOnline,
        sizeof(DsmOnline));
        iOutTime++;
    }
}
else
{
    fprintf(stderr, "Please input server IP: ");
    gets_s(cIP, 16);
    tNormal.iSize = sizeof(LogonPara);
    tNormal.iNvsPort = 3000;
    strcpy(tNormal.cNvsIP, cIP);
    strcpy(tNormal.cUserName, cUserName);
    strcpy(tNormal.cUserPwd, cPassword);
```

```
        strcpy(tNormal.cCharSet, "UTF-8");
        pvPara = &tNormal;
        iBufLen = sizeof(LogonPara);
    }

    g_iLogonID = NetClient_Logon_V4(_iLogonType, pvPara, iBufLen);
    if(g_iLogonID < 0)
    {
        fprintf(stderr, "[NetClient_Logon_V4] fail! %d\n", g_iLogonID);
    }
    else
    {
        fprintf(stderr, "[NetClient_Logon_V4] Success! %d\n", g_iLogonID);
    }
    return g_iLogonID;
}

void PrintFileInfo()
{
    NVS_FILE_DATA fileInfo = {0};
    for(int i = 0; i < g_iCurrentCount; ++i)
    {
        int iRet = NetClient_NetFileGetQueryfile(g_iLogonID, i, &fileInfo); //获取文件信息
        if(0 == iRet)
        {
            printf("File[%d]:Name(%s),Type(%d),Channel(%d),Size(%d)\n", i, fileInfo.cFileName, fileInfo.iType, fileInfo.iChannel, fileInfo.iFileSize);
            g_NvsFileInfo[i] = fileInfo;
        }
        else
        {
            printf("Err : NetFileGetMultiChanQueryFilefile\n");
            break;
        }
    }
    printf("<Please Choose The File[x] Number To Download>, Press [Enter] Will Download File[0], Press 'q' To Exit.");
    char cTemp[6] = {0};
    gets_s(cTemp, 6);
    if( NULL != cTemp )
    {
        if( 0 == strcmp("q", cTemp))
        {
            g_iCurrentCount = -1;
        }
    }
}
```

```

        return;
    }
    g_iFileNum = atoi(cTemp);
    memset(cTemp, 0, 6);
    if( g_iFileNum < 0 || g_iFileNum > 19)
    {
        printf("<Please Input Correct File Number>\n");
        return;
    }
    else
    {
        DownloadFileByFile( g_iFileNum , 0, -1, 8);
    }
}

#ifdef __WIN__
    _beginthread( WinGetDownloadProcessThread, 0, NULL);
#else
    pthread_t t;
    pthread_create(&t, NULL, LinuxGetDownloadProcessThread, NULL);
#endif
}
}

int DownloadFileByFile( int _iFileNum,int _iFlag, int _iDownloadPos , int _iDownloadSpeed )
{
    if( -1 == _iFileNum)
    {
        return -1;
    }
    Int iRet = -1;
    char cLocalFileName[256] = {0};
    DOWNLOAD_FILE downloadFile = {0};
    strncpy_ss(cLocalFileName,g_NvsFileInfo[_iFileNum].cFileName,strlen(g_NvsFileInfo[_iFileNum].cFileName)-3);
    printf("Input 0 = SDV; 1 = PS; 2 = MP4");
    char cTemp[6] = {0};
    gets_s(cTemp, 6);
    if( 0 == strcmp("0", cTemp))
    {
        strcat(cLocalFileName, ".sdv");
        downloadFile.m_iSaveFileType = DOWNLOAD_FILE_TYPE_SDV;
    }
    else if(0 == strcmp("1", cTemp))
    {

```

```
        strcat(cLocalFileName, ".ps");
        downloadFile.m_iSaveFileType = DOWNLOAD_FILE_TYPE_PS;
    }
    else if(0 == strcmp("2", cTemp))
    {
        strcat(cLocalFileName, ".mp4");
        downloadFile.m_iSaveFileType = DOWNLOAD_FILE_TYPE_MP4;
    }
    else
    {
        printf("Please Input Correctly!");
        return -1;
    }
    memset(cTemp, 0 , 6);
    downloadFile.m_iSize = sizeof(DOWNLOAD_FILE);
    downloadFile.m_iPosition = -1;
    strcpy(downloadFile.m_cLocalFilename ,cLocalFileName);
    strcpy(downloadFile.m_cRemoteFilename ,g_NvsFileInfo[_iFileNum].cFileName);
    downloadFile.m_iSpeed = 32;

    iRet=NetClient_NetFileDownload(&g_uiConnID,g_iLogonID,DOWNLOAD_CMD_FILE,
    &downloadFile, sizeof(DOWNLOAD_FILE));
    if( 0 == iRet)
    {
        printf("Success : NetClient_NetFileDownload\n");
        if( -1 != (int)g_uiConnID)
        {
            //设置原始流回调
            iRet = NetClient_SetRawFrameCallBack(g_uiConnID, GetRawNotify, NULL);
            if( iRet < 0 )
            {
                printf("Err : NetClient_SetRawFrameCallBack\n");
                return -1;
            }
        }
    }
    else
    {
        printf("Err : NetClient_NetFileDownload\n");
        return -1;
    }
    return g_uiConnID;
}
```

```
void QueryFile()
{
    Int iRet = -1;
    NETFILE_QUERY_V5 tMultiChanQueryFile = {0};
    tMultiChanQueryFile.iType = 0xFF;
    tMultiChanQueryFile.iQueryChannelNo = 0;
    tMultiChanQueryFile.iStreamNo = 0;
    tMultiChanQueryFile.tStartTime.iYear = 2019;
    tMultiChanQueryFile.tStartTime.iMonth = 7;
    tMultiChanQueryFile.tStartTime.iDay = 21;
    tMultiChanQueryFile.tStartTime.iHour = 0;
    tMultiChanQueryFile.tStartTime.iSecond = 0;
    tMultiChanQueryFile.tStartTime.iSecond = 0;

    tMultiChanQueryFile.tStopTime.iYear = 2019;
    tMultiChanQueryFile.tStopTime.iMonth = 7;
    tMultiChanQueryFile.tStopTime.iDay = 21;
    tMultiChanQueryFile.tStopTime.iHour = 20;
    tMultiChanQueryFile.tStopTime.iMinute = 0;
    tMultiChanQueryFile.tStopTime.iSecond = 0;
    tMultiChanQueryFile.iPageSize = 20;
    tMultiChanQueryFile.iPageNo = 0;
    tMultiChanQueryFile.iTrigger = 0;
    tMultiChanQueryFile.iFiletype = 1;
    tMultiChanQueryFile.iTriggerType = 0x7FFFFFFF;

    iRet=NetClient_Query_V4(g_iLogonID,CMD_NETFILE_MULTI_CHANNEL_QUERY_FILE, 0, &tMultiChanQueryFile, sizeof(tMultiChanQueryFile));
    if(0 == iRet)
    {
        printf("Success : NetFileQuery\n");
    }
    else
    {
        printf("Err : NetFileQuery\n");
        return;
    }
}

void DwonloadByTime()
{
    DOWNLOAD_TIMESPAN downlaodTimespan = {0};
    downlaodTimespan.m_iSize = sizeof(DOWNLOAD_TIMESPAN);
    downlaodTimespan.m_tTimeBegin.iYear = 2019;
```

```
downloadTimespan.m_tTimeBegin.iMonth = 7;
downloadTimespan.m_tTimeBegin.iDay = 21;
downloadTimespan.m_tTimeBegin.iHour = 0;
downloadTimespan.m_tTimeBegin.iMinute = 5;
downloadTimespan.m_tTimeBegin.iSecond = 0;

downloadTimespan.m_tTimeEnd.iYear = 2019;
downloadTimespan.m_tTimeEnd.iMonth = 7;
downloadTimespan.m_tTimeEnd.iDay = 21;
downloadTimespan.m_tTimeEnd.iHour = 0;
downloadTimespan.m_tTimeEnd.iMinute = 5;
downloadTimespan.m_tTimeEnd.iSecond = 20;

downloadTimespan.m_iChannelNO = 0;
downloadTimespan.m_iPosition = -1;
downloadTimespan.m_iSpeed = 32;
downloadTimespan.m_iReqMode = 1;
printf("Input 0 = SDV; 1 = PS; 2 = MP4: ");
char cTemp[6] = {0};
gets_s(cTemp, 6);
if( 0 == strcmp( "0", cTemp)) //sdv 格式
{
    downloadTimespan.m_iSaveFileType = DOWNLOAD_FILE_TYPE_SDV;
    sprintf(downloadTimespan.m_cLocalFilename,
"%d%d%d%d%d%d-%d%d%d%d%d%d.sdv",
        downloadTimespan.m_tTimeBegin.iYear,
        downloadTimespan.m_tTimeBegin.iMonth,
        downloadTimespan.m_tTimeBegin.iDay,
        downloadTimespan.m_tTimeBegin.iHour,
        downloadTimespan.m_tTimeBegin.iMinute,
        downloadTimespan.m_tTimeBegin.iSecond,
        downloadTimespan.m_tTimeEnd.iYear,
        downloadTimespan.m_tTimeEnd.iMonth ,
        downloadTimespan.m_tTimeEnd.iDay ,
        downloadTimespan.m_tTimeEnd.iHour ,
        downloadTimespan.m_tTimeEnd.iMinute,
        downloadTimespan.m_tTimeEnd.iSecond);
}
else if(0 == strcmp( "1", cTemp)) //PS 格式
{
    downloadTimespan.m_iSaveFileType = DOWNLOAD_FILE_TYPE_PS;
    sprintf(downloadTimespan.m_cLocalFilename,
"%d%d%d%d%d%d-%d%d%d%d%d%d.ps",
        downloadTimespan.m_tTimeBegin.iYear,
```

```

        downloaodTimespan.m_tTimeBegin.iMonth,
        downloaodTimespan.m_tTimeBegin.iDay,
        downloaodTimespan.m_tTimeBegin.iHour,
        downloaodTimespan.m_tTimeBegin.iMinute,
        downloaodTimespan.m_tTimeBegin.iSecond,
        downloaodTimespan.m_tTimeEnd.iYear,
        downloaodTimespan.m_tTimeEnd.iMonth ,
        downloaodTimespan.m_tTimeEnd.iDay ,
        downloaodTimespan.m_tTimeEnd.iHour ,
        downloaodTimespan.m_tTimeEnd.iMinute,
        downloaodTimespan.m_tTimeEnd.iSecond);
    }
    else if(0 == strcmp( "2", cTemp)) //MP4 格式
    {
        downloaodTimespan.m_iSaveFileType = DOWNLOAD_FILE_TYPE_ZFMP4;
        sprintf(downloaodTimespan.m_cLocalFilename,
            "%d%d%d%d%d-%d%d%d%d%d.mp4",
                downloaodTimespan.m_tTimeBegin.iYear,
                downloaodTimespan.m_tTimeBegin.iMonth,
                downloaodTimespan.m_tTimeBegin.iDay,
                downloaodTimespan.m_tTimeBegin.iHour,
                downloaodTimespan.m_tTimeBegin.iMinute,
                downloaodTimespan.m_tTimeBegin.iSecond,
                downloaodTimespan.m_tTimeEnd.iYear,
                downloaodTimespan.m_tTimeEnd.iMonth ,
                downloaodTimespan.m_tTimeEnd.iDay ,
                downloaodTimespan.m_tTimeEnd.iHour ,
                downloaodTimespan.m_tTimeEnd.iMinute,
                downloaodTimespan.m_tTimeEnd.iSecond);
    }
    else
    {
        printf("Please Input Correctly!");
        return;
    }
    memset(cTemp, 0 , 6);
    if( -1 == g_iLogonID)
    {
        printf("Err : LogonId\n");
        return;
    }
    Int iRet = -1;
    iRet=NetClient_NetFileDownload(&g_uiConnID,g_iLogonID,DOWNLOAD_CMD_TIMES
    PAN,&downloaodTimespan, sizeof(DOWNLOAD_TIMESPAN));

```



```
    if(iRet < 0)
    {
        printf("Err : NetClient_NetFileDownload\n");
        return;
    }
    else
    {
        printf("Success : NetClient_NetFileDownload\n");
        if( -1 != (int)g_uiConnID)
        {
            //设置原始流回调
            iRet = NetClient_SetRawFrameCallBack(g_uiConnID, GetRawNotify, NULL);
            if( iRet < 0 )
            {
                printf("Err : NetClient_SetRawFrameCallBack\n");
                return;
            }
        }
    }

#ifdef __WIN__
    _beginthread( WinGetDownloadProcessThread, 0, NULL);
#else
    pthread_t t;
    pthread_create(&t, NULL, LinuxGetDownloadProcessThread, NULL);
#endif
    }
}
```

4.4 录像回放

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include "NetSdk.h"

#ifdef __WIN__
#include <process.h>
#else
#include <pthread.h>
#include <unistd.h>
#include <dlfcn.h>
```

```
#endif

int g_iLogonID = -1;
unsigned int g_uiConnID = -1;//连接 ID

int LogonDevice(int _iLogonType);
void __stdcall RecvRawFramNotify(unsigned int _uiID, unsigned char* _pucData,int _iLen,
RAWFRAME_INFO* _ptRawFrameInfo, void* _pvUser);
void StartSuperPlayBackByFile();
void StartSuperPlayBackByTime();

#ifdef __WIN__
void usleep(int _iMicroSecond)
{
    if (_iMicroSecond < 1000 && _iMicroSecond != 0)
    {
        _iMicroSecond = 1000;
    }
    Sleep(_iMicroSecond/1000);
}
#else
int gets_s(char *_pcStr, int _iCount)
{
    if (_pcStr == NULL)
        return -1;

    char* pcRet = fgets(_pcStr, _iCount, stdin);
    size_t uiLen = strlen(_pcStr);
    if (pcRet == NULL || uiLen == 0)
        return -2;

    if (_iCount - 1 > (int)uiLen || _pcStr[uiLen-1] == '\n')
    {
        _pcStr[uiLen-1] = '\0';
    }
    return 0;
}
#endif

void Notify_Main(int _iLogonID, long _lWparam, void* _pvLParam, void* _pvUsr)
{
    int lMsgType = LOWORD(_lWparam);
    long iMsgValue = (long)_pvLParam;
    switch (lMsgType)
```

```
{
case WCM_LOGON_NOTIFY:
    {
        printf("[Notify_Main]->WCM_LOGON_NOTIFY\n");
        if((LOGON_SUCCESS == iMsgValue) )
        {
            printf("[Notify_Main]->Success : Logon\n");
        }
        else
        {
            printf("[Notify_Main]->Err = %d : Logon, Please Retry\n", iMsgValue);
            return;
        }
    }
    break;
default:
    break;
}
}

int to_int_def(const char* _pstrFrom, int _iDef)
{
    if(NULL == _pstrFrom)
    {
        return _iDef;
    }
    for (size_t i=0; i<strlen(_pstrFrom); i++)
    {
        if (!isdigit(_pstrFrom[i]))
        {
            if(i == 0 && _pstrFrom[i] == '-' && strlen(_pstrFrom) > 1)
            {
                continue;
            }
            return _iDef;
        }
    }
    return atoi(_pstrFrom);
}

int main(int argc, char *argv[])
{
    char cCmd[16] = {0};
    int iLogonType = SERVER_NORMAL;
    char cTemp[8] = {0};
```

```
int iRet = LoadNVSSDK();
if (0 != iRet)
{
    printf("Err : LoadNVSSDK\n");
    return -1;
}

fprintf(stderr, "Please input LogonType: 0----Normal  1----Active\n");
gets_s(cTemp, 8);
iLogonType = to_int_def(cTemp, 0);
if (SERVER_ACTIVE == iLogonType)
{
    fprintf(stderr, "Please input listening port:");
    gets_s(cTemp, 8);
    int iListeningPort = to_int_def(cTemp, 0);
    iRet = NetClient_Startup_V4(iListeningPort, 0, 0);//初始化 SDK
}
else
{
    iRet = NetClient_Startup_V4(0, 0, 0);//初始化 SDK
}
if( 0 == iRet )
{
    printf("Success : NetClient_Startup_V4()\n");
}
else
{
    printf("Err : NetClient_Startup_V4()\n");
    return -1;
}
NetClient_SetNotifyFunction_V4(Notify_Main, NULL, NULL, NULL, NULL);
//登录设备
iRet = LogonDevice(iLogonType);
if( 0 == iRet )
{
    printf("Success : LogonDevice()\n");
}
else
{
    printf("Err : LogonDevice()\n");
}
while(1)
{
    printf("<Please Input play back Mode[pf=PlaybackFile, pt=PlaybackTime, s=Stop,
```

```
q=Exit]>:\n");
gets_s( cCmd, 16);
//退出
if( 0 == strcmp(cCmd, "q"))
{
    if( -1 != g_iLogonID)
    {
        NetClient_Logoff(g_iLogonID);//注销用户
    }
    NetClient_Cleanup();//释放 SDK 资源
    FreeNVSSDK();//释放接口库
    return 0;
}
//文件模式回放
else if( 0 == strcmp(cCmd, "pf"))
{
    if(-1 != g_iLogonID)
    {
        StartSuperPlayBackByFile();
    }
}
//时间段模式回放
else if( 0 == strcmp(cCmd, "pt"))
{
    if(-1 != g_iLogonID)
    {
        StartSuperPlayBackByTime();
    }
}
else if( 0 == strcmp(cCmd, "s"))
{
    if(-1 != g_iLogonID)
    {
        NetClient_StopPlayBack(g_uiConnID);
    }
}
}
getchar();
return 0;
}

int LogonDevice( int _iLogonType )
{
    char cIP[16] = {0};
```

```
char cUserName[16] = {0};
char cPassword[16] = {0};
char cProductID[32] = {0};

fprintf(stderr, "Please input user name: ");
gets_s(cUserName, 16);
fprintf(stderr, "Please input password: ");
gets_s(cPassword, 16);
LogonPara tNormal = {0};
LogonActiveServer tActive = {0};
void* pvPara = NULL;
int iBufLen = 0;
if (SERVER_ACTIVE == _iLogonType)
{
    fprintf(stderr, "Please input ProductID: ");
    gets_s(cProductID, 32);
    tActive.iSize = sizeof(LogonActiveServer);
    strcpy(tActive.cUserName, cUserName);
    strcpy(tActive.cUserPwd, cPassword);
    strcpy(tActive.cProductID, cProductID);
    pvPara = &tActive;
    iBufLen = sizeof(LogonActiveServer);

    DsmOnline tOnline = {0};
    tOnline.iSize = sizeof(DsmOnline);
    strncpy(tOnline.cProductID, cProductID, LEN_32);
    NetClient_GetDsmRegstierInfo(DSM_CMD_GET_ONLINE_STATE, &tOnline,
        sizeof(DsmOnline));
    int iOutTime = 0;
    while (DSM_STATE_ONLINE != tOnline.iOnline)
    {
        if (iOutTime >= 20)
        {
            fprintf(stderr, "Device not register!\n");
            break;
        }
        usleep(1000 * 1000);
        NetClient_GetDsmRegstierInfo(DSM_CMD_GET_ONLINE_STATE, &tOnline,
            sizeof(DsmOnline));
        iOutTime++;
    }
}
else
{
```

```
        fprintf(stderr, "Please input server IP: ");
        gets_s(cIP, 16);
        tNormal.iSize = sizeof(LogonPara);
        tNormal.iNvsPort = 3000;
        strcpy(tNormal.cNvsIP, cIP);
        strcpy(tNormal.cUserName, cUserName);
        strcpy(tNormal.cUserPwd, cPassword);
        strcpy(tNormal.cCharSet, "UTF-8");
        pvPara = &tNormal;
        iBufLen = sizeof(LogonPara);
    }

    g_iLogonID = NetClient_Logon_V4(_iLogonType, pvPara, iBufLen);
    if(g_iLogonID < 0)
    {
        fprintf(stderr, "[NetClient_Logon_V4] fail! %d\n", g_iLogonID);
    }
    else
    {
        fprintf(stderr, "[NetClient_Logon_V4] Success! %d\n", g_iLogonID);
    }
    return g_iLogonID;
}

void StartSuperPlayBackByFile()
{
    PlayerParam tParam = {0};
    tParam.iSize = sizeof(PlayerParam);
    tParam.iLogonID = g_iLogonID;
    tParam.tRawNotifyInfo.pcbkRawFrameNotify = &RecvRawFramNotify;
    tParam.tRawNotifyInfo.pUserData = NULL;
    tParam.tRawNotifyInfo.iForbidDecodeEnable = RAW_NOTIFY_FORBID_DECODE;
    strcpy(tParam.strFileName, "H00000E80001F6C00000C00.sdv.sdv");
    int iRet = NetClient_PlayBack(&g_uiConnID, PLAYBACK_TYPE_FILE, &tParam,
    NULL);
    if (iRet < 0)
    {
        printf("Err : NetClient_PlayBack:PLAYBACK_TYPE_FILE fail!\n");
    }
}

void StartSuperPlayBackByTime()
{
    PlayerParam tParam = {0};
```

```

tParam.iSize = sizeof(PlayerParam);
tParam.iLogonID = g_iLogonID;
tParam.iChannNo = 0;
tParam.tBeginTime.iYear = 2019;
tParam.tBeginTime.iMonth = 7;
tParam.tBeginTime.iDay = 21;
tParam.tBeginTime.iHour = 9;
tParam.tBeginTime.iMinute = 0;
tParam.tBeginTime.iSecond = 0;
tParam.tEndTime.iYear = 2019;
tParam.tEndTime.iMonth = 7;
tParam.tEndTime.iDay = 21;
tParam.tEndTime.iHour = 10;
tParam.tEndTime.iMinute = 5;
tParam.tEndTime.iSecond = 20;
tParam.tRawNotifyInfo.pcbkRawFrameNotify = &RecvRawFramNotify;
tParam.tRawNotifyInfo.pUserData = NULL;
tParam.tRawNotifyInfo.iForbidDecodeEnable = RAW_NOTIFY_FORBID_DECODE;

int iRet = NetClient_PlayBack(&g_uiConnID,    PLAYBACK_TYPE_TIME,    &tParam,
NULL);
if (iRet < 0)
{
    printf("Err : NetClient_PlayBack:PLAYBACK_TYPE_TIME fail!\n");
    return ;
}
}

int g_iCnt = 0;
void __stdcall RecvRawFramNotify(unsigned int _uiID, unsigned char* _pucData,int _iLen,
RAWFRAME_INFO* _ptRawFrameInfo, void* _pvUser)
{
    RAWFRAME_INFO tRawFrameInfo = {0};
    memcpy(&tRawFrameInfo, _ptRawFrameInfo, sizeof(tRawFrameInfo));
    char cLog[1024] = {0};
    sprintf(cLog, "[RecvRawFramNotify PlayerId(%d), FrameType(%d), DataLen(%d),
nWidth(%d), nHeight(%d),nStamp(%d) nEnCoder(%d), nFrameRate(%d) nAbsStamp(%d)
ucBitsPerSample(%d), uiSamplesPerSec(%d)\n]", _uiID, tRawFrameInfo.nType, _iLen,
tRawFrameInfo.nWidth, tRawFrameInfo.nHeight, tRawFrameInfo.nStamp,
tRawFrameInfo.nEnCoder, tRawFrameInfo.nFrameRate, tRawFrameInfo.nAbsStamp,
tRawFrameInfo.ucBitsPerSample, tRawFrameInfo.uiSamplesPerSec);
    if (0 == g_iCnt % 1000) {
        printf(cLog);
    }
}

```



```

    g_iCnt++;
    if (0 == tRawFrameInfo.nType || 1 == tRawFrameInfo.nType)
    {    //视频数据
    }
    else if (5 == tRawFrameInfo.nType)
    {
        //音频数据
    }
}

```

4.5 人脸检测

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include "NetSdk.h"

int g_iLogonId = -1;
int g_iConnectId = -1;
int g_iVcaStatus = 0;

#define VCA_SUSPEND_STATUS_PAUSE    0    //暂停智能分析
#define VCA_SUSPEND_STATUS_RESUME    1    //恢复智能分析

#define VCA_SUSPEND_RESULT_SUCCESS    1    //智能分析暂停成功
#define VCA_SUSPEND_RESULT_CONFIGING    2    //智能分析暂停失败，正在设置，不可设参

#ifndef __WIN__
#include <unistd.h>
#include <sys/types.h>
#include <sys/stat.h>
#define min(a,b)    (((a) < (b)) ? (a) : (b))
int gets_s(char *_pcStr, int _iCount)
{
    if (_pcStr == NULL)
        return -1;
    char* pcRet = fgets(_pcStr, _iCount, stdin);
    size_t uiLen = strlen(_pcStr);
    if (pcRet == NULL || uiLen == 0)
        return -2;
}

```

```
    if (_iCount - 1 > (int)uiLen || _pcStr[uiLen-1] == '\n')
    {
        _pcStr[uiLen-1] = '\0';
    }
    stdin->_IO_read_ptr = stdin->_IO_read_end;
    return 0;
}
#else
#include
void usleep(int _iMicroSecond)
{
    if (_iMicroSecond < 1000 && _iMicroSecond != 0)
    {
        _iMicroSecond = 1000;
    }
    Sleep(_iMicroSecond/1000);
}
#endif

int LogonDevice()
{
    LogonPara tInfo = {0};
    tInfo.iSize = sizeof(LogonPara);
    tInfo.iNvsPort = 3000;
    fprintf(stderr, "Please input server IP: ");
    gets_s(tInfo.cNvsIP, 32);
    if (0 == strlen(tInfo.cNvsIP)) {
        strncpy_ss(tInfo.cNvsIP, "10.30.41.224", sizeof(tInfo.cNvsIP));
    }
    fprintf(stderr, "Please input user name: ");
    gets_s(tInfo.cUserName, 16);
    if (0 == strlen(tInfo.cUserName)) {
        strncpy_ss(tInfo.cUserName, "Admin", sizeof(tInfo.cUserName));
    }
    fprintf(stderr, "Please input password: ");
    gets_s(tInfo.cUserPwd, 16);
    if (0 == strlen(tInfo.cUserPwd)) {
        strncpy_ss(tInfo.cUserPwd, "Admin", sizeof(tInfo.cUserPwd));
    }
    fprintf(stderr, "Please input port: ");
    char cPort[16] = {0};
    gets_s(cPort, 16);
    tInfo.iNvsPort = atoi(cPort);
    if (tInfo.iNvsPort <= 0) {
```

```

        tInfo.iNvsPort = 3000;
    }
    int iLogonId = NetClient_Logon_V4(SERVER_NORMAL, &tInfo, tInfo.iSize);
    if(iLogonId < 0)
    {
        fprintf(stderr, "[LogonDevice] NetClient_Logon_V4(%s) failed, %d\n", tInfo.cNvsIP,
iLogonId);
        return -1;
    }

    //注:NVSSDK 异步运行模式, 登录是否成功, 需要等主回调消息, 或者循环获取登录
    状态

    //此处阻塞主线程, 循环获取登录状态
    int iTimes = 0;
    while (LOGON_SUCCESS != NetClient_GetLogonStatus(iLogonId))
    {
        if (iTimes++ > 100)//5 秒超时
        {
            fprintf(stderr, "[LogonDevice] logon(%s) timeout.\n", tInfo.cNvsIP);
            return -1;
        }
        usleep(50000);    //50 毫秒获取一次登录状态
    }
    return 0;
}

void Notify_Main(int _iLogonID, long _lWparam, void* _pvLParam, void* _pvUsr)
{
    switch (LOWORD(_lWparam))
    {
        case WCM_LOGON_NOTIFY:    //登录消息
        {
            int iLogonStatus = NetClient_GetLogonStatus(_iLogonID);
            if (LOGON_SUCCESS == iLogonStatus) {    //登录成功
                printf("[Notify_Main] logon, id=%d, success.\n", _iLogonID);
                g_iLogonId = _iLogonID;
            } else if (LOGON_FAILED == iLogonStatus) {    //登录失败
                printf("[Notify_Main] logon, id=%d, failed.\n", _iLogonID);
            } else if (LOGON_TIMEOUT == iLogonStatus) {    //登录超时
                printf("[Notify_Main] logon, id=%d, timeout.\n", _iLogonID);
            } else if (LOGON_RETRY == iLogonStatus) {    //重新登录
                printf("[Notify_Main] logon, id=%d, retry.\n", _iLogonID);
            } else if (LOGON_ING == iLogonStatus) {    //登录中

```

```
        printf("[Notify_Main] logon, id=%d, ing.\n", _iLogonID);
    } else {
        printf("[Notify_Main] logon, id=%d, %d.\n", _iLogonID, iLogonStatus);
    }
}
break;
case WCM_VCA_SUSPEND:    //智能分析暂停消息
{
    int iResult = 0;
    VCASuspendResult tParam = {0};
    tParam.iBufSize = sizeof(VCASuspendResult);
    NetClient_GetDevConfig(g_iLogonId,    NET_CLIENT_VCA_SUSPEND,    0,
    &tParam, sizeof(tParam), &iResult);
    g_iVcaStatus = tParam.iResult;        //结果
    if (VCA_SUSPEND_STATUS_PAUSE == tParam.iStatus) {
        if (VCA_SUSPEND_RESULT_SUCCESS == tParam.iResult) {
            printf("[Notify_Main] pause vca success.\n");
        } else {
            printf("[Notify_Main] pause vca failed.\n");
        }
    }
}
break;
default:
    break;
}
}

int FaceDetectionEnable()
{
    AnyScene tParam = {0};
    tParam.iBufSize = sizeof(AnyScene);
    tParam.iSceneID = 0;    //场景号 0-15
    tParam.iDevType = 1;    //0-IPC, 1-NVR
    int iBytesReturned = 0;
    int iRet = NetClient_GetDevConfig(g_iLogonId, NET_CLIENT_ANYSCENE, 0, &tParam,
    sizeof(tParam), &iBytesReturned);
    if (iRet >= 0)
    {
        tParam.iArithmetic = 1<<2;//人脸检测算法开启
        tParam.iDevType = 1;    //0-IPC, 1-NVR
        iRet =NetClient_SetDevConfig(g_iLogonId, NET_CLIENT_ANYSCENE, 0, &tParam,
        sizeof(tParam));
        if (iRet >= 0)
```

```
    {
        printf("人脸检测算法开启成功.\n");
    }
    else
    {
        printf("NetClient_SetDevConfig NET_CLIENT_ANYSCENE failed.\n");
    }
}
else
{
    printf("NetClient_GetDevConfig NET_CLIENT_ANYSCENE failed.\n");
}

return iRet;
}

int SetDetectParam()
{
    FaceDetectArithmetic fda = {0};
    fda.iBufSize = sizeof(FaceDetectArithmetic);
    fda.iDevType = 1; //0-IPC, 1-NVR
    int iByteReturn = 0;
    int iRet = -1;
    iRet=NetClient_GetDevConfig(g_iLogonId,NET_CLIENT_FACE_DETECT_ARITHMETIC,0,&fda,sizeof(FaceDetectArithmetic),&iByteReturn);
    if (iRet < 0)
    {
        printf("NetClient_GetDevConfig:NET_CLIENT_FACE_DETECT_ARITHMETIC failed.\n");
        return -1;
    }
    if (fda.iMinSize>=fda.iMaxSize)
    {
        fda.iMaxSize = fda.iMinSize+1;//调整参数有效性(规避)
    }
    fda.iPushMode = 2;
    fda.iSnapTimes = 1;

    iRet=NetClient_SetDevConfig(g_iLogonId,NET_CLIENT_FACE_DETECT_ARITHMETIC, 0, &fda, sizeof(FaceDetectArithmetic));
    if (iRet >= 0)
    {
        printf("NetClient_SetDevConfig->NET_CLIENT_FACE_DETECT_ARITHMETIC success.\n");
    }
}
```

```
}
else
{
    printf("NetClient_SetDevConfig->NET_CLIENT_FACE_DETECT_ARITHMETIC
    failed.\n");
}

return iRet;
}

int SetPicStreamUploadParam()
{
    //设置背景图质量和上传使能
    PicStreamUploadParam tInfo = {0};
    tInfo.iSize= sizeof(PicStreamUploadParam);
    tInfo.iSceneId = 0;
    tInfo.iPicType = 0;//0-背景图
    int iRet = -1;

    iRet=NetClient_VCAGetConfig(g_iLogonId,VCA_CMD_PICSTREAM_UPLOADPARAM,
    0, &tInfo, sizeof(PicStreamUploadParam));
    if (iRet < 0)
    {
        printf("NetClient_GetDevConfig->VCA_CMD_PICSTREAM_UPLOADPARAM
        failed.\n");
    }
    else
    {
        tInfo.iSnapEnable = 1;
        tInfo.iQpvalue = 80;
        tInfo.iIsOsd = 1;
        iRet=NetClient_VCASetConfig(g_iLogonId,VCA_CMD_PICSTREAM_UPLOADPAR
        AM, 0, &tInfo, sizeof(PicStreamUploadParam));
        if (iRet >= 0)
        {
            printf("NetClient_VCASetConfig VCA_CMD_PICSTREAM_UPLOADPARAM
            success.\n");
        }
        else
        {
            printf("NetClient_VCASetConfig VCA_CMD_PICSTREAM_UPLOADPARAM
            failed.\n");
        }
    }
}
```

```
//设置特写图质量和上传使能
memset(&tInfo, 0, sizeof(tInfo));
tInfo.iSize = sizeof(PicStreamUploadParam);
tInfo.iSceneId = 0;
tInfo.iPicType = 1;//1-特写图
iRet=NetClient_VCAGetConfig(g_iLogonId,VCA_CMD_PICSTREAM_UPLOADPARAM,
0, &tInfo, sizeof(PicStreamUploadParam));
if (iRet < 0)
{
    printf("NetClient_GetDevConfig->VCA_CMD_PICSTREAM_UPLOADPARAM
    failed.\n");
}
else
{
    tInfo.iSnapEnable = 1;
    tInfo.iQpvalue = 30;

    iRet=NetClient_VCASetConfig(g_iLogonId,VCA_CMD_PICSTREAM_UPLOADPAR
    AM, 0, &tInfo, sizeof(PicStreamUploadParam));
    if (iRet < 0)
    {
        printf("NetClient_VCASetConfig VCA_CMD_PICSTREAM_UPLOADPARAM
        failed.\n");
    }
}
return iRet;
}

int SetVcaStatue(int _iStatus)
{
    int iChanNo = 0; //通道号，0 表示第一通道，IPC 只有 1 个通道
    VCASuspend tInfo = {0};
    tInfo.iStatus = _iStatus;
    tInfo.iDevType = 1;
    return NetClient_SetDevConfig(g_iLogonId, NET_CLIENT_VCA_SUSPEND, iChanNo,
    &tInfo, sizeof(tInfo));
}

int SavePicture(char* _pcFileName, char* _pcData, int _iLen)
{
    if (NULL == _pcFileName || NULL == _pcData || _iLen <= 0)
    {
        return -1;
    }
}
```

```

    }
    FILE* pFile = fopen(_pcFileName, "wb");
    if (NULL == pFile)
    {
        return -1;
    }
    fwrite(_pcData, _iLen, 1, pFile);
    fclose(pFile);
    pFile = NULL;
    return 0;
}

//人脸图片流回调
int __stdcall CallBack_PicStreamInfo(unsigned int _uiRecvID, long _lCommand, void* _lpInfo,
int _iBufLen, void* _pvUser)
{
    if (NULL == _lpInfo || NET_PICSTREAM_CMD_FACE != _lCommand)
    {
        return -1;
    }
    FacePicStream* pFace = (FacePicStream*)_lpInfo;

    //保存全景图
    PicData tFullData;
    memset(&tFullData, 0, sizeof(tFullData));
    if (NULL != pFace->ptFullData)
    {
        memcpy(&tFullData, pFace->ptFullData, min(pFace->iSizeOfFull, (int)sizeof(PicData)));
        char cFullPicName[256] = {0};
        sprintf(cFullPicName, ".//Pic//FullPic_Time(2%03d%02d%02d%02d%02d%02d).jpg",
            tFullData.tPicTime.uiYear, tFullData.tPicTime.uiMonth, tFullData.tPicTime.uiDay, tFull
            Data.tPicTime.uiHour, tFullData.tPicTime.uiMinute, tFullData.tPicTime.uiSecondsr, tFull
            Data.tPicTime.uiMilliseconds);
        SavePicture(cFullPicName, tFullData.pcPicData, tFullData.iDataLen);
    }
    //保存人脸图和地图
    for (int i = 0; i < pFace->iFaceCount && i < MAX_FACE_PICTURE_COUNT; ++i)
    {
        FacePicData tFaceData = {0};
        memcpy(&tFaceData, pFace->ptFaceData[i], min(pFace->iSizeOfFace, (int)sizeof(FacePi
            cData)));
        char cFacePicName[256] = {0};

        sprintf(cFacePicName, ".//Pic//FacePic_No%d_Time(2%03d%02d%02d%02d%02d%02d%02d%02d%02d%02d).jpg",
            tFaceData.tPicTime.uiYear, tFaceData.tPicTime.uiMonth, tFaceData.tPicTime.uiDay, tFaceData.tPicTime.uiHour, tFaceData.tPicTime.uiMinute, tFaceData.tPicTime.uiSecondsr, tFaceData.tPicTime.uiMilliseconds, i);
        SavePicture(cFacePicName, tFaceData.pcPicData, tFaceData.iDataLen);
    }
}

```



```

d%d).jpg",i,tFullData.tPicTime.uiYear,tFullData.tPicTime.uiMonth,tFullData.tPicTime.
uiDay,tFullData.tPicTime.uiHour,tFullData.tPicTime.uiMinute,tFullData.tPicTime.uiSec
onds,tFullData.tPicTime.uiMilliseconds);
//人脸图
SavePicture(cFacePicName, tFaceData.pcPicData, tFaceData.iDataLen);

//底图
if (1 == tFaceData.iAramType)
{
    char cNegPicName[256] = {0};
    sprintf(cNegPicName,"./Pic//NegPic_No%d_Time(2%03d%02d%02d%02d%0
2d%02d).jpg",i,tFullData.tPicTime.uiYear,tFullData.tPicTime.uiMonth,tFull
Data.tPicTime.uiDay,
    tFullData.tPicTime.uiHour,tFullData.tPicTime.uiMinute,tFullData.tPicTime.uiSe
conds, tFullData.tPicTime.uiMilliseconds);
    SavePicture(cFacePicName, tFaceData.pcNegPicData, tFaceData.iNegPicLen);
}
}
return 0;
}

//开启图片流
int StartSnap()
{
    NetPicPara tNetPicParam = {0};
    tNetPicParam.iStructLen = sizeof(tNetPicParam);
    tNetPicParam.iChannelNo = 0;
    tNetPicParam.cbkPicStreamNotify = CallBack_PicStreamInfo; //抓拍回调函数
    tNetPicParam.pvUser = NULL;
    unsigned int iConnectID = -1;
    int iRet = -1;
    iRet=NetClient_StartRecvNetPicStream(g_iLogonId,&tNetPicParam,tNetPicParam.iStructLe
n, &iConnectID);
    if (iRet < 0) {
        g_iConnectId = -1;
        fprintf(stderr,"StartRecvNetPicStream Failed!");
    } else {
        g_iConnectId = (int)iConnectID;
        fprintf(stderr,"StartRecvNetPicStream Success! ConnectID(%d)\n\n", g_iConnectId);
    }
    return 0;
};

int main(int argc,char *argv[])

```

```
{
    //加载库
    LoadLib();
    //注册主回调函数
    NetClient_SetNotifyFunction_V4(Notify_Main, NULL, NULL, NULL, NULL);
    //启动 SDK
    NetClient_Startup_V4(0, 0, 0);
    //登录设备
    if (LogonDevice() < 0)
    {
        fprintf(stderr, "[main] LogonDevice failed.\n");
        getchar();
        return -1;
    }
    //开启图片流
#ifdef __WIN__
    _mkdir(".\\Pic");
#else
    mkdir("./Pic", S_IRWXU | S_IRWXG | S_IROTH | S_IXOTH);
#endif
    StartSnap();
    while (1)
    {
        fprintf(stderr, "请选择: 0 退出, 1 开启人脸检测, 2 设置人脸检测参数\n");
        char cTemp[16] = {0};
        gets_s(cTemp, 16);
        if (0 == strlen(cTemp))
        {
            continue;
        }
        int iOpt = atoi(cTemp);
        if (0 == iOpt)
        {
            break;
        }
        else if (1 == iOpt) //开启人脸检测
        {
            FaceDetectionEnable();
        }
        else if (2 == iOpt) //设置人脸检测参数
        {
            SetVcaStatue(VCA_SUSPEND_STATUS_PAUSE); //暂停智能分析
            getchar();
            if (VCA_SUSPEND_RESULT_SUCCESS != g_iVcaStatus) {
```

```

        fprintf(stderr, "[main] 智能分析暂停失败!\n");
        getchar();
        continue;
    }
    SetDetectParam();//人脸检测相关参数(需要暂停智能分析)
    SetVcaStatue(VCA_SUSPEND_STATUS_RESUME);//设置完需要恢复智能分析
    SetPicStreamUploadParam();//抓拍图片质量、上传相关参数
}
}
//注销用户，释放 SDK 资源
NetClient_Logoff(g_iLogonId);
NetClient_Cleanup();
return 0;
}

```

4.6 人脸识别

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include "NetSdk.h"

int g_iLogonId = -1;
int g_iConnectId = -1;
int g_iVcaStatus = 0;
FaceLibInfo g_tLastLibInfo = {0};           //保存最后一个库信息，便于修改和删除
FaceInfo g_tLastPicInfo = {0};             //保存最后一个人脸信息，便于修改和删除
#define VCA_SUSPEND_STATUS_PAUSE    0      //暂停智能分析
#define VCA_SUSPEND_STATUS_RESUME    1      //恢复智能分析
#define VCA_SUSPEND_RESULT_SUCCESS   1      //智能分析暂停成功
#define VCA_SUSPEND_RESULT_CONFIGING 2      //智能分析暂停失败，正在设置，不可设参数

#ifdef __WIN__
#include <unistd.h>
#include <sys/types.h>
#include <sys/stat.h>
#define min(a,b)    (((a) < (b)) ? (a) : (b))

int gets_s(char *_pcStr, int _iCount)
{

```

```
if (_pcStr == NULL)
    return -1;

char* pcRet = fgets(_pcStr, _iCount, stdin);
size_t uiLen = strlen(_pcStr);
if (pcRet == NULL || uiLen == 0)
    return -2;

if (_iCount - 1 > (int)uiLen || _pcStr[uiLen-1] == '\n')
{
    _pcStr[uiLen-1] = '\0';
}

stdin->_IO_read_ptr = stdin->_IO_read_end;
return 0;
}
#else
#include<direct.h>
void usleep(int _iMicroSecond)
{
    if (_iMicroSecond < 1000 && _iMicroSecond != 0)
    {
        _iMicroSecond = 1000;
    }
    Sleep(_iMicroSecond/1000);
}
#endif

int ToIntDef(const char* _pstrFrom, int _iDef=0)
{
    if (NULL == _pstrFrom || strlen(_pstrFrom) <= 0)
    {
        return _iDef;
    }
    size_t i=0;
    if (0 == memcmp("-",_pstrFrom,1))
    {
        i++;
    }
    for (; i < strlen(_pstrFrom); i++)
    {
        if (!isdigit(_pstrFrom[i]))
        {
            return _iDef;
        }
    }
}
```

```
    }  
}  
  
int iFlag = 10;  
if(NULL != strstr(_pstrFrom, "0x") || NULL != strstr(_pstrFrom, "0X"))  
{  
    iFlag = 16;  
}  
//return atoi(_pstrFrom);--此方式无符号数有可能溢出  
return (int)strtoul(_pstrFrom, 0, iFlag);  
}  
  
int LogonDevice()  
{  
    LogonPara tInfo = {0};  
    tInfo.iSize = sizeof(LogonPara);  
    tInfo.iNvsPort = 3000;  
  
    fprintf(stderr, "Please input server IP: ");  
    gets_s(tInfo.cNvsIP, 32);  
    if (0 == strlen(tInfo.cNvsIP)) {  
        strncpy_ss(tInfo.cNvsIP, "192.168.1.2", sizeof(tInfo.cNvsIP));  
    }  
  
    fprintf(stderr, "Please input user name: ");  
    gets_s(tInfo.cUserName, 16);  
    if (0 == strlen(tInfo.cUserName)) {  
        strncpy_ss(tInfo.cUserName, "Admin", sizeof(tInfo.cUserName));  
    }  
  
    fprintf(stderr, "Please input password: ");  
    gets_s(tInfo.cUserPwd, 16);  
    if (0 == strlen(tInfo.cUserPwd)) {  
        strncpy_ss(tInfo.cUserPwd, "Admin", sizeof(tInfo.cUserPwd));  
    }  
  
    fprintf(stderr, "Please input port: ");  
    char cPort[16] = {0};  
    gets_s(cPort, 16);  
    tInfo.iNvsPort = atoi(cPort);  
    if (tInfo.iNvsPort <= 0)  
    {  
        tInfo.iNvsPort = 3000;  
    }  
}
```

```

int iLogonId = NetClient_Logon_V4(SERVER_NORMAL, &tInfo, tInfo.iSize);
if(iLogonId < 0)
{
    fprintf(stderr, "[LogonDevice] NetClient_Logon_V4(%s) failed, %d\n", tInfo.cNvsIP,
iLogonId);
    return -1;
}

```

//注:NVSSDK 异步运行模式，登录是否成功，需要等主回调消息，或者循环获取登录状态

```

//此处阻塞主线程，循环获取登录状态
int iTimes = 0;
while (LOGON_SUCCESS != NetClient_GetLogonStatus(iLogonId))
{
    if (iTimes++ > 100)//5 秒超时
    {
        fprintf(stderr, "[LogonDevice] logon(%s) timeout.\n", tInfo.cNvsIP);
        return -1;
    }
    usleep(50000);    //50 毫秒获取一次登录状态
}
return 0;
}

```

```

void Notify_Main(int _iLogonID, long _lWparam, void* _pvLParam, void* _pvUsr)
{
    switch (LOWORD(_lWparam))
    {
        case WCM_LOGON_NOTIFY:    //登录消息
        {
            int iLogonStatus = NetClient_GetLogonStatus(_iLogonID);
            if (LOGON_SUCCESS == iLogonStatus) {    //登录成功
                printf("[Notify_Main] logon, id=%d, success.\n", _iLogonID);
                g_iLogonId = _iLogonID;
            } else if (LOGON_FAILED == iLogonStatus) {    //登录失败
                printf("[Notify_Main] logon, id=%d, failed.\n", _iLogonID);
            } else if (LOGON_TIMEOUT == iLogonStatus) {    //登录超时
                printf("[Notify_Main] logon, id=%d, timeout.\n", _iLogonID);
            } else if (LOGON_RETRY == iLogonStatus) {    //重新登录
                printf("[Notify_Main] logon, id=%d, retry.\n", _iLogonID);
            } else if (LOGON_ING == iLogonStatus) {    //登录中
                printf("[Notify_Main] logon, id=%d, ing.\n", _iLogonID);
            }
        }
    }
}

```

```

        } else {
            printf("[Notify_Main] logon, id=%d, %d.\n", _iLogonID, iLogonStatus);
        }
    }
    break;
case WCM_VCA_SUSPEND:    //智能分析暂停消息
{
    int iResult = 0;
    VCASuspendResult tParam = {0};
    tParam.iBufSize = sizeof(VCASuspendResult);
    NetClient_GetDevConfig(g_iLogonId,    NET_CLIENT_VCA_SUSPEND,    0,
    &tParam, sizeof(tParam), &iResult);
    g_iVcaStatus = tParam.iResult;    //结果
    if (VCA_SUSPEND_STATUS_PAUSE == tParam.iStatus) {
        if (VCA_SUSPEND_RESULT_SUCCESS == tParam.iResult) {
            printf("[Notify_Main] pause vca success.\n");
        } else {
            printf("[Notify_Main] pause vca failed.\n");
        }
    }
}
    break;
default:
    break;
}
}

//人脸库查询
int FaceLibraryQuery()
{
    int iRet = -1;
    FaceLibQuery tQuery = {0};
    tQuery.iSize = sizeof(tQuery);
    tQuery.iChanNo = 0;    //通道号, 0 表示第一通道, IPC 只有 1 个通道
    tQuery.iPageCount = FACE_MAX_PAGE_COUNT; //每页个数, 每页最大查询 20 个

    //清空之前查询的数据
    usleep(20*1000);    //防止操作后立即查询
    printf("人脸库信息-----\n");
    int iPageNo = 0;    //查询页码, 0 表示第一页
    while (true)
    {
        //查询库信息
        tQuery.iPageNo = iPageNo;

```

```

FaceLibQueryResult tResult[FACE_MAX_PAGE_COUNT];
memset(&tResult, 0, sizeof(tResult));
iRet= NetClient_FaceConfig(g_iLogonId, FACE_CMD_LIB_QUERY, tQuery.iChanNo,
&tQuery, tQuery.iSize, &tResult, sizeof(FaceLibQueryResult));
if (0 != iRet) {
    printf("[FaceLibraryQuery] failed, ret=%d.\n", iRet);
    return iRet;
}

//查询完后，打印出来
for(int i = 0; i < tResult[0].iTotal && i < FACE_MAX_PAGE_COUNT; ++i) {
    //保存库信息，此处只保存最后一个，便于修改和删除
    g_tLastLibInfo = tResult[i].tFaceLib;
    int iIndex = i + 1;
    char* strType = "上传";
    if(1 == tResult[i].tFaceLib.iAlarmType){
        strType = "不上传";
    }
    printf("序号:%d, 库键值:%d, 识别阈值:%d, 库名称:%s, 识别信息:%s, 描述:%s\n",iIndex,tResult[i].tFaceLib.iLibKey,tResult[i].tFaceLib.iThreshold,tResult[i].tFaceLib.cName,strType,tResult[i].tFaceLib.cExtrInfo);
}
//计算总页数
int iTotlPage = tResult[0].iTotal / FACE_MAX_PAGE_COUNT;
if (tResult[0].iTotal % FACE_MAX_PAGE_COUNT > 0) {
    iTotlPage = iTotlPage + 1;
}
iPageNo++;
if (iPageNo >= iTotlPage || iPageNo > 1) {
    break;
}
}
printf("\n");
return iRet;
}

//人脸库添加
int FaceLibraryAdd()
{
    FaceLibEdit tInfo = {0};
    //必须字段
    tInfo.iSize = sizeof(tInfo);
    tInfo.iChanNo = 0;    //通道号，0 表示第一通道，IPC 只有 1 个通道

```



```

tInfo.tFaceLib.iThreshold = 70; //识别阈值，范围 0~100
tInfo.tFaceLib.iLibKey = 0; //0 表示添加
tInfo.tFaceLib.iAlarmType = 0; //0 不上传，1 上传
//end

//非必需字段
strncpy_ss(tInfo.tFaceLib.cName, "libname", sizeof(tInfo.tFaceLib.cName));
strncpy_ss(tInfo.tFaceLib.cExtrInfo, "facelib~~~~~add~~~~~", sizeof(tInfo.tFaceLib.cExtrInfo));
//end

//普通设备不需要此字段
tInfo.tFaceLib.iOptType = 1; //1 添加,2 修改
//end
//添加
FaceReply tReply = {0};
int iRet = NetClient_FaceConfig(g_iLogonId, FACE_CMD_LIB_EDIT, tInfo.iChanNo,
&tInfo, tInfo.iSize, &tReply, sizeof(FaceReply));
if(0 != iRet){
    printf("人脸库添加失败: %d\n", iRet);
} else {
    printf("人脸库添加结果: %d\n", tReply.iResult);
}

//显示添加人脸库结果
FaceLibraryQuery();
return iRet;
}

//人脸库修改
int FaceLibraryModify()
{
    if (g_tLastLibInfo.iLibKey <= 0)
    {
        printf("人脸库修改失败: 请先添加或查询人脸库\n");
        return -1;
    }
    FaceLibEdit tInfo = {0};
    tInfo.iSize = sizeof(tInfo);
    tInfo.iChanNo = 0; //通道号，0 表示第一通道，IPC 只有 1 个通道
    tInfo.tFaceLib.iThreshold = 80; //识别阈值，范围 0~100
    tInfo.tFaceLib.iLibKey = g_tLastLibInfo.iLibKey; //大于 0 表示修改，此处默认修改最后一个库
    tInfo.tFaceLib.iAlarmType = 1; //0 不上传，1 上传

```

```

    strncpy_ss(tInfo.tFaceLib.cName, "libname2", sizeof(tInfo.tFaceLib.cName));
    strncpy_ss(tInfo.tFaceLib.cExtrInfo, "facelib~~~~~modify~~~~~", sizeof(tInfo.tFaceLib.cExtrInfo));
    //普通设备不需要此字段
    tInfo.tFaceLib.iOptType = 2; //1 添加,2 修改
    //end
    //修改
    FaceReply tReply = {0};
    int iRet = NetClient_FaceConfig(g_iLogonId, FACE_CMD_LIB_EDIT, tInfo.iChanNo,
    &tInfo, tInfo.iSize, &tReply, sizeof(FaceReply));
    if(0 != iRet){
        printf("人脸库修改失败: %d\n", iRet);
    } else {
        printf("人脸库修改结果: %d\n", tReply.iResult);
    }

    //显示修改人脸库结果
    FaceLibraryQuery();
    return iRet;
}

//人脸库删除
int FaceLibraryDelete()
{
    if (g_tLastLibInfo.iLibKey <= 0)
    {
        printf("人脸库删除失败: 请先添加或查询人脸库\n");
        return -1;
    }

    FaceLibDelete tInfo = {0};
    tInfo.iSize = sizeof(tInfo);
    tInfo.iChanNo = 0; //通道号, 0 表示第一通道, IPC 只有 1 个通道
    tInfo.iLibKey = g_tLastLibInfo.iLibKey;//此处默认删除最后一个库;

    FaceReply tReply = {0};
    int iRet = NetClient_FaceConfig(g_iLogonId, FACE_CMD_LIB_DELETE, tInfo.iChanNo,
    &tInfo, tInfo.iSize, &tReply, sizeof(FaceReply));
    if(0 != iRet){
        printf("人脸库删除失败: %d\n", iRet);
    } else {
        printf("人脸库删除结果: %d\n", tReply.iResult);
    }
}

```

```

//显示删除人脸库结果
FaceLibraryQuery();
return iRet;
}

//人脸底图查询
int FacePictureQuery(int _iPageNo)
{
    //选择人脸库，默认最后一个库
    int iLibKey = g_tLastLibInfo.iLibKey;
    if(iLibKey <= 0) {
        printf("人脸底图查询失败：请先查询或添加人脸库.\n\n");
        return -1;
    }
    usleep(20*1000);          //防止操作后立即查询
    //查询条件
    FaceQuery tInfo = {0};
    //必须字段
    tInfo.iSize = sizeof(tInfo);
    tInfo.iChanNo = 0;        //通道号，0 表示第一通道，IPC 只有 1 个通道
    tInfo.iLibKey = iLibKey;
    tInfo.iPageNo = _iPageNo;
    tInfo.iPageCount = FACE_MAX_PAGE_COUNT;
    strncpy_ss(tInfo.cBirthStart, "1970-01-01", sizeof(tInfo.cBirthStart)); //开始出生日期
    strncpy_ss(tInfo.cBirthEnd, "2018-10-16", sizeof(tInfo.cBirthEnd));      //结束出生日期
    //end
    //其余的为非必需字段
    tInfo.iSex = 0;           //性别，0 未知，1 男，2 女
    tInfo.iNation = 0;        //民族，0 未知
    tInfo.iPlace = 0;         //籍贯，0 未知
    tInfo.iCertType = 0;      //证件类型，0 未知，1 二代身份证，2 军官证
    tInfo.iModeling = 0;      //建模状态，0 未知，1 建模成功，2 建模失败，3 未建模
    //end
    //查询
    FaceQueryResult tResult[FACE_MAX_PAGE_COUNT];
    memset(&tResult, 0, sizeof(tResult));
    int iRet = NetClient_FaceConfig(g_iLogonId, FACE_CMD_QUERY, tInfo.iChanNo, &tInfo,
    tInfo.iSize, &tResult, sizeof(FaceQueryResult));
    if(0 != iRet){
        printf("人脸底图查询失败： %d\n", iRet);
    } else {
        printf("人脸底图信息(第一页)-----\n");
        for(int i= 0; i < tResult[0].iPageCount && i < FACE_MAX_PAGE_COUNT; ++i) {
            g_tLastPicInfo = tResult[i].tFace; //保存人脸信息，此处只保存最后一个，便于

```

修改和删除

```

        int iIndex = i + 1;
        printf("序号:%d, 库键值:%d, 人脸键值:%d, 姓名:%s, 出生日期:%s, 建模状态:%d\n", iIndex, tResult[i].tFace.iLibKey, tResult[i].tFace.iFaceKey,
            tResult[i].tFace.cName, tResult[i].tFace.cBirthTime, tResult[i].tFace.iModeling);
    }
    printf("\n");
}
return iRet;
}

```

//人脸底图添加

```

int FacePictureAdd()
{
    //选择人脸库，默认最后一个库
    int iLibKey = g_tLastLibInfo.iLibKey;
    if(iLibKey <= 0) {
        printf("人脸底图添加失败：请先查询或添加人脸库.\n\n");
        return -1;
    }

    FaceEdit tInfo = {0};
    //必须字段
    tInfo.iSize = sizeof(tInfo);
    tInfo.iChanNo = 0;           //通道号，0 表示第一通道，IPC 只有 1 个通道
    tInfo.tFace.iLibKey = iLibKey; //人脸库键值
    tInfo.tFace.iModeling = 1;    //是否建模，1 建模，0 不建模
    tInfo.tFace.iFaceKey = 0;    //人脸底图键值，0 表示添加
    strncpy_ss(tInfo.tFace.cName, "张三", sizeof(tInfo.tFace.cName)); //人脸底图名称
    strncpy_ss(tInfo.tFace.cBirthTime, "2000-01-01", sizeof(tInfo.tFace.cBirthTime)); //出生日期
    strncpy_ss(tInfo.cFacePic, "../face.jpg", sizeof(tInfo.cFacePic)); //底图图片路径
    //end

    //非必需字段
    tInfo.tFace.iSex = 0;        //性别，0 未知，1 男，2 女
    tInfo.tFace.iNation = 0;    //民族，0 未知
    tInfo.tFace.iPlace = 0;    //籍贯，0 未知;
    tInfo.tFace.iCertType = 1;  //证件类型，0 未知，1 二代身份证，2 军官证;
    strncpy_ss(tInfo.tFace.cCertNum, "232321199909090909", sizeof(tInfo.tFace.cCertNum)); //证件号码
    //end
    //普通设备不需要此字段
    tInfo.tFace.iOptType = 1;    //1 添加
}

```

```
//end
FaceReply tReply = {0};
int iRet = NetClient_FaceConfig(g_iLogonId, FACE_CMD_EDIT, tInfo.iChanNo, &tInfo,
tInfo.iSize, &tReply, sizeof(FaceReply));
if(0 != iRet){
    printf("人脸底图添加失败: %d\n", iRet);
} else {
    printf("人脸底图添加结果: %d\n", tReply.iResult);
}

//显示人脸底图添加后结果
FacePictureQuery(0);
return 0;
}

//人脸底图修改
int FacePictureModify()
{
    //此处默认修改最后一张底图
    if (g_tLastPicInfo.iFaceKey <= 0) {
        printf("人脸底图修改失败: 请先查询或添加人脸底图.\n\n");
        return -1;
    }
    FaceEdit tInfo = {0};
    //必须字段
    tInfo.iSize = sizeof(tInfo);
    tInfo.iChanNo = 0; //通道号, 0 表示第一通道, IPC 只有 1 个通道
    tInfo.tFace.iLibKey = g_tLastPicInfo.iLibKey; //人脸库键值
    tInfo.tFace.iFaceKey = g_tLastPicInfo.iFaceKey; //人脸底图键值
    strncpy_ss(tInfo.tFace.cName, "李四", sizeof(tInfo.tFace.cName)); //人脸底图名称
    strncpy_ss(tInfo.tFace.cBirthTime, "2014-04-04", sizeof(tInfo.tFace.cBirthTime)); //出生日期
    //end

    //非必需字段
    tInfo.tFace.iSex = 0; //性别, 0 未知, 1 男, 2 女
    tInfo.tFace.iNation = 0; //民族, 0 未知
    tInfo.tFace.iPlace = 0; //籍贯, 0 未知;
    tInfo.tFace.iCertType = 1; //证件类型, 0 未知, 1 二代身份证, 2 军官证;
    strncpy_ss(tInfo.tFace.cCertNum, "232321201404040404", sizeof(tInfo.tFace.cCertNum));
    //证件号码
    //end
    //普通设备不需要此字段
    tInfo.tFace.iOptType = 2; //2 修改
```

```
//end

FaceReply tReply = {0};
int iRet = NetClient_FaceConfig(g_iLogonId, FACE_CMD_EDIT, tInfo.iChanNo, &tInfo,
tInfo.iSize, &tReply, sizeof(FaceReply));
if(0 != iRet){
    printf("人脸底图修改失败: %d\n", iRet);
} else {
    printf("人脸底图修改结果: %d\n", tReply.iResult);
}
//显示人脸底图修改后结果
FacePictureQuery(0);
return 0;
}

//人脸底图删除
int FacePictureDelete()
{
    //此处默认删除最后一张底图
    if(g_tLastPicInfo.iFaceKey <= 0) {
        printf("人脸底图删除失败: 请先查询或添加人脸底图.\n\n");
        return -1;
    }

    FaceDelete tInfo = {0};
    tInfo.iSize = sizeof(tInfo);
    tInfo.iChanNo = 0;    //通道号, 0 表示第一通道, IPC 只有 1 个通道
    tInfo.iLibKey = g_tLastPicInfo.iLibKey;
    tInfo.iFaceKey = g_tLastPicInfo.iFaceKey;

    FaceReply tReply = {0};
    int iRet = NetClient_FaceConfig(g_iLogonId, FACE_CMD_DELETE, tInfo.iChanNo,
&tInfo, tInfo.iSize, &tReply, sizeof(FaceReply));
    if(0 != iRet){
        printf("人脸底图删除失败: %d\n", iRet);
    } else {
        printf("人脸底图删除结果: %d\n", tReply.iResult);
    }

    //显示人脸底图删除后结果
    FacePictureQuery(0);
    return iRet;
}
```

```
int FaceDetectionEnable()
{
    AnyScene tParam = {0};
    tParam.iBufSize = sizeof(AnyScene);
    tParam.iSceneID = 0;    //场景号 0-15
    tParam.iDevType = 1;    //0-IPC, 1-NVR
    int iBytesReturned = 0;
    int iRet = NetClient_GetDevConfig(g_iLogonId, NET_CLIENT_ANYSCENE, 0, &tParam,
    sizeof(tParam), &iBytesReturned);
    if (iRet >= 0)
    {
        tParam.iArithmetic = 1<<2;//人脸检测算法开启
        tParam.iDevType = 1;    //0-IPC, 1-NVR
        iRet =NetClient_SetDevConfig(g_iLogonId, NET_CLIENT_ANYSCENE, 0, &tParam,
        sizeof(tParam));
        if (iRet >= 0)
        {
            printf("人脸检测算法开启成功.\n");
        }
        else
        {
            printf("NetClient_SetDevConfig  NET_CLIENT_ANYSCENE failed.\n");
        }
    }
    else
    {
        printf("NetClient_GetDevConfig  NET_CLIENT_ANYSCENE failed.\n");
    }

    return iRet;
}
```

```
int FaceRecognitionEnable()
{
    FaceDetectArithmetic fda = {0};
    fda.iBufSize = sizeof(FaceDetectArithmetic);
    fda.iDevType = 1;    //0-IPC, 1-NVR
    int iByteReturn = 0;
    int iRet = -1;
    iRet=NetClient_GetDevConfig(g_iLogonId,NET_CLIENT_FACE_DETECT_ARITHMETI
    C, 0, &fda, sizeof(FaceDetectArithmetic), &iByteReturn);
    if (iRet < 0)
    {
        printf("NetClient_GetDevConfig  NET_CLIENT_FACE_DETECT_ARITHMETIC
```

```
        failed.\n");
        return -1;
    }

    if (fda.iMinSize>=fda.iMaxSize)
    {
        fda.iMaxSize = fda.iMinSize+1;//调整参数有效性(规避)
    }

    fda.iDevType = 1; //0-IPC, 1-NVR
    fda.iDentification = 2; //人脸识别开关 0-不支持, 1-关闭, 2-开启
    iRet=NetClient_SetDevConfig(g_iLogonId,NET_CLIENT_FACE_DETECT_ARITHMETIC, 0, &fda, sizeof(FaceDetectArithmetic));
    if (iRet >= 0)
    {
        printf("NetClient_SetDevConfig->NET_CLIENT_FACE_DETECT_ARITHMETIC success.\n");
    }
    else
    {
        printf("NetClient_SetDevConfig->NET_CLIENT_FACE_DETECT_ARITHMETIC failed.\n");
    }
    return iRet;
}

int SetDetectParam()
{
    FaceDetectArithmetic fda = {0};
    fda.iBufSize = sizeof(FaceDetectArithmetic);
    fda.iDevType = 1; //0-IPC, 1-NVR
    int iByteReturn = 0;
    int iRet = -1;
    iRet=NetClient_GetDevConfig(g_iLogonId,NET_CLIENT_FACE_DETECT_ARITHMETIC, 0, &fda, sizeof(FaceDetectArithmetic), &iByteReturn);
    if (iRet < 0)
    {
        printf("NetClient_GetDevConfig NET_CLIENT_FACE_DETECT_ARITHMETIC failed.\n");
        return -1;
    }

    if (fda.iMinSize>=fda.iMaxSize)
    {
        fda.iMaxSize = fda.iMinSize+1;//调整参数有效性(规避)
```



```
}
fda.iPushMode = 2;
fda.iSnapTimes = 1;
iRet=NetClient_SetDevConfig(g_iLogonId,NET_CLIENT_FACE_DETECT_ARITHMETI
C, 0, &fda, sizeof(FaceDetectArithmetic));
if (iRet >= 0)
{
    printf("NetClient_SetDevConfig->NET_CLIENT_FACE_DETECT_ARITHMETIC
success.\n");
}
else
{
    printf("NetClient_SetDevConfig->NET_CLIENT_FACE_DETECT_ARITHMETIC
failed.\n");
}
return iRet;
}

int SetPicStreamUploadParam()
{
    //设置背景图质量和上传使能
    PicStreamUploadParam tInfo = {0};
    tInfo.iSize= sizeof(PicStreamUploadParam);
    tInfo.iSceneId = 0;
    tInfo.iPicType = 0;//0-背景图
    int iRet = -1;
    iRet=NetClient_VCAGetConfig(g_iLogonId,VCA_CMD_PICSTREAM_UPLOADPARAM,
0, &tInfo, sizeof(PicStreamUploadParam));
    if (iRet < 0)
    {
        printf("NetClient_GetDevConfig->VCA_CMD_PICSTREAM_UPLOADPARAM
failed.\n");
    }
    else
    {
        tInfo.iSnapEnable = 1;
        tInfo.iQpvalue = 80;
        tInfo.iIsOsd = 1;
        iRet=NetClient_VCASetConfig(g_iLogonId,VCA_CMD_PICSTREAM_UPLOADPAR
AM, 0, &tInfo, sizeof(PicStreamUploadParam));
        if (iRet >= 0)
        {
            printf("NetClient_VCASetConfig VCA_CMD_PICSTREAM_UPLOADPARAM
success.\n");
        }
    }
}
```

```
    }
    else
    {
        printf("NetClient_VCASetConfig VCA_CMD_PICSTREAM_UPLOADPARAM
        failed.\n");
    }
}

//设置特写图质量和上传使能
memset(&tInfo, 0, sizeof(tInfo));
tInfo.iSize= sizeof(PicStreamUploadParam);
tInfo.iSceneId = 0;
tInfo.iPicType = 1;//1-特写图

iRet=NetClient_VCAGetConfig(g_iLogonId,VCA_CMD_PICSTREAM_UPLOADPARAM,
0, &tInfo, sizeof(PicStreamUploadParam));
if (iRet < 0)
{
    printf("NetClient_GetDevConfig->VCA_CMD_PICSTREAM_UPLOADPARAM
    failed.\n");
}
else
{
    tInfo.iSnapEnable = 1;
    tInfo.iQpvalue = 30;
    iRet=NetClient_VCASetConfig(g_iLogonId,VCA_CMD_PICSTREAM_UPLOADPAR
    AM, 0, &tInfo, sizeof(PicStreamUploadParam));
    if (iRet < 0)
    {
        printf("NetClient_VCASetConfig VCA_CMD_PICSTREAM_UPLOADPARAM
        failed.\n");
    }
}
return iRet;
}

int SetFaceScheduleEnable()
{
    //选择人脸库，默认最后一个库
    int iLibKey = g_tLastLibInfo.iLibKey;
    if(iLibKey <= 0) {
        printf("人脸布防设置失败：请先查询或添加人脸库.\n\n");
        return -1;
    }
}
```

```

fprintf(stderr, "Please input MainAlarmType(20-FaceRecognition 21-NVRVCA other-NVRVCA):\n");
char cMainAlarmType[16] = {0};
gets_s(cMainAlarmType, 16);
int iMainAlarmType = atoi(cMainAlarmType);
char cSubAlarmType[16] = {0};
int iSubAlarmType = 0;
if (ALARM_TYPE_FACE_IDENT == iMainAlarmType)
{
    fprintf(stderr, "Please input SubAlarmTypeWhite list(0-Blacklist 1-Whitelist other-Blacklist):\n");
    gets_s(cSubAlarmType, 16);
    iSubAlarmType = atoi(cSubAlarmType);
    if (1 == iSubAlarmType) //白名单和库没有关系，所以 iLibKey 赋值为 0
    {
        iLibKey = 0;
    }
}
else
{
    iMainAlarmType = ALARM_TYPE_NVR_VCA;
    fprintf(stderr, "Please input SubAlarmTypeWhite list(0:Face detection. 1:Face recognition - comparison. 2:Face recognition - stranger. 3:Face recognition - frequency. 4:Face recognition - retention. other-Face recognition - comparison.): \n");
    gets_s(cSubAlarmType, 16);
    iSubAlarmType = atoi(cSubAlarmType);
}
usleep(20*1000); //防止操作后立即查询
//获取布防使能
int iChanNo = 0; //通道号，0 表示第一通道
TAlarmScheEnableParam tSchEnabel = {0};
tSchEnabel.iBuffSize = sizeof(tSchEnabel);
tSchEnabel.iSceneID = 0;
tSchEnabel.iParam1 = iLibKey; //人脸库 ID
tSchEnabel.iParam2 = iSubAlarmType; //报警类型
_iAlarmType=ALARM_TYPE_FACE_IDENT 20 时，iParam2:0-黑名单，1-白名单；报警类型 _iAlarmType=ALARM_TYPE_NVR_VCA 21 时，表 NVR 的智能分析,iParam2: 0-人脸检测 1-人脸识别-比对 2-人脸识别-陌生人 3-人脸识别-频次 4-人脸识别-滞留
int iRet = NetClient_GetAlarmConfig(g_iLogonId, iChanNo, iMainAlarmType, CMD_GET_ALARMSCH_ENABLE, &tSchEnabel);
if (iRet >= 0)
{
    //设置布防使能
    tSchEnabel.iEnable = 1;
}

```

```
iRet = NetClient_SetAlarmConfig(g_iLogonId, iChanNo, ALARM_TYPE_NVR_VCA,
CMD_ALARMSCH_ENABLE_EX, &tSchEnabel);
if (iRet >= 0)
{
    printf("NetClient_SetAlarmConfig  CMD_ALARMSCH_ENABLE_EX
success.\n");
}
else
{
    printf("NetClient_SetAlarmConfig  CMD_ALARMSCH_ENABLE_EX
failed.\n");
}
}
else
{
    printf("NetClient_GetAlarmConfig  CMD_GET_ALARMSCH_ENABLE failed.\n");
}

return iRet;
}

int GetFaceScheduleEnable()
{
    //选择人脸库，默认最后一个库
    int iLibKey = g_tLastLibInfo.iLibKey;
    if(iLibKey <= 0) {
        printf("人脸布防获取失败：请先查询或添加人脸库.\n\n");
        return -1;
    }

    fprintf(stderr, "Please input MainAlarmType(20-FaceRecognition 21-NVRVCA
other-NVRVCA):\n");
    char cMainAlarmType[16] = {0};
    gets_s(cMainAlarmType, 16);
    int iMainAlarmType = atoi(cMainAlarmType);
    char cSubAlarmType[16] = {0};
    int iSubAlarmType = 0;
    if (ALARM_TYPE_FACE_IDENT == iMainAlarmType)
    {
        fprintf(stderr, "Please input SubAlarmTypWhite liste(0-Blacklist 1-Whitelist
other-Blacklist):\n");
        gets_s(cSubAlarmType, 16);
        iSubAlarmType = atoi(cSubAlarmType);
        if (1 == iSubAlarmType)//白名单和库没有关系，所以 iLibKey 赋值为 0
```

```

        {
            iLibKey = 0;
        }
    }
else
{
    iMainAlarmType = ALARM_TYPE_NVR_VCA;
    fprintf(stderr, "Please input SubAlarmTypeWhite list(0:Face detection. 1:Face
    recognition - comparison. 2:Face recognition - stranger. 3:Face recognition - frequency.
    4:Face recognition - retention. other-Face recognition - comparison.):\\n");
    gets_s(cSubAlarmType, 16);
    iSubAlarmType = atoi(cSubAlarmType);
}

usleep(20*1000);          //防止操作后立即查询
//获取布防使能
int iChanNo = 0;          //通道号, 0 表示第一通道
TAlarmScheEnableParam tSchEnabel = {0};
tSchEnabel.iBuffSize = sizeof(tSchEnabel);
tSchEnabel.iSceneID = 0;
tSchEnabel.iParam1 = iLibKey;//人脸库 ID
tSchEnabel.iParam2 = iSubAlarmType;//报警类型_iAlarmType=
ALARM_TYPE_FACE_IDENT 20 时, iParam2:0-黑名单, 1-白名单; 报警类型
_iAlarmType=ALARM_TYPE_NVR_VCA 21 时, 表 NVR 的智能分析,iParam2: 0-人脸检
测 1-人脸识别-比对 2-人脸识别-陌生人 3-人脸识别-频次 4-人脸识别-滞留
int iRet = NetClient_GetAlarmConfig(g_iLogonId, iChanNo, iMainAlarmType,
CMD_GET_ALARMSCH_ENABLE, &tSchEnabel);
if (iRet >= 0)
{
    printf("NetClient_GetAlarmConfig  CMD_GET_ALARMSCH_ENABLE success.
    Enable(%d).\\n", tSchEnabel.iEnable);
}
else
{
    printf("NetClient_GetAlarmConfig  CMD_GET_ALARMSCH_ENABLE failed.\\n");
}
return iRet;
}

int SetFaceSchedule()
{
    //选择人脸库, 默认最后一个库
    int iLibKey = g_tLastLibInfo.iLibKey;
    if(iLibKey <= 0) {

```

```

        printf("人脸布防设置失败：请先查询或添加人脸库.\n\n");
        return -1;
    }

    fprintf(stderr, "Please input weekday num(0-6 Sunday-Saturday):\n");
    char cWeekday[16] = {0};
    gets_s(cWeekday, 16);
    int iWeekday = atoi(cWeekday);
    if (iWeekday < 0 || iWeekday > 6)
    {
        iWeekday = 0;
    }

    fprintf(stderr, "Please input enable (0:disable 1:enable other:enable):\n");
    char cEnable[16] = {0};
    gets_s(cEnable, 16);
    int iEnable = atoi(cEnable);
    if (0 != iEnable)
    {
        iEnable = 1;
    }

    usleep(20*1000);          //防止操作后立即查询
    int iChannelNo = 0;
    //布防模板
    TAlarmScheduleParam tSchParam = {0};
    tSchParam.iBuffSize = sizeof(tSchParam);
    tSchParam.iWeekday = iWeekday;
    tSchParam.iSceneID = 0;    //场景号
    tSchParam.iParam1 = iLibKey;//人脸库 ID 白名单和库没有关系，所以当_iAlarmType=20
    iParam2=1 时，iLibKey 赋值为 0
    tSchParam.iParam2 = 0; //报警类型 _iAlarmType= ALARM_TYPE_FACE_IDENT 20 时，
    iParam2:0-黑名单，1-白名单；报警类型 _iAlarmType=ALARM_TYPE_NVR_VCA 21 时，
    表 NVR 的智能分析,iParam2: 0-人脸检测 1-人脸识别-比对 2-人脸识别-陌生人 3-人
    脸识别-频次 4-人脸识别-滞留
    for(int i = 0; i < MAX_TIMESEGMENT; i++)
    {
        if (0 == i)//此处默认只设置第一个时间段
        {
            NVS_SCHEDULETIME &tSeg = tSchParam.timeSeg[iWeekday][i];
            tSeg.iRecordMode = iEnable;//时间段使能
            if (tSeg.iRecordMode)
            {
                tSeg.iStartHour = 0;
                tSeg.iStartMin = 0;
            }
        }
    }

```

```
        tSeg.iStopHour = 23;
        tSeg.iStopMin = 59;
    }
}

int iRet = NetClient_SetAlarmConfig(g_iLogonId, iChannelNo,
ALARM_TYPE_FACE_IDENT, CMD_SETALARMSCHEDULE, &tSchParam);
if (iRet >= 0)
{
    printf("NetClient_SetAlarmConfig CMD_SETALARMSCHEDULE success.\n\n");
}
else
{
    printf("NetClient_SetAlarmConfig CMD_SETALARMSCHEDULE failed.\n\n");
}

return iRet;
}

int GetFaceSchedule()
{
    //选择人脸库，默认最后一个库
    int iLibKey = g_tLastLibInfo.iLibKey;
    if(iLibKey <= 0) {
        printf("人脸布防获取失败：请先查询或添加人脸库.\n\n");
        return -1;
    }

    fprintf(stderr, "Please input weekday num(0-6 Sunday-Saturday):\n");
    char cWeekday[16] = {0};
    gets_s(cWeekday, 16);
    int iWeekday = atoi(cWeekday);
    if (iWeekday < 0 || iWeekday > 6)
    {
        iWeekday = 0;
    }

    usleep(20*1000);    //防止操作后立即查询
    int iChannelNo = 0;

    //布防模板
    TAlarmScheduleParam tSchParam = {0};
    tSchParam.iBuffSize = sizeof(tSchParam);
    tSchParam.iWeekday = iWeekday;
```

```

tSchParam.iSceneID = 0;    //场景号
tSchParam.iParam1 = iLibKey;//人脸库 ID _iAlarmType=20, iParam2=1 时, 和库 ID 无关,
iLibKey 赋值为 0; _iAlarmType=21, iParam2=2 时, 和库 ID 无关, iLibKey 赋值为 0;
tSchParam.iParam2 = 0; //报警类型 _iAlarmType= ALARM_TYPE_FACE_IDENT 20 时,
iParam2:0-黑名单, 1-白名单; 报警类型 _iAlarmType=ALARM_TYPE_NVR_VCA 21 时,
表 NVR 的智能分析,iParam2: 0-人脸检测 1-人脸识别-比对 2-人脸识别-陌生人 3-人
脸识别-频次 4-人脸识别-滞留
int iRet=NetClient_GetAlarmConfig(g_iLogonId, iChannelNo,
ALARM_TYPE_FACE_IDENT, CMD_GET_ALARMSCHEDULE, &tSchParam);
if (iRet >= 0)
{
    for(int i = 0; i < MAX_TIMESEGMENT; i++)
    {
        NVS_SCHEDTIME &tSeg = tSchParam.timeSeg[iWeekday][i];

        if (tSeg.iRecordMode)
        {
            printf("Time%d enable(%d) (%02d:%02d:00 %02d:%02d:00 ).\n\n", i+1,
            tSeg.iRecordMode, tSeg.iStartHour, tSeg.iStartMin, tSeg.iStopHour,
            tSeg.iStopMin);
        }
    }
}
return iRet;
}

int SetFaceAlarm()
{
    //选择人脸库, 默认最后一个库
    int iLibKey = g_tLastLibInfo.iLibKey;
    if(iLibKey <= 0) {
        printf("人脸布防获取失败: 请先查询或添加人脸库.\n\n");
        return -1;
    }
    usleep(20*1000);    //防止操作后立即查询
    int iChannelNo = 0;    //通道号, 0 表示第一通道
    FaceAlarmParam tInfo = {0};
    tInfo.iSize = sizeof(tInfo);
    tInfo.iChanNo = iChannelNo;
    tInfo.iAlarmType = ALARM_TYPE_NVR_VCA;//报警类型, 20-人脸检测 21-人脸识别
    tInfo.iParam1 = 1; //报警类型 _iAlarmType=ALARM_TYPE_NVR_VCA 21 时, 表 NVR
    的智能分析,iParam1: 0-人脸检测 1-人脸识别-比对 2-人脸识别-陌生人 3-人脸识别-
    频次 4-人脸识别-滞留
    tInfo.iEnable = 1; //是否启用算法 0-不使能 1-使能

```



```

tInfo.iParam2 = 0; //当 iAlarmType=21, iParam1=3,4 时, iParam2 有效, 代表时间
tInfo.iParam3 = 0; //当 iAlarmType=21, iParam2=3 时, iParam3 有效, 代表频次
tInfo.iSimilar = 75; // 相似度
tInfo.iDevType = 1; //0-IPC 1-NVR
tInfo.iRecognition = 2; //0-不支持, 1-不上传, 2-上传
sprintf(tInfo.cLibkey, "%d", iLibKey);
tInfo.iLibEnable = 1; //0 不使能, 1 使能

int iRet=NetClient_SetAlarmConfig(g_iLogonId, iChannelNo, ALARM_TYPE_NVR_VCA,
CMD_ALARM_FACE_PARAM, &tInfo);
if (iRet >= 0)
{
    printf("NetClient_SetAlarmConfig CMD_ALARM_FACE_PARAM success.\n");
}
else
{
    printf("NetClient_SetAlarmConfig CMD_ALARM_FACE_PARAM failed.\n");
}

return iRet;
}

int GetFaceAlarm()
{
    fprintf(stderr, "Please input subalarm type (1:comparison. 2:stranger. 3:frequency. 4:retention.
other:comparison.): \n");
    char cType[16] = {0};
    gets_s(cType, 16);
    int iType = atoi(cType);
    if (iType <= 0 || iType > 4)
    {
        iType = 1;
    }

    int iChannelNo = 0; //通道号, 0 表示第一通道
    FaceAlarmParamIn tInfo = {0};
    tInfo.iSize = sizeof(tInfo);
    tInfo.iType = iType; //NVR 本地智能分析类型 1-人脸识别-比对 2-人脸识别-陌生人 3-
人脸识别-频次 4-人脸识别-滞留
    FaceAlarmParam tFaceAlarmInfo[FACE_MAX_KEY_COUNT];
    memset(tFaceAlarmInfo, 0, sizeof(tFaceAlarmInfo));

    int iRet = NetClient_FaceConfig(g_iLogonId, FACE_CMD_ALARM_PARAM, iChannelNo,
&tInfo, sizeof(FaceAlarmParamIn), &tFaceAlarmInfo, sizeof(FaceAlarmParam));

```

```
if (iRet >= 0)
{
    for (int i = 0; i < FACE_MAX_KEY_COUNT; ++i)
    {
        if (tFaceAlarmInfo[i].iSize <= 0)
        {
            break;
        }
        if (0 == i)
        {
            printf("iParam2(%d) iParam3(%d) iSimilar(%d) iRecognition(%d) iEnable(%d).\n",
                tFaceAlarmInfo[i].iParam2, tFaceAlarmInfo[i].iParam3,
                tFaceAlarmInfo[i].iSimilar, tFaceAlarmInfo[i].iRecognition,
                tFaceAlarmInfo[i].iEnable);
        }
        printf("%d:Libkey(%d)iLibEnable(%d).\n", i+1, ToIntDef(tFaceAlarmInfo[i].cLibkey), tFaceAlarmInfo[i].iLibEnable);
    }
}
else
{
    printf("NetClient_GetAlarmConfig CMD_ALARM_FACE_PARAM failed.\n");
}

return iRet;
}

int SetVcaStatue(int _iStatus)
{
    int iChanNo = 0;    //通道号, 0 表示第一通道, IPC 只有 1 个通道
    VCASuspend tInfo = {0};
    tInfo.iStatus = _iStatus;
    tInfo.iDevType = 1;
    return NetClient_SetDevConfig(g_iLogonId, NET_CLIENT_VCA_SUSPEND, iChanNo, &tInfo, sizeof(tInfo));
}

int SavePicture(char* _pcFileName, char* _pcData, int _iLen)
{
    if (NULL == _pcFileName || NULL == _pcData || _iLen <= 0)
    {
        return -1;
    }
}
```

```
FILE* pFile = fopen(_pcFileName, "wb");
if (NULL == pFile)
{
    return -1;
}
fwrite(_pcData, _iLen, 1, pFile);
fclose(pFile);
pFile = NULL;
return 0;
}

//人脸图片流回调
int __stdcall CallBack_PicStreamInfo(unsigned int _uiRecvID, long _lCommand, void* _lpInfo,
int _iBufLen, void* _pvUser)
{
    if (NULL == _lpInfo || NET_PICSTREAM_CMD_FACE != _lCommand)
    {
        return -1;
    }
    FacePicStream* pFace = (FacePicStream*)_lpInfo;

    //保存全景图
    PicData tFullData;
    memset(&tFullData, 0, sizeof(tFullData));
    if (NULL != pFace->ptFullData)
    {
        memcpy(&tFullData, pFace->ptFullData, min(pFace->iSizeOfFull,
(int)sizeof(PicData)));
        char cFullPicName[256] = {0};
        sprintf(cFullPicName, ".//Pic//FullPic_Time(2%03d%02d%02d%02d%02d%02d).jpg",
tFullData.tPicTime.uiYear, tFullData.tPicTime.uiMonth, tFullData.tPicTime.uiDay,
tFullData.tPicTime.uiHour, tFullData.tPicTime.uiMinute,
tFullData.tPicTime.uiSecondsr, tFullData.tPicTime.uiMilliseconds);
        SavePicture(cFullPicName, tFullData.pcPicData, tFullData.iDataLen);
    }

    //保存人脸图和地图
    for (int i = 0; i < pFace->iFaceCount && i < MAX_FACE_PICTURE_COUNT; ++i)
    {
        FacePicData tFaceData = {0};
        memcpy(&tFaceData, pFace->ptFaceData[i], min(pFace->iSizeOfFace,
(int)sizeof(FacePicData)));
        char cFacePicName[256] = {0};
        sprintf(cFacePicName, ".//Pic//FacePic_No%d_Time(2%03d%02d%02d%02d%02d%02d%02d%02d%02d%02d%02d).jpg",
tFaceData.tPicTime.uiYear, tFaceData.tPicTime.uiMonth, tFaceData.tPicTime.uiDay,
tFaceData.tPicTime.uiHour, tFaceData.tPicTime.uiMinute, tFaceData.tPicTime.uiSecondsr,
tFaceData.tPicTime.uiMilliseconds, tFaceData.tPicTime.uiMicroseconds, tFaceData.tPicTime.uiNanoseconds);
        SavePicture(cFacePicName, tFaceData.pcPicData, tFaceData.iDataLen);
    }
}
```

```

        d%d).jpg", i, tFullData.tPicTime.uiYear, tFullData.tPicTime.uiMonth,
        tFullData.tPicTime.uiDay, tFullData.tPicTime.uiHour, tFullData.tPicTime.uiMinute,
        tFullData.tPicTime.uiSecondsr, tFullData.tPicTime.uiMilliseconds);
//保存人脸图
SavePicture(cFacePicName, tFaceData.pcPicData, tFaceData.iDataLen);
//底图
if (1 == tFaceData.iAframType)
{
    char cNegPicName[256] = {0};
    sprintf(cNegPicName, "../Pic/NegPic_No%d_Time(2%03d%02d%02d%02d%02d%02d%02d).jpg", i, tFullData.tPicTime.uiYear, tFullData.tPicTime.uiMonth,
        tFullData.tPicTime.uiDay, tFullData.tPicTime.uiHour,
        tFullData.tPicTime.uiMinute, tFullData.tPicTime.uiSecondsr,
        tFullData.tPicTime.uiMilliseconds);
    SavePicture(cFacePicName, tFaceData.pcNegPicData, tFaceData.iNegPicLen);
}
}
return 0;
}

//开启图片流
int StartSnap()
{
    NetPicPara tNetPicParam = {0};
    tNetPicParam.iStructLen = sizeof(tNetPicParam);
    tNetPicParam.iChannelNo = 0;
    tNetPicParam.cbkPicStreamNotify = CallBack_PicStreamInfo; //抓拍回调函数
    tNetPicParam.pvUser = NULL;

    unsigned int iConnectID = -1;
    int iRet = NetClient_StartRecvNetPicStream(g_iLogonId, &tNetPicParam,
        tNetPicParam.iStructLen, &iConnectID);
    if (iRet < 0) {
        g_iConnectId = -1;
        fprintf(stderr, "StartRecvNetPicStream Failed!");
    } else {
        g_iConnectId = (int)iConnectID;
        fprintf(stderr, "StartRecvNetPicStream Success! ConnectID(%d)\n\n", g_iConnectId);
    }
    return 0;
};

int main(int argc, char *argv[])
{

```

```
//加载库
LoadLib();
//注册主回调函数
NetClient_SetNotifyFunction_V4(Notify_Main, NULL, NULL, NULL, NULL);
//启动 SDK
NetClient_Startup_V4(0, 0, 0);
//登录设备
if (LogonDevice() < 0)
{
    fprintf(stderr, "[main] LogonDevice failed.\n");
    getchar();
    return -1;
}

//开启图片流
#ifdef __WIN__
    _mkdir(".\\Pic");
#else
    mkdir("./Pic", S_IRWXU | S_IRWXG | S_IROTH | S_IXOTH);
#endif
StartSnap();
while (1)
{
    fprintf(stderr,
        "[0]Quit    [1]QueryFaceLibrary    [2]AddFaceLibrary\n"
        "[3]ModifyFaceLibrary    [4]DeleteFaceLibrary    [5]QueryFaceBasemap \n"
        "[6]AddFaceBasemap    [7]ModifyFaceBasemap    [8]DeleteFaceBasemap\n"
        "[9]EnableFaceDetection    [10]EnableFaceRecognition\n"
        "[11]SetFaceDetectionParameters\n"    "[12]SetFaceScheduleEnable\n"
        "[13]GetFaceScheduleEnable    [14]SetFaceScheduleTemplate\n"
        "[15]GetFaceScheduleTemplate    [16]SetFaceRecognitionAlarm\n"
        "[17]GetFaceRecognitionAlarm\n");
    char cTemp[16] = {0};
    gets_s(cTemp, 16);
    if (0 == strlen(cTemp))
    {
        continue;
    }

    int iOpt = atoi(cTemp);
    if (0 == iOpt)
    {
        break;
    }
}
```

```
else if (1 == iOpt) //查询人脸库
{
    FaceLibraryQuery();
}
else if (2 == iOpt) //添加人脸库
{
    FaceLibraryAdd();
}
else if (3 == iOpt) //修改人脸库
{
    FaceLibraryModify();
}
else if (4 == iOpt) //删除人脸库
{
    FaceLibraryDelete();
}
else if (5 == iOpt) //查询人脸底图
{
    FacePictureQuery(0);          //此处只查询第一页
}
else if (6 == iOpt) //添加人脸底图
{
    //添加人脸需要暂停智能分析
    fprintf(stderr, "[main] 请等待智能分析暂停结果!\n");
    g_iVcaStatus = 0;
    SetVcaStatue(VCA_SUSPEND_STATUS_PAUSE); //暂停智能分析
    getchar();
    if (VCA_SUSPEND_RESULT_SUCCESS != g_iVcaStatus) {
        fprintf(stderr, "[main] 智能分析暂停失败，退出添加!\n");
        getchar();
        continue;
    }
    FacePictureAdd(); //暂停成功后，可以多次添加
    SetVcaStatue(VCA_SUSPEND_STATUS_RESUME); //添加完人脸需要恢复智能分析
}
else if (7 == iOpt) //修改人脸底图
{
    FacePictureModify();
}
else if (8 == iOpt) //删除人脸底图
{
    FacePictureDelete();
}
```

```
else if (9 == iOpt) //开启人脸检测
{
    FaceDetectionEnable();
}
else if (10 == iOpt)//开启人脸识别
{
    SetVcaStatue(VCA_SUSPEND_STATUS_PAUSE); //暂停智能分析
    getchar();
    if (VCA_SUSPEND_RESULT_SUCCESS != g_iVcaStatus) {
        fprintf(stderr, "[main] 智能分析暂停失败!\n");
        getchar();
        continue;
    }
    FaceRecognitionEnable();//需要暂停智能分析
    SetVcaStatue(VCA_SUSPEND_STATUS_RESUME);//设置完需要恢复智能分析
}
else if (11 == iOpt)//设置人脸检测参数
{
    SetVcaStatue(VCA_SUSPEND_STATUS_PAUSE); //暂停智能分析
    getchar();
    if (VCA_SUSPEND_RESULT_SUCCESS != g_iVcaStatus) {
        fprintf(stderr, "[main] 智能分析暂停失败!\n");
        getchar();
        continue;
    }
    SetDetectParam();//人脸检测相关参数(需要暂停智能分析)
    SetVcaStatue(VCA_SUSPEND_STATUS_RESUME);//设置完需要恢复智能分析
    SetPicStreamUploadParam();//抓拍图片质量、上传相关参数
}
else if (12 == iOpt) //设置人脸布防使能
{
    SetFaceScheduleEnable();
}
else if (13 == iOpt) //获取人脸布防使能
{
    GetFaceScheduleEnable();
}
else if (14 == iOpt) //设置人脸布防模板
{
    SetFaceSchedule();
}
else if (15 == iOpt) //获取人脸布防模板
```

```

    {
        GetFaceSchedule();
    }
    else if (16 == iOpt) //设置人脸识别报警
    {
        SetFaceAlarm();
    }
    else if (17 == iOpt) //获取人脸识别报警
    {
        GetFaceAlarm();
    }
}

//注销用户，释放 SDK 资源
NetClient_Logoff(g_iLogonId);
NetClient_Cleanup();
return 0;
}

```

4.7 人脸检索

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include "NetSdk.h"

int g_iLogonId = -1;
int g_iConnectId = -1;
int g_iVcaStatus = 0;
FaceLibInfo g_tLastLibInfo = {0}; //保存最后一个库信息，便于修改和删除
FaceInfo g_tLastPicInfo = {0}; //保存最后一个人脸信息，便于修改和删除

#define VCA_SUSPEND_STATUS_PAUSE    0        //暂停智能分析
#define VCA_SUSPEND_STATUS_RESUME    1        //恢复智能分析
#define VCA_SUSPEND_RESULT_SUCCESS    1        //智能分析暂停成功
#define VCA_SUSPEND_RESULT_CONFIGING    2        //智能分析暂停失败，正在设置，不可设参
#define MAX_LOGON_WAIT_TIME          5 * 1000

#ifndef __WIN__
#include <unistd.h>

```



```
#include <sys/types.h>
#include <sys/stat.h>
#define min(a,b)  (((a) < (b)) ? (a) : (b))
int gets_s(char *_pcStr, int _iCount)
{
    if (_pcStr == NULL)
        return -1;

    char* pcRet = fgets(_pcStr, _iCount, stdin);
    size_t uiLen = strlen(_pcStr);
    if (pcRet == NULL || uiLen == 0)
        return -2;

    if (_iCount - 1 > (int)uiLen || _pcStr[uiLen-1] == '\n')
    {
        _pcStr[uiLen-1] = '\0';
    }

    stdin->_IO_read_ptr = stdin->_IO_read_end;
    return 0;
}
#else
#include<direct.h>
void usleep(int _iMicroSecond)
{
    if (_iMicroSecond < 1000 && _iMicroSecond != 0)
    {
        _iMicroSecond = 1000;
    }
    Sleep(_iMicroSecond/1000);
}
#endif

int ToIntDef(const char* _pstrFrom, int _iDef=0)
{
    if (NULL == _pstrFrom || strlen(_pstrFrom) <= 0)
    {
        return _iDef;
    }
    size_t i=0;
    if (0 == memcmp("-",_pstrFrom,1))
    {
        i++;
    }
}
```

```
for (; i<strlen(_pstrFrom); i++)
{
    if (!isdigit(_pstrFrom[i]))
    {
        return _iDef;
    }
}

int iFlag = 10;
if(NULL != strstr(_pstrFrom, "0x") || NULL != strstr(_pstrFrom, "0X"))
{
    iFlag = 16;
}
//return atoi(_pstrFrom);--此方式无符号数有可能溢出
return (int)strtoul(_pstrFrom, 0, iFlag);
}
```

```
unsigned long netsdk_get_tick_count()
{
    unsigned long    currentTime = 0;
#ifdef WIN32
    struct timeval current;
    gettimeofday(&current, NULL);
    currentTime = current.tv_sec * 1000 + current.tv_usec/1000;
#else
    currentTime = GetTickCount();
#endif
    return currentTime;
}
```

```
int LogonDevice()
{
    LogonPara tInfo = {0};
    tInfo.iSize = sizeof(LogonPara);
    tInfo.iNvsPort = 3000;

    fprintf(stderr, "Please input server IP: ");
    gets_s(tInfo.cNvsIP, 32);
    if (0 == strlen(tInfo.cNvsIP)) {
        strncpy_ss(tInfo.cNvsIP, "192.168.1.2", sizeof(tInfo.cNvsIP));
    }

    fprintf(stderr, "Please input user name: ");
    gets_s(tInfo.cUserName, 16);
}
```

```
if (0 == strlen(tInfo.cUserName)) {
    strncpy_ss(tInfo.cUserName, "Admin", sizeof(tInfo.cUserName));
}

fprintf(stderr, "Please input password: ");
gets_s(tInfo.cUserPwd, 16);
if (0 == strlen(tInfo.cUserPwd)) {
    strncpy_ss(tInfo.cUserPwd, "Admin", sizeof(tInfo.cUserPwd));
}

fprintf(stderr, "Please input port: ");
char cPort[16] = {0};
gets_s(cPort, 16);
tInfo.iNvsPort = atoi(cPort);
if (tInfo.iNvsPort <= 0) {
    tInfo.iNvsPort = 3000;
}

int iLogonId = NetClient_Logon_V4(SERVER_NORMAL, &tInfo, tInfo.iSize);
if(iLogonId < 0)
{
    fprintf(stderr, "[LogonDevice] NetClient_Logon_V4(%s) failed, %d\n", tInfo.cNvsIP,
        iLogonId);
    return -1;
}
int iTimes = 0;
while (LOGON_SUCCESS != NetClient_GetLogonStatus(iLogonId))
{
    if (iTimes++ > 100)//5 秒超时
    {
        fprintf(stderr, "[LogonDevice] logon(%s) timeout.\n", tInfo.cNvsIP);
        return -1;
    }
    usleep(50000);    //50 毫秒获取一次登录状态
}
return 0;
}

void Notify_Main(int _iLogonID, long _lWparam, void* _pvLParam, void* _pvUsr)
{
    switch (LOWORD(_lWparam))
    {
        case WCM_LOGON_NOTIFY:    //登录消息
        {
```

```

        int iLogonStatus = NetClient_GetLogonStatus(_iLogonID);
        if (LOGON_SUCCESS == iLogonStatus) { //登录成功
            printf("[Notify_Main] logon, id=%d, success.\n", _iLogonID);
            g_iLogonId = _iLogonID;
        } else if (LOGON_FAILED == iLogonStatus) { //登录失败
            printf("[Notify_Main] logon, id=%d, failed.\n", _iLogonID);
        } else if (LOGON_TIMEOUT == iLogonStatus) { //登录超时
            printf("[Notify_Main] logon, id=%d, timeout.\n", _iLogonID);
        } else if (LOGON_RETRY == iLogonStatus) { //重新登录
            printf("[Notify_Main] logon, id=%d, retry.\n", _iLogonID);
        } else if (LOGON_ING == iLogonStatus) { //登录中
            printf("[Notify_Main] logon, id=%d, ing.\n", _iLogonID);
        } else {
            printf("[Notify_Main] logon, id=%d, %d.\n", _iLogonID, iLogonStatus);
        }
    }
    break;
case WCM_VCA_SUSPEND: //智能分析暂停消息
{
    int iResult = 0;
    VCASuspendResult tParam = {0};
    tParam.iBufSize = sizeof(VCASuspendResult);
    NetClient_GetDevConfig(g_iLogonId, NET_CLIENT_VCA_SUSPEND, 0,
    &tParam, sizeof(tParam), &iResult);
    g_iVcaStatus = tParam.iResult; //结果
    if (VCA_SUSPEND_STATUS_PAUSE == tParam.iStatus) {
        if (VCA_SUSPEND_RESULT_SUCCESS == tParam.iResult) {
            printf("[Notify_Main] pause vca success.\n");
        } else {
            printf("[Notify_Main] pause vca failed.\n");
        }
    }
}
break;
default:
    break;
}
}

//人脸库查询
int FaceLibraryQuery()
{
    int iRet = -1;
    FaceLibQuery tQuery = {0};

```

```

tQuery.iSize = sizeof(tQuery);
tQuery.iChanNo = 0;          //通道号，表示第一通道，IPC只有个通道
tQuery.iPageCount = FACE_MAX_PAGE_COUNT; //每页个数，每页最大查询个
//清空之前查询的数据
usleep(20*1000);           //防止操作后立即查询
printf("Face library information-----\n");
int iPageNo = 0;            //查询页码，表示第一页
while (true)
{
    //查询库信息
    tQuery.iPageNo = iPageNo;
    FaceLibQueryResult tResult[FACE_MAX_PAGE_COUNT];
    memset(&tResult, 0, sizeof(tResult));
    iRet = NetClient_FaceConfig(g_iLogonId, FACE_CMD_LIB_QUERY,
    tQuery.iChanNo, &tQuery, tQuery.iSize, &tResult, sizeof(FaceLibQueryResult));
    if (0 != iRet) {
        printf("[FaceLibraryQuery] failed, ret=%d.\n", iRet);
        return iRet;
    }
    //查询完后，打印出来
    for(int i = 0; i < tResult[0].iTotal && i < FACE_MAX_PAGE_COUNT; ++i) {
        g_tLastLibInfo = tResult[i].tFaceLib; //保存库信息，此处只保存最后一个，便于修
        改和删除

        int iIndex = i + 1 + iPageNo*FACE_MAX_PAGE_COUNT;
        char* strType = "Upload";
        if(1 == tResult[i].tFaceLib.iAlarmType){
            strType = "Not upload";
        }
        printf("Serial number: :%d, Library key: %d, Similarity: %d, Library name: %,
        Identification information: %, Description: %s\n", iIndex,
        tResult[i].tFaceLib.iLibKey, tResult[i].tFaceLib.iThreshold,
        tResult[i].tFaceLib.cName, strType, tResult[i].tFaceLib.cExtrInfo);
    }

    //计算总页数
    int iTotalPage = tResult[0].iTotal / FACE_MAX_PAGE_COUNT;
    if (tResult[0].iTotal % FACE_MAX_PAGE_COUNT > 0) {
        iTotalPage = iTotalPage + 1;
    }
    iPageNo++;
    if (iPageNo >= iTotalPage || iPageNo > 1) {
        break;
    }
}

```

```
    }
    printf("\n");
    return iRet;
}

int SearchFace()
{
    //选择人脸库，默认最后一个库
    int iLibKey = g_tLastLibInfo.iLibKey;
    if(iLibKey <= 0) {
        printf("[SearchFace]Face library modification failed: please add or query face library first.\n\n");
        return -1;
    }
    usleep(20*1000);
    //先抠图
    FaceCutEx tCut = {0};
    tCut.iSize = sizeof(tCut);
    tCut.iPicType = 0;        //0-jpg, -png
    tCut.iChanNo = 0;
    tCut.iPageNo = 0;
    tCut.iPageCount = 1;    //只抠1张人脸进行检索
    strncpy_ss(tCut.cPicPath, ".Pic/1111.png", sizeof(tCut.cPicPath)); //图片路径
    if (0 == strlen(tCut.cPicPath))
    {
        printf("[SearchFace]Please select a picture first!\n");
        return -1;
    }

    FaceCutQueryResult tCutRet = {0};
    int iRet = NetClient_FaceConfig(g_iLogonId, FACE_CMD_CUT_EX, 0, &tCut,
    sizeof(tCut), &tCutRet, sizeof(FaceCutQueryResult));
    if (0 != iRet || 0 == strlen(tCutRet.cFileName))
    {
        printf("[SearchFace]Face matching failure!\n");
        return -1;
    }

    //抠图结果进行检索
    FaceSearch tInfo = {0};
    tInfo.iSize = sizeof(tInfo);
    tInfo.iTaskId = tCutRet.iTaskId;
    tInfo.iSimilar = 10;
    tInfo.iLibKey = iLibKey;
```

```

strcpy_s(tInfo.cPicName, sizeof(tInfo.cPicName), tCutRet.cFileName);
tInfo.iPageCount = FACE_MAX_PAGE_COUNT;

FaceQueryResult tSearchRet[FACE_MAX_PAGE_COUNT];
iRet = NetClient_FaceConfig(g_iLogonId, FACE_CMD_SEARCH, 0, &tInfo, tInfo.iSize,
&tSearchRet, sizeof(FaceQueryResult));
if (0 != iRet)
{
    printf("[SearchFace]Face searching failure!\n");
    return -1;
}
int iTotalCount = tSearchRet[0].iTotal;
printf("Face Search Result(first page)-----\n");
for(int i= 0; i < tSearchRet[0].iPageCount && i < FACE_MAX_PAGE_COUNT; ++i)
{
    printf("Serial:%d, Library key:%d, Face key:%d, Name:%s, Birth:%s, Modeling
state:%d\n", tSearchRet[i].iIndex, tSearchRet[i].tFace.iLibKey,
tSearchRet[i].tFace.iFaceKey, tSearchRet[i].tFace.cName,
tSearchRet[i].tFace.cBirthTime, tSearchRet[i].tFace.iModeling);
}
printf("\n");
return 0;
}

int SearchCapture()
{
    //选择人脸库，默认最后一个库
    int iLibKey = g_tLastLibInfo.iLibKey;
    unsigned long ulStartWaitTime = 0;
    unsigned long ulTimeSpan = 0;
    QueryChanNo tQueryChan[32];

    if(iLibKey <= 0) {
        printf("[SearchCapture] Face library modification failed: please add or query face
library first\n");
        return -1;
    }

    usleep(20*1000);

    //先抠图
    FaceCutEx tCut = {0};
    tCut.iSize = sizeof(tCut);
    tCut.iPicType = 1;    //0-jpg, -png

```

```
tCut.iChanNo = 0;
tCut.iPageNo = 0;
tCut.iPageCount = 1;    //只抠1张人脸进行检索
strncpy_ss(tCut.cPicPath, "/Pic/1111.png", sizeof(tCut.cPicPath)); //图片路径
if (0 == strlen(tCut.cPicPath))
{
    printf("[SearchCapture] Please select a picture first!\n");
    return -1;
}

FaceCutQueryResult tCutRet[FACE_MAX_PAGE_COUNT] = {0};
int iRet = NetClient_FaceConfig(g_iLogonId, FACE_CMD_CUT_EX, 0, &tCut,
sizeof(tCut), &tCutRet, sizeof(FaceCutQueryResult));
if (0 != iRet || 0 == strlen(tCutRet->cFileName))
{
    printf("[SearchCapture] Face matching failure!\n");
    return -1;
}

//抠图结果处理
for(int i = 0; i < tCutRet[0].iTotal && i < FACE_MAX_PAGE_COUNT; ++i)
{
    printf("[SearchCapture] Cut Result: Index = %d, FileName = %s\n", tCutRet[i].iIndex,
tCutRet[i].cFileName);
}

if (0 == strlen(tCutRet[0].cFileName))
{
    printf("[SearchCapture] Face Cut Failed!\n");
    return -1;
}

//按条件查询
if (tCutRet[0].iTaskId <= 0)
{
    printf("[SearchCapture] Please cutout first!\n");
    return -1;
}

FaceSearchSnap tQuery = {0};
tQuery.iSize = sizeof(FaceSearchSnap);
//通道列表
tQuery.iChanCount = 0;
tQuery.iChanSize = sizeof(QueryChanNo);
tQueryChan->iChanNo = 0;
```



```
tQueryChan->iStream = 0;
tQuery.pChanList = tQueryChan;
//开始结束时间
tQuery.tBegTime.iYear = 2019;
tQuery.tBegTime.iMonth = 8;
tQuery.tBegTime.iDay = 5;
tQuery.tBegTime.iHour = 0;
tQuery.tBegTime.iMinute = 0;
tQuery.tBegTime.iSecond = 0;
tQuery.tEndTime.iYear = 2019;
tQuery.tEndTime.iMonth = 8;
tQuery.tEndTime.iDay = 21;
tQuery.tEndTime.iHour = 15;
tQuery.tEndTime.iMinute = 0;
tQuery.tEndTime.iSecond = 0;
strncpy_ss(tQuery.cPicturePath, tCutRet[0].cFileName, sizeof(tQuery.cPicturePath));
tQuery.iSimilarity = 9;
tQuery.iSortMode = 1;
tQuery.iTaskId = tCutRet[0].iTaskId;

iRet = NetClient_FaceConfig(g_iLogonId, FACE_CMD_SEARCH_SNAP, 0, &tQuery,
sizeof(tQuery), NULL, 0);
if (0 != iRet)
{
    printf("[SearchCapture] Start search failed!\n");
    return -1;
}

usleep(20*1000);

ulStartWaitTime = netsdk_get_tick_count();
//进度查询
while(1)
{
    FaceReply tOutInfo = {0};
    FaceSearchSnapProcess tInfo = {0};
    tInfo.iSize = sizeof(FaceSearchSnapProcess);
    tInfo.iTaskId = tCutRet[0].iTaskId;
    iRet = NetClient_FaceConfig(g_iLogonId, FACE_CMD_SEARCH_SNAP_PROCESS,
0, &tInfo, sizeof(FaceSearchSnapProcess), &tOutInfo, sizeof(FaceReply));
    if (0 != iRet)
    {
        printf("[SearchCapture] Progress query failed!\n");
        return -1;
    }
}
```

```

    }
    if(6 == tOutInfo.iResult)
    {
        if(100 == tOutInfo.iDelLibProgress)
        {
            printf("[SearchCapture] Search Progress is 100%\n");
            break;
        }
    }
    //秒超时
    ulTimeSpan = netsdk_get_tick_count() - ulStartWaitTime;
    if (ulTimeSpan >= MAX_LOGON_WAIT_TIME)
    {
        printf("[SearchCapture] Wait TIMEOUT!\n");
        return -1;
    }
}

usleep(20*1000);

//结果查询
FaceSearchSnapQuery tInfo = {0};
tInfo.iSize = sizeof(FaceSearchSnapQuery);
tInfo.iTaskId = tCutRet[0].iTaskId;
tInfo.iPageSize = MAX_QUERY_PAGE_COUNT;
tInfo.iPageNo = 0;

FaceSearchSnapResult tOutInfo[MAX_QUERY_PAGE_COUNT] = {0};
iRet = NetClient_FaceConfig(g_iLogonId, FACE_CMD_SEARCH_SNAP_RESULT, 0,
&tInfo, sizeof(tInfo), &tOutInfo, sizeof(FaceSearchSnapResult));
if (0 != iRet)
{
    printf("[SearchCapture] The result query failed!\n");
    return -1;
}
printf("[SearchCapture] Search Result,Total Number = %d-----",
tOutInfo[0].iTotal);
//此处最多显示条
for (int i = 0; i < tOutInfo[0].iCurPageCount && i < MAX_QUERY_PAGE_COUNT; ++i)
{
    printf("Num: %d, Chan: %d, SnapName: %s, NegName: %s\n", tOutInfo[i].iItemIndex,
tOutInfo[i].iChanNo, tOutInfo[i].tPicNeg.cFileName, tOutInfo[i].tPicSnap.cFileName);
}

```

```

        return 0;
    }

int SearchEvent()
{
    QueryChanNo tQueryChan[32] = {0};
    NetFileQueryVca tQuery = {0};
    tQuery.iSize = sizeof(FaceSearchSnap);
    //通道列表，本demo中使用通道0，主码流演示，用户可根据实际情况赋值
    tQuery.iChanCount = 0;
    tQuery.iChanSize = sizeof(QueryChanNo);
    tQueryChan->iChanNo = 0;
    tQueryChan->iStream = 0;
    tQuery.pChanList = tQueryChan;

    tQuery.iVcaCount = 1; //本demo只查询一种智能分析类型，用户可根据实际需求选择
    tQuery.iVcaList[0] = VCA_EVENT_FACEREC; //9：人脸识别
    //开始结束时间，用户可根据实际情况赋值
    tQuery.tBegTime.iYear = 2019;
    tQuery.tBegTime.iMonth = 8;
    tQuery.tBegTime.iDay = 5;
    tQuery.tBegTime.iHour = 0;
    tQuery.tBegTime.iMinute = 0;
    tQuery.tBegTime.iSecond = 0;
    tQuery.tEndTime.iYear = 2019;
    tQuery.tEndTime.iMonth = 8;
    tQuery.tEndTime.iDay = 21;
    tQuery.tEndTime.iHour = 15;
    tQuery.tEndTime.iMinute = 0;
    tQuery.tEndTime.iSecond = 0;
    tQuery.iPageCount = 20;
    tQuery.iPageNo = 0;
    tQuery.iFileType = 2; //查询文件类型2-图片
    tQuery.iConditionCount = 2; //查询条件个数
    IntToStr((1<<16) + 7, tQuery.cQueryCondition[0]); //检索类型1-按事件检索
    IntToStr(1, tQuery.cQueryCondition[1]); //本demo使用人脸检测演示，用户可根据实际情况选择，事件类型1-人脸检测2-人脸比对3-陌生人4-频次5-时长

    NetFileQueryVcaResult tResult[MAX_QUERY_PAGE_COUNT] = {0};
    int iRet = NetClient_Query_V5(g_iLogonId, CMD_NETFILE_QUERY_VCA, 0, &tQuery,
        sizeof(NetFileQueryVca), &tResult, sizeof(NetFileQueryVcaResult));
    if (iRet < 0)
    {
        printf("[SearchEvent] NetClient_Query_V5 Failed!\n");
    }
}

```

```

        return -1;
    }
    printf("[SearchEvent] Search Total Num Is %d -----\\n",
    tResult[0].iTotal);
    //本demo只显示20条做演示，用户可根据实际情况选择
    for(int i = 0; i < tResult[0].iCurPageCount; ++i)
    {
        printf("Num: %d Age: %s FileName: %s\\n", tResult[i].iItemIndex,
        tResult[i].cExAttr[0], tResult[i].tFileAttr[1].cFileName);
    }
    return 0;
}

int SearchFeature()
{

    QueryChanNo tQueryChan[32] = {0};
    NetFileQueryVca tQuery = {0};
    tQuery.iSize = sizeof(FaceSearchSnap);
    //通道列表，本demo中使用通道0，主码流演示，用户可根据实际情况赋值
    tQuery.iChanCount = 0;
    tQuery.iChanSize = sizeof(QueryChanNo);
    tQueryChan->iChanNo = 0;
    tQueryChan->iStream = 0;
    tQuery.pChanList = tQueryChan;

    tQuery.iVcaCount = 1;//本demo只查询一种智能分析类型，用户可根据实际需求选择
    tQuery.iVcaList[0] = VCA_EVENT_FACEREC;    //9：人脸识别
    //开始结束时间，用户可根据实际情况赋值
    tQuery.tBegTime.iYear = 2019;
    tQuery.tBegTime.iMonth = 8;
    tQuery.tBegTime.iDay = 5;
    tQuery.tBegTime.iHour = 0;
    tQuery.tBegTime.iMinute = 0;
    tQuery.tBegTime.iSecond = 0;
    tQuery.tEndTime.iYear = 2019;
    tQuery.tEndTime.iMonth = 8;
    tQuery.tEndTime.iDay = 21;
    tQuery.tEndTime.iHour = 15;
    tQuery.tEndTime.iMinute = 0;
    tQuery.tEndTime.iSecond = 0;
    tQuery.iPageCount = 20;
    tQuery.iPageNo = 0;
    tQuery.iFileType = 2;    //查询文件类型2-图片

```

```

tQuery.iConditionCount = 7; //查询条件个数
IntToStr((0<<16) + 7, tQuery.cQueryCondition[0]); //检索类型0-按特征检索
IntToStr(1, tQuery.cQueryCondition[1]); //本demo使用1-少年演示, 用户可根据实际情况选择, 表年龄, -少年, -青年, -中年, -老年
IntToStr(1, tQuery.cQueryCondition[2]); //本demo使用1-男演示, 用户可根据实际情况选择, 表示性别, -男, -女, -未知
IntToStr(1, tQuery.cQueryCondition[3]); //本demo使用1-汉族演示, 用户可根据实际情况选择, 表示民族, -汉族, -少数民族
IntToStr(2, tQuery.cQueryCondition[5]); //本demo使用2-未佩戴演示, 用户可根据实际情况选择, 表示戴眼镜0-预留, -佩戴, -未佩戴
IntToStr(2, tQuery.cQueryCondition[6]); //本demo使用2-未佩戴演示, 用户可根据实际情况选择, 表示戴口罩0-预留, -佩戴, -未佩戴

NetFileQueryVcaResult tResult[MAX_QUERY_PAGE_COUNT] = {0};
int iRet = NetClient_Query_V5(g_iLogonId, CMD_NETFILE_QUERY_VCA, 0, &tQuery, sizeof(NetFileQueryVca), &tResult, sizeof(NetFileQueryVcaResult));
if (iRet < 0)
{
    printf("[SearchEvent] NetClient_Query_V5 Failed!\n");
    return -1;
}
printf("[SearchEvent] Search Total Num Is %d -----\n", tResult[0].iTotal);
//本demo只显示20条做演示, 用户可根据实际情况选择
for(int i = 0; i < tResult[0].iCurPageCount; ++i)
{
    printf("Num: %d FileName: %s BeginTime:%d-%d-%d %d:%d:%d\n", tResult[i].iItemIndex, tResult[i].tFileAttr[1].cFileName, tResult[i].tBegTime.iYear, tResult[i].tBegTime.iMonth, tResult[i].tBegTime.iDay, tResult[i].tBegTime.iHour, tResult[i].tBegTime.iMinute, tResult[i].tBegTime.iSecond, tResult[i].tEndTime.iYear, tResult[i].tEndTime.iMonth, tResult[i].tEndTime.iDay, tResult[i].tEndTime.iHour, tResult[i].tEndTime.iMinute, tResult[i].tEndTime.iSecond);
}
return 0;
}

int main(int argc, char *argv[])
{
    //加载库
    LoadLib();
    //注册主回调函数
    NetClient_SetNotifyFunction_V4(Notify_Main, NULL, NULL, NULL, NULL);
    //启动SDK

```

```
NetClient_Startup_V4(0, 0, 0);
//登录设备
if (LogonDevice() < 0)
{
    fprintf(stderr, "[main] LogonDevice failed.\n");
    getchar();
    return -1;
}
while (1)
{
    fprintf(stderr, "Please select: \n");
    fprintf(stderr, "*****\n");
    fprintf(stderr, "[0]Quit  [1]QueryFaceLibrary [2]FaceSearch  [3]SearchCapture\n"
        "[4]SearchEvent  [5]FaceFeature\n");
    fprintf(stderr, "*****\n");
    char cTemp[16] = {0};
    gets_s(cTemp, 16);
    if (0 == strlen(cTemp))
    {
        continue;
    }
    int iOpt = atoi(cTemp);
    if (0 == iOpt)
    {
        break;
    }
    else if (1 == iOpt) //查询人脸库
    {
        FaceLibraryQuery();
    }
    else if (2 == iOpt) //以图搜图
    {
        SearchFace();
    }
    else if (3 == iOpt) //以图搜抓拍图
    {
        SearchCapture();
    }
    else if (4 == iOpt) //按事件检索
    {
        SearchEvent();
    }
    else if (5 == iOpt) //按特征检索
    {

```

```
        SearchFeature();
    }
}
//注销用户，释放SDK资源
NetClient_Logoff(g_iLogonId);
NetClient_Cleanup();
return 0;
}
```

4.8 数字通道

```
#include "NetSDK.h"
#include <stdio.h>

int g_iLogonID = -1;

bool IsDigitalChannel(int _iLogonID,int _iChannelNo)
{
    int iChannelType = 0;
    NetClient_GetChannelProperty(_iLogonID,_iChannelNo,
    GENERAL_CMD_GET_CHANNEL_TYPE
    ,&iChannelType,sizeof(iChannelType));//获得通道属性
    if (iChannelType != CHANNEL_TYPE_DIGITAL)
    {
        printf("Channel(%d) is not digital channel!\n",_iChannelNo);
        return false;
    }

    return true;
}

int SetDigitalChannel(int _iLogonID,int _iChannelNo)
{
    if (false == IsDigitalChannel(_iLogonID,_iChannelNo))
    {
        return -1;
    }
    DigitalChnPara dcp = {0};
    dcp.iChannel = _iChannelNo;
    dcp.iEnable = 1;
    dcp.iConnectMode = 0;//连接模式
```

```
dcp.iServerType = 0;//前端设备类型 0:私有 1:onvif 3:rtsp
dcp.iDevChannel = 0;//前端设备通道号
dcp.iDevPort = 3000;
dcp.iStreamType = 0;
dcp.iNetMode = 1;

strcpy_s(dcp.cIP,"192.168.1.11");
strcpy_s(dcp.cUserName,"Admin");
strcpy_s(dcp.cPassword,"1111");

int iRet = NetClient_SetDigitalChannelConfig(_iLogonID, _iChannelNo
, DC_CMD_SET_ALL, &dcp, sizeof(dcp));//设置数字通道参数
if(iRet == 0)
{
    printf("SetDigitalChannelConfig success!\n");
}
else
{
    printf("SetDigitalChannelConfig failed!\n");
}

return 0;
}

int GetDigitalChannel(int _iLogonID,int _iChannelNo)
{
    if (false == IsDigitalChannel(_iLogonID,_iChannelNo))
    {
        return -1;
    }

    DigitalChnPara dcp = {0};
    dcp.iChannel = _iChannelNo;
    int iRet = NetClient_GetDigitalChannelConfig(_iLogonID, _iChannelNo
        , DC_CMD_GET_ALL, &dcp, sizeof(dcp));//获得数字通道参数
    if(iRet == 0)
    {
        printf("Channel(%d),Enable(%d),ConnectMode(%d),ServerType(%d),DevIP(%s),DevCh
        annel(%d),DevPort(%d),StreamType(%d),NetMode(%d),UserName(%s),Password(%s)\n
        \n",dcp.iChannel,dcp.iEnable,dcp.iConnectMode,dcp.iServerType,dcp.cIP,dcp.iDevChan
        nel,dcp.iDevPort,dcp.iStreamType,dcp.iNetMode,dcp.cUserName,dcp.cPassword);
    }
    else
    {

```



```
        printf("GetDigitalChannelConfig failed!\n");
    }
    return 0;
}

void Notify_Main(int _iLogonID, long _iWparam, void* _iLParam, void* _iUserData)
{
    int iMsgType = LOWORD(_iWparam);
    switch (iMsgType)
    {
    case WCM_LOGON_NOTIFY:
        {
            printf("WCM_LOGON_NOTIFY\n");
            if(_iLParam == LOGON_SUCCESS)
            {
                printf("Logon success!\n");
                SetDigitalChannel(_iLogonID, 0);
            }
            else
            {
                printf("Logon failed!\n");
            }
        }
        break;
    default:
        break;
    }
}

void ParaChangeNotify(int _iLogonID, int _iCh, PARATYPE _iParaType, STR_Para*
_strPara, void* _lpUserData)
{
    switch(_iParaType)
    {
    case PARA_DIGITALCHANNEL:
        {
            GetDigitalChannel(_iLogonID, _iCh);
        }
        break;
    default:
        break;
    }
}
```

```
int main(int argc, char *argv[])
{
    LoadNVSSDK(); // 初始化接口库
    int iBufLen = 0;
    NetClient_Startup_V4(3000, 6000, 0); // 初始化 SDK
    NetClient_SetNotifyFunction_V4(Notify_Main, NULL, ParaChangeNotify, NULL,
    NULL); // 设置回调函数

    // 登录设备
    LogonPara tNormal = {0};
    tNormal.iSize = sizeof(LogonPara);
    tNormal.cNvsIP = "192.168.1.1";
    tNormal.iNvsPort = 3000;
    tNormal.cUserName = "admin";
    tNormal.cUserPwd = "1111";
    tNormal.cCharSet = "UTF-8";
    iBufLen = sizeof(LogonPara);
    g_iLogonID = NetClient_Logon_V4(SERVER_NORMAL, tNormal, iBufLen);
    if(g_iLogonID < 0)
    {
        printf("Logon failed!\n");
    }
    getchar(); // 等待用户输入
    NetClient_Logoff(g_iLogonID); // 注销用户
    NetClient_Cleanup(); // 释放 SDK 资源
    FreeNVSSDK();

    return 0;
}
```

第 5 章 函数说明

5.1 SDK 初始化

5.1.1 初始化 SDK

启动 SDK，运行此函数后，方可发送和接收命令

```
int _stdcall NetClient_Startup_V4(
```

```

int      _iServerPort,
int      _iClientPort,
int      _iWnd = 0
);

```

Parameters*_iServerPort*

[in] 主动模式通信端口，取值范围[70-65535)。若客户端不需要使用主动模式连接设备，默认传 0 即可。

_iClientPort

[in] 此参数暂时没用，任意赋值。

_iWnd

[in] 窗口 ID，默认值为 0。

Return Values

0: 成功;

<0: 操作失败，通过 NetClient_GetLastError 获得错误码信息。

5.1.2 设置回调函数

1. 设置主回调

```

int __stdcall NetClient_SetNotifyFunction_V4(
    MAIN_NOTIFY_V4          _MainNotify,
    ALARM_NOTIFY_V4         _AlarmNotify,
    PARACHANGE_NOTIFY_V4    _ParaNotify,
    COMRECV_NOTIFY_V4       _ComNotify,
    PROXY_NOTIFY            _ProxyNotify
);

```

Parameters*_MainNotify*

[in] 主回调

_AlarmNotify

[in] 报警回调

_ParaNotify

[in] 视频参数被更改回调

_ComNotify

[in] 串口接收数据回调

_ProxyNotify

[in] 命令字回调

Callback function

设置回调函数。如果不设置，则系统只使用消息机制，如果设置了回调函数则系统消息和回调函数同时起作用，这里 5 个回调函数可以选择使用，不使用的设置为 NULL。

```

typedef void (*MAIN_NOTIFY_V4)(
    int      _ulLogonID,
    int      _iWparam,
    int      _iLParam,

```

```
void*      _iUser
);
```

CallBack function parameters

_ulLogonID

[in] NetClient_Logon_V4 的返回值。

_iWparam

[in] 消息定义，参照 WCM_LOGON_NOTIFY 等详细消息定义。

_iLParam

[in] 不同协议下相关参数

_iUser

[in] 用户数据

CallBack function

报警消息回调

```
typedef void (*ALARM_NOTIFY_V4)(
```

```
    int          _ulLogonID,
```

```
    int          _iChan,
```

```
    int          _iAlarmState,
```

```
    ALARMTYPE    _iAlarmType,
```

```
    void*        _iUser
```

```
);
```

CallBack function parameters

_ulLogonID

[in] NetClient_Logon 的返回值。

_iChan

[in] 通道号。取值范围：[0, 通道数-1]，通道数由 NetClient_GetChannelNum 参数输出返回。

_iAlarmState

[in] 报警状态

_iAlarmType

[in] 报警类型.报警类型参数如下所示

<i>_iAlarmType</i> 宏定义	宏定义值	<i>_iAlarmType</i> 含义
ALARM_VDO_MOTION	0	移动侦测报警
ALARM_VDO_LOST	2	视频丢失报警
ALARM_VDO_INPORT	3	输入端口报警
ALARM_VDO_OUTPORT	4	输出端口报警
ALARM_VDO_COVER	5	视频遮挡报警
ALARM_AUDIO_LOST	7	音频丢失报警
ALARM_EXCEPTION	8	异常报警
ALARM_VCA_INFO_EX	9	VCA 报警信息
ALARM_UNIQUE_ALERT_MSG	10	单一警报消息

_iUser

[in] 用户数据

CallBack function

设备参数改变回调

```
typedef void (*PARACHANGE_NOTIFY_V4)(
    int            _ulLogonID,
    int            _iChan,
    PARATYPE      _iParaType,
    STR_Para*      _strPara,
    void*          _iUser,
);
```

CallBack function parameters

_ulLogonID

[in] NetClient_Logon 的返回值

_iChan

[in] 通道号。取值范围：[0, 通道数-1]，通道数由 NetClient_GetChannelNum 参数输出返回。

_iParaType

[in] 视频参数和设备定义

_strPara

[in] 设备参数改变结构体

_iUser

[in] 用户数据

CallBack function

串口接收数据回调

```
typedef void (*COMRECV_NOTIFY_V4)(
    int            _ulLogonID,
    char*          _cData,
    int            _iLen,
    int            _iComNo,
    void*          _iUser
);
```

CallBack function parameters

_ulLogonID

[in] NetClient_Logon 的返回值

_cData

[in] 回调数据

_iLen

[in] 回调数据长度

_iComNo

[in] 串口号

_iUser

[in] 用户数据

CallBack function

命令字的回调函数

```
typedef void (*PROXY_NOTIFY)(
    int            _ulLogonID,
```

```
int      _iCmdKey,  
char*    _cData,  
int      _iLen,  
void*    _iUser  
);
```

CallBack function parameters

_ulLogonID

[in] NetClient_Logon 的返回值

_iCmdKey

[in] 命令字

_cData

[in] 回调数据

_iLen

[in] 回调数据长度

_iUser

[in] 用户数据

Return Values

0:成功;

<0:操作失败, 通过 NetClient_GetLastError 获得错误码信息。

Remarks

设置回调函数, 如果不设置, 系统只使用消息机制, 设置回调函数后, 系统消息机制和回调函数同时起作用, 五个回调函数可选择使用, 不使用则将对应的参数设为 NULL。

2. 设置用户数据

```
int __stdcall NetClient_SetNotifyUserData_V4(  
    int      _iLogonID,  
    void*    _iUserData  
);
```

Parameters

_iLogonID

[in] NetClient_Logon_V4 的返回值

_iUserData

[in] 用户数据, 用来区分用户自定义回调用途。

Return Values

0:成功;

<0:操作失败, 通过 NetClient_GetLastError 获得错误码信息。

3. 报警回调

```
int __stdcall NetClient_SetAlarmNotify_V5(  
    ALARM_NOTIFY_V5    _pAlarm,  
);
```

Parameters`_pAlarm`**CallBack function**

```
typedef void (*ALARM_NOTIFY_V5)(
    int                _iLogonID,
    int                _iAlarmType,
    void*              _pInfo,
    int                _iSize,
    void*              _pUser
);
```

CallBack function parameters

`_ulID`

[in] 登录 ID， NetClient_Logon_V4 的返回值

`_iAlarmType`

[in] 报警类型，人脸识别使用 ALARM_TYPE_FACE_IDENT=20

`_pInfo`

[in] 回调数据，人脸识别使用 FaceAlarmNotify（见 6.1.50）

`_iSize`

[out] 数据长度。

`_pUser`

[out] 用户自定义数据。

5.1.3 设置响应消息

设置响应消息（系统消息、参数改变消息和报警消息），在客户端启动（NetClient_Startup）前调用。

```
int _stdcall NetClient_SetMSGHandleEx(
    UINT                _uiMessage,
    HWND                _hWnd,
    UINT                _uiParamMsg,
    UINT                _uiAlarmMsg
);
```

Parameters`_uiMessage`

[in] 系统消息，具体请参照系统消息扩展。

`_hWnd`

[in] 接收消息的窗口句柄。

`_uiParamMsg`

[in] 参数更改消息。

`_uiAlarmMsg`

[in] 报警消息。

Return Values

0: 成功。

<0: 操作失败，通过 NetClient_GetLastError 获得错误码信息。

5.1.4 释放 SDK

在系统退出前运行此函数，运行此函数后将不能再调用 SDK 中其他接口。

```
int _stdcall NetClient_Cleanup();
```

Parameters

无

Return Values

0: 成功。

<0: 操作失败，通过 GetLastError 获得错误码信息。

Remarks

手册中除特殊说明外，所有函数都必须在 NetClient_Startup（NetClient_Startup_V4）和 NetClient_Cleanup 函数之间调用。

5.2 主动模式（不带目录服务器）

1. 获取主动模式相关配置信息

```
int __stdcall NetClient_GetDsmRegstierInfo(
    int          _iCommand,
    void*        _pvBuf,
    int          _iBufSize
);
```

Parameters

_iCommand

宏定义	宏定义值	含义	对应结构体
DSM_CMD_GET_ONLINE_STATE	0	按出厂 ID 获取设备是否在线状态	DsmOnline（见 6.1.49）
DSM_CMD_GET_REGISTER_COUNT	1	获取注册的设备数量	int
DSM_CMD_GET_REGISTER_DEVLIST	2	获取注册的设备 ID 列表，一个设备 ID 存储单元为 32 字节。	char*

_pvBuf

[in] 登录参数结构体

_iBufSize

[in] 输入数据的缓冲区长度（以字节为单位）

Return Values

0: 成功。

<0: 操作失败，通过 GetLastError 获得错误码信息。

5.2 登录设备

5.2.1 登录设备

1、异步模式登录指定设备，该接口非阻塞运行，接口返回成功不一定成功登陆设备，需要在消息回调里等待登录成功的异步消息，对应主消息类型是 WCM_LOGON_NOTIFY；该接口是 NetClient_Logon 的扩展，支持主动模式登陆设备。

```
int __stdcall NetClient_Logon_V4(
    int          _iLogonType,
    void*        _pvPara,
    int          _iInBufferSize
);
```

Parameters

_iLogonType
[in] 连接类型，0-直连模式，1-主动模式。

_pvPara
[in] 登录参数结构体

_iInBufferSize
[in] 输入数据的缓冲区长度（以字节为单位）

Return Values
>=0: 成功。
<0: 操作失败，通过 GetLastError 获得错误码信息。

Remarks

宏定义	宏定义值	含义	对应结构体
SERVER_NORMAL	0	直连模式	LogonPara(详 6.1.10)
SERVER_ACTIVE	1	主动模式（不带目录服务器）	LogonActiveServer(详见 6.1.30)
SERVER_REG_ACTIVE	4	主动模式（带目录服务器）	LogonActiveServer(详见 6.1.30)

2、同步模式登录指定设备，该接口阻塞运行，接口返回成功即登陆设备成功，支持主动模式登陆设备，具体用法可参考 SyncConsoleDemo。

```
int __stdcall NetClient_SyncLogon
```

同步阻塞方式登录指定的设备，不需要等待异步回调消息，接口返回成功即登陆成功，支持主动模式登陆设备

```
int __stdcall NetClient_SyncLogon(
    int          _iLogonType,
    void*        _pvPara,
    int          _iParaSize
);
```

Parameters

_iLogonType
[in] 连接类型，0-正常模式，1-主动模式。

_pvPara
[in] 登录参数结构体

_iParaSize
[in] 输入数据的缓冲区长度（以字节为单位）

Return Values
>=0: 成功
<0: 操作失败

Remarks

宏定义	宏定义值	含义	对应结构体
SERVER_NORMAL	0	直连模式	LogonPara(详 6.1.10)
SERVER_ACTIVE	1	主动模式（不带目录服务器）	LogonActiveServer(详见 6.1.30)
SERVER_REG_ACTIVE	4	主动模式（带目录服务器）	LogonActiveServer(详见 6.1.30)

5.2.2 退出登录

```
int _stdcall NetClient_Logoff(
    int      _iLogonID
);
```

Parameters
_iLogonID
[in] NetClient_Logon_V4 的返回值。

Return Values
0: 成功。
<0: 操作失败，通过 NetClient_GetLastError 获得错误码信息。

5.3 连接视频

5.3.1 开始连接

1、异步模式实时预览视频，该接口非阻塞运行，接口返回成功不一定成功拉流，需要等待视频头消息 WCM_VIDEO_HEAD 后，调用 NetClient_StartPlay 接口播放视频。

```
iint _stdcall NetClient_StartRecv_V5(
    unsigned int*      _puiRecvID,
    NetClientPara*      _ptPara,
    int                 _iParaSize
);
```

Parameters
_puiRecvID
[out] 返回 ID 号，供后续功能使用
_ptPara
[in] 指向 NetClientPara 的结构指针
_iParaSize
[in] 参数大小。

Return Values
0: 成功，已经开始接收，并返回索引号_ulConID;

1: 已经接收数据, 并返回回索引号_ulConID, 可以直接调用 NetClient_StartPlay 进行播放;
<0: 操作失败, 通过 GetLastError 获得错误码信息。

CallBack & Message function

主消息类型	消 息 值	消息含义
WCM_VIDEO_HEAD	8	视频头消息, 当收到视频头时产生。用户调用接收数据后, 如果能正确接收到数据头, 则产生该消息, 表示用户可以播放或存储录像文件。
WCM_ERR_DIGITCHANNEL_NOT_ENABLED	39	数字通道未使能消息
WCM_ERR_DIGITCHANNEL_NOT_CONNECTED	40	数字通道未连接消息

Remarks

设置数据回调函数以后, 要立即调用: NetClient_StartCaptureData 才起作用。操作成功之后 (返回 0), 不能直接播放, 需要等待数据头, 数据头到达后会通过消息 WCM_VIDEO_HEAD 通知应用程序, 此时应用程序再调用 NetClient_StartPlay 进行播放或者录像。此接口 Windows SDK 支持两次调用, Linux SDK 只支持一次调用, 可将一路视频显示在多个不同的窗口, 目前只支持 2 个窗口。

2、同步模式实时预览视频, 该接口阻塞运行, 传入有效的视频显示窗口句柄, 接口返回成功可实现一键预览视频功能, 也支持只拉流不解码不显示和拉流解码不显示等多种组合形式, 满足不同客户需求, 具体用法可参考 SyncConsoleDemo。

```
int _stdcall NetClient_SyncRealPlay(
    unsigned int*      _puiRecvID,
    NetClientPara*     _ptPara,
    int                _iParaSize
);
```

Parameters

_puiRecvID

[out] 返回 ID 号, 供后续功能使用

_ptPara

[in] 指向 NetClientPara 的结构指针

_iParaSize

[in] 参数大小

Return Values

0: 成功

<0: 操作失败。

5.3.2 停止连接

1、异步停止预览接口，与 NetClient_StartRecv_V4 接口配合使用。

```
int _stdcall NetClient_StopRecv(  
    unsigned int      _ulConID  
);
```

Parameters

_ulConID

[in] 连接标识，在 NetClient_StartRecv 或 NetClient_StartRecv_V4 中返回的索引 ID 号。

Return Values

0: 成功。

<0: 操作失败，通过 NetClient_GetLastError 获得错误码信息。

Remarks

断开数据连接后，该 _ulConID 号被释放。

2、同步停止预览接口，与 NetClient_StartRealPlay 接口配合使用。

```
int _stdcall NetClient_StopRealPlay(  
    unsigned int      _uiRecvID  
    int               _iParam  
);
```

Parameters

_uiRecvID

[in] 接收 id，NetClient_StartRealPlaySync 的返回值。

_iParam

[in] 0-保留最后一帧图像； 1-不保留最后一帧图像

Return Values

0: 成功。

<0: 操作失败

5.3.2 设置取原始帧数据的回调函数

```
int _stdcall NetClient_SetRawFrameCallBack(  
    unsigned int      _ulConID,  
    RAWFRAME_NOTIFY   _cbkGetFrame,  
    void*             _pContext  
);
```

Parameters

_ulConID

[in] 连接标识，在 NetClient_NetFileDownload 或 NetClient_StartRecv_V4 中返回的索引 ID 号。

_cbkGetFrame

[in] 回调函数，获得未解码的音视频数据。

_pContext

[in] 用户自定义数据，默认为 NULL。

Callback function

```
typedef void (__stdcall *RAWFRAME_NOTIFY)(
    unsigned int                _ulID,
    unsigned char*              _ucData,
    int                          _iLen,
    RAWFRAME_INFO*              _pRawFrameInfo,
    void*                        _iUser
);
```

Callback function parameters

_ulID

[in] 连接标识，在 NetClient_Logon_V4 或 NetClient_StartRecv_V4 中返回的索引 ID 号。

_ucData

[in] 原始流回调数据。

_iSize

[in] 数据大小。

_pRawFrameInfo

[in] 音视频数据信息结构体。

_iUser

[in] 用户数据。

Return Values

0: 成功。

<0: 操作失败，通过 NetClient_GetLastError 获得错误码信息。

5.3 下载录像

5.3.1 查询录像文件

1、同步模式查询录像，该接口阻塞运行，接口直接返回查询结果。

```
int __stdcall NetClient_Query_V5(
    int                _iLogonID,
    int                _iCmd,
    int                _iChannel,
    void*              _lpIn,
    int                _iInLen,
    void*              _lpOut,
    int                _iOutLen
);
```

Parameters

_iLogonID

[in] 登陆 ID，调用函数 NetClient_Logon_V4 的返回值。

_iCmd[in] 使用 **CMD_NETFILE_QUERY_FILE***_iChannel*

[in] 通道号

_lpIn[in] 结构体 **NETFILE_QUERY_V5**(见 6.1.29)指针*_iBufLen*[in] 结构体 **NETFILE_QUERY_V5** (见 6.1.29)大小*_lpOut*[out] 结构体 **NVS_FILE_DATA** (见 6.1.31)数组*_iOutLen*[out] 结构体 **NVS_FILE_DATA** (见 6.1.31)大小**Return Values**

0: 成功。

<0: 操作失败, 通过 **NetClient_GetLastError** 获得错误码信息。

5.3.2 远程下载录像文件

int __stdcall NetClient_NetFileDownloadFile(

 unsigned int* *_ulConID*, int *_iLogonID*, char* *_cRemoteFilename*, char* *_cLocalFilename*, int *_iFlag*, int *_iPosition*, int *_iSpeed*

);

Parameters*_ulConID*

[out] 下载连接的 ID 号。

_iLogonID[in] **NetClient_Logon_V4** 的返回值。*_cRemoteFilename*

[in] 要下载的远程文件名。

_cLocalFilename

[in] 要保存的本地文件名。

_iFlag

[in] 0: 首次请求该录像文件, 1: 操作录像文件, 默认为 0。

_iPosition

[in] 0-100: 按百分比定位文件播放位置, -1: 不进行定位, 默认为-1, >100: 按时间点定位。

_iSpeed

[in] {1, 2, 4, 8}: 控制文件下载速度, 其他值: 保持当前速度。范围[1-32], 默认为 1。

Return Values

0: 成功。

<0: 操作失败, 通过 NetClient_GetLastError 获得错误码信息。

Remarks

在远程下载文件的同时可以进行网络播放。_iFlag, _iPosition, _iSpeed 目前只有 TC-2816AN-SH、TC-2808AN-S 支持, 其余设备传入默认值即可。

5.3.3 按时间段下载前端存储的录像文件

```
int _stdcall NetClient_NetFileDownloadByTimeSpanEx(
```

```
    unsigned int*      _ulConID,
    int                _iLogonID,
    char*              _pcLocalFile,
    int                _iChannelNO,
    NVS_FILE_TIME*     _uiFromSecond,
    NVS_FILE_TIME*     _uiToSecond,
    int                _iFlag = 0,
    int                _iPosition = -1,
    int                _iSpeed = 1
```

```
);
```

Parameters

_ulConID

[out] 下载连接的 ID 号。

_iLogonID

[in] NetClient_Logon_V4 的返回值。

_pcLocalFile

[in] 保存本地文件的文件名。

_iChannelNO

[in] 通道号。取值范围: [0, 通道数-1], 通道数由 NetClient_GetChannelNum 参数输出返回。

_pTimeBegin

[in] 查找的开始时间 (包括年, 月, 日, 时, 分, 秒), NVS_FILE_TIME(见 6.1.27)结构体指针。

_uiToSecond

[in] 查找的结束时间 (包括年, 月, 日, 时, 分, 秒), NVS_FILE_TIME 结构体指针。

_iFlag

[in] 录像文件标志, 0: 首次请求该录像文件; 1: 操作正在下载的录像文件; 默认值为 0。

_iPosition

[in] 0~100, 按设定时间段内的百分比定位文件;

>100, 从 1970 年 1 月 1 日起至要定位文件时间点的秒数。

默认值为-1。

_iSpeed

[in] 下载速度(1, 2, 4, 8), 控制文件下载速度; 其他值, 保持当前速度。默认值为 1。

Return Values

0: 成功。

<0: 操作失败, 通过 NetClient_GetLastError 获得错误码信息。

Remarks

无

5.4 录像回放

5.4.1 录像回放

```
int __stdcall NetClient_PlayBack(
    unsigned int*    _ulConID,
    int              _iCmd,
    PlayerParam*     _PlayerParam,
    void*            _hWnd
);
```

Parameters

_ulConID

[out] 连接 ID。

_iCmd

[in] 配置命令, 参见配置命令。

_PlayerParam

[in] 下载回放参数。

_hWnd

[in] 播放窗口句柄。

Return Values

0: 成功。

<0: 操作失败, 通过 NetClient_GetLastError 获得错误码信息。

Remarks

不同功能对应不同的结构体和命令号, 如下表所示:

_iCmd 宏定义	_iCmd 宏定义值	_iCmd 含义	_pvCmdBuf 对应结构体
PLAYBACK_TYPE_FILE	1	按文件回放	PlayerParam (见 6.1.32)
PLAYBACK_TYPE_TIME	2	按时间段回放	PlayerParam (见 6.1.32)

5.4.2 录像回放控制

```
int __stdcall NetClient_PlayBackControl(
    unsigned long      _ulConID,
    int                _iControlCode,
    void*              _pcInBuffer,
    int                _iInLen,
    void*              _pcOutBuffer,
    int*               _iOutLen
);
```

Parameters

- _ulConID**
[in] 连接 ID: 在 NetClient_PlayBack 中返回的索引 ID
- _iControlCode**
[in] 配置命令, 参见配置命令。
- _pcInBuffer**
[in] 输入参数。
- _iInLen**
[in] 输入参数长度。
- _pcOutBuffer**
[out] 输出参数。
- _iOutLen**
[in] 输出参数长度。

Return Values

- 0: 成功。
- <0: 操作失败, 通过 NetClient_GetLastError 获得错误码信息。

Remarks

不同功能对应不同的结构体和命令号, 如下表所示:

_iCmd 宏定义	_iCmd 宏定义值	_iCmd 含义	_pvCmdBuf 对应结构体
PLAY_CONTROL_PLAY	1	播放开始	NULL 预留
PLAY_CONTROL_PAUSE	2	播放暂停	NULL 预留
PLAY_CONTROL_SEEK	3	播放定位	int 播放进度百分比 0-100 或者是绝对时间
PLAY_CONTROL_FAST_FORWARD	4	快进播放	int 快进速度 0-4
PLAY_CONTROL_SLOW_FORWARD	5	慢放	int 慢放速度 0-4
PLAY_CONTROL_OSD	6	叠加字符	tPlayBackOsd
PLAY_CONTROL_GET_PROCESS	7	获取进度	tPlaybackProcess
PLAY_CONTROL_START_AUDIO	8	开启音频	int true
PLAY_CONTROL_STOP_AUDIO	9	停止音频	int

			false
PLAY_CONTROL_SET_VOLUME	10	设置音量	tPlaybackVolume
PLAY_CONTROL_GET_VOLUME	11	获取音量值	tPlaybackVolume
PLAY_CONTROL_STEPFORWARD	12	步进	NULL 预留
PLAY_CONTROL_START_CAPTURE FILE	13	开始截取文件	tPlaybackCapture
PLAY_CONTROL_STOP_CAPTURE FILE	14	停止截取文件	
PLAY_CONTROL_START_CAPTURE PIC	15	截图	tPlaybackCapture
PLAY_CONTROL_PLAYRECT	16	设置窗口大小	
PLAY_CONTROL_GETUSERINFO	17	获取用户信息	
PLAY_CONTROL_SETDECRYPTKEY	18	设置解密密码	tDecryptKey
PLAY_CONTROL_GETFILEHEAD	19	获取文件头	
PLAY_CONTROL_SETFILEHEAD	20	设置文件头	
PLAY_CONTROL_GETFRAMERATE	21	获取码率	
PLAY_CONTROL_SYC_ADD	22	同步添加	PlayBackSyc
PLAY_CONTROL_SYC_DEL	23	同步删除	PlayBackSyc
PLAY_CONTROL_GET_PLAYER_INFO	24	通过 _ulConID 获取当前播放信息	PlayBackInfo

5.4.3 停止录像回放

```
int __stdcall NetClient_StopPlayBack(
    unsigned long          _ulConID,
);
```

Parameters

_ulConID
[in] 连接 ID: 在 NetClient_PlayBack 中返回的索引 ID

Return Values

0: 成功。

<0: 操作失败, 通过 NetClient_GetLastError 获得错误码信息。

5.5 人脸检测

5.5.1 开启\暂停智能分析

1. 获取智能分析状态

```
int __stdcall NetClient_GetDevConfig(  
    int          _iLogonID,  
    int          _iCommand,  
    int          _iChannel,  
    void*        _lpInBuffer,  
    int          _iInBufferSize,  
    int*         _lpBytesReturned,  
);
```

Parameters

_iLogonID

[in] NetClient_Logon_V4 的返回值。

_iCommand

[in] 使用 NET_CLIENT_VCA_SUSPEND

_iChannel

[in] 通道号。取值范围：[0, 通道数-1]，通道数由 NetClient_GetChannelNum 参数输出返回。如果命令不需要通道号，改参数无效，置为 0x7FFFFFFF 即可。

_lpInBuffer

[in] 结构体 VCASuspendResult（见 6.1.5）指针

_iInBufferSize

[in] 结构体 VCASuspendResult（见 6.1.5）大小

_lpBytesReturned

[in] 实际收到的数据长度指针，不能为 NULL。

Return Values

0: 成功。

<0: 操作失败，通过 NetClient_GetLastError 获得错误码信息。

2.1 异步模式设置暂停/开启智能分析功能，该接口非阻塞运行，接口返回成功不一定成功暂停/开启智能分析，需要等待异步回调消息 WCM_VCA_SUSPEND 后，调用 NetClient_GetDevConfig 接口获取结果。

```
int __stdcall NetClient_SetDevConfig(  
    int          _iLogonID,  
    int          _iCommand,  
    int          _iChannel,  
    int*         _lpInBuffer,  
    int          _iInBufferSize,
```

```
);
```

Parameters

_iLogonID

[in] NetClient_Logon_V4 的返回值。

_iCommand

[in] 使用 NET_CLIENT_VCA_SUSPEND

_iChannel

[in] 通道号。取值范围：[0, 通道数-1]，通道数由 NetClient_GetChannelNum 参数输出返回。如果命令不需要通道号，该参数无效，置为 0X7FFFFFFF 即可。

_lpInBuffer

[in] 结构体 VCASuspend（见 6.1.6）指针

_iInBufferSize

[in] 结构体 VCASuspend（见 6.1.6）大小

Return Values

0: 成功。

<0: 操作失败，通过 NetClient_GetLastError 获得错误码信息。

2.2 同步模式设置暂停/开启智能分析，该接口阻塞运行，接口返回调用结果。

```
int __stdcall NetClient_SyncSetDevCfg(
```

```
    int        _iLogonID,
```

```
    int        _iChanNo,
```

```
    int        _iCmd,
```

```
    void*      _pvInPara,
```

```
    int        _iInLen,
```

```
    void*      _pvOutRet,
```

```
    int        _iOutLen,
```

```
);
```

Parameters

_iLogonID

[in] NetClient_Logon_V4 的返回值。

_iChanNo

[in] 通道号。取值范围：[0, 通道数-1]，通道数由 NetClient_GetChannelNum 参数输出返回。如果命令不需要通道号，该参数无效，置为 0X7FFFFFFF 即可。

_iCmd

[in] 设备配置命令，参见配置命令。

_pvInPara

[in] 输入参数结构体指针。

_iInLen

[in] 输入参数结构体大小。

_pvOutRet

[out] 输出参数结构体指针。

_iOutLen

[out] 输出参数结构体大小。

Return Values

0: 成功。
-309: 操作失败，智能分析被其他客户端占用
-310: 操作失败，设备未回码

Remarks
不同功能对应不同的结构体和命令号，如下表所示:。

_iCmd 宏定义	_iCmd 宏定义值	_iCmd 含义	_pvInPara 对应结构体	_pvOutRet 对应结构体
SYNC_NET_CLIENT_VCA_SUSPEND	0	暂停\开启智能分析	VCASuspend	VCASuspendResult

5.5.2 开启/关闭人脸检测算法

1.获取人脸检测算法参数

```
int __stdcall NetClient_GetDevConfig(  
    int      _iLogonID,  
    int      _iCommand,  
    int      _iChannel,  
    void*    _lpInBuffer,  
    int      _iInBufferSize,  
    int*     _lpBytesReturned,  
);
```

Parameters

_iLogonID

[in] NetClient_Logon_V4 的返回值。

_iCommand

[in] 使用命令宏 NET_CLIENT_ANYSCENE

_iChannel

[in] 通道号。取值范围: [0, 通道数-1], 通道数由 NetClient_GetChannelNum 参数输出返回。
如果命令不需要通道号, 改参数无效, 置为 0x7FFFFFFF 即可。

_lpInBuffer

[in] 结构体 AnyScene (见 6.1.1) 指针

_iInBufferSize

[in] 结构体 AnyScene (见 6.1.1) 大小

_lpBytesReturned

[in] 实际收到的数据长度指针, 不能为 NULL。

Return Values

0: 成功。

<0: 操作失败, 通过 NetClient_GetLastError 获得错误码信息。

2.设置人脸检测算法相关参数

```
int __stdcall NetClient_SetDevConfig(
    int        _iLogonID,
    int        _iCommand,
    int        _iChannel,
    int*       _lpInBuffer,
    int        _iInBufferSize,
);
```

Parameters*_iLogonID*

[in] NetClient_Logon_V4 的返回值。

_iCommand

[in] 使用 NET_CLIENT_ANYSCENE

_iChannel

[in] 通道号。取值范围：[0, 通道数-1]，通道数由 NetClient_GetChannelNum 参数输出返回。如果命令不需要通道号，该参数无效，置为 0X7FFFFFFF 即可。

_lpInBuffer

[in] 结构体 AnyScene（见 6.1.1）指针

_iInBufferSize

[in] 结构体 AnyScene（见 6.1.1）大小

Return Values

0: 成功。

<0: 操作失败，通过 NetClient_GetLastError 获得错误码信息。

5.5.3 设置/获取人脸检测算法参数

1. 获取人脸检测算法参数

```
int __stdcall NetClient_GetDevConfig(
    int        _iLogonID,
    int        _iCommand,
    int        _iChannel,
    void*       _lpInBuffer,
    int        _iInBufferSize,
    int*       _lpBytesReturned,
);
```

Parameters*_iLogonID*

[in] NetClient_Logon_V4 的返回值。

_iCommand

[in] 使用 NET_CLIENT_FACE_DETECT_ARITHMETIC

_iChannel

[in] 通道号。取值范围：[0, 通道数-1]，通道数由 NetClient_GetChannelNum 参数输出返回。如果命令不需要通道号，改参数无效，置为 0x7FFFFFFF 即可。

_lpInBuffer

[in] 结构体 FaceDetectArithmetic（见 6.1.2）指针

_iInBufferSize

[in] 结构体 FaceDetectArithmetic（见 6.1.2）大小

_lpBytesReturned

[in] 实际收到的数据长度指针，不能为 NULL。

Return Values

0: 成功。

<0: 操作失败，通过 NetClient_GetLastError 获得错误码信息。

2. 设置人脸检测算法参数

```
int __stdcall NetClient_SetDevConfig(
```

```
    int          _iLogonID,
```

```
    int          _iCommand,
```

```
    int          _iChannel,
```

```
    void*        _lpInBuffer,
```

```
    int          _iInBufferSize,
```

```
);
```

Parameters

_iLogonID

[in] NetClient_Logon_V4 的返回值。

_iCommand

[in] 使用 NET_CLIENT_FACE_DETECT_ARITHMETIC

_iChannel

[in] 通道号。取值范围：[0, 通道数-1]，通道数由 NetClient_GetChannelNum 参数输出返回。如果命令不需要通道号，该参数无效，置为 0X7FFFFFFF 即可。

_lpInBuffer

[in] 结构体 FaceDetectArithmetic（见 6.1.2）指针

_iInBufferSize

[in] 结构体 FaceDetectArithmetic（见 6.1.2）大小

Return Values

0: 成功。

<0: 操作失败，通过 NetClient_GetLastError 获得错误码信息。

3. 获取人脸检测图片流上传参数

```
int __stdcall NetClient_VCAGetConfig(
```

```
    int          _iLogonID,
```

```
    int          _iVCACmdID,
```

```
    int          _iChannel,
```

```
    void*        _lpCmdBuf,
```

```
    int          _iCmdBufLen
```

```
);
```

Parameters

_iLogonID

[in] NetClient_Logon_V4 的返回值。

_iVCACmdID

[in] VCA_CMD_PICSTREAM_UPLOADPARAM

[in] 通道号。取值范围：[0, 通道数-1]，通道数由 NetClient_GetChannelNum 参数输出返回。

_lpCmdBuf

[out] 结构体 PicStreamUploadParam（见 6.1.3）指针

_iCmdBufLen

[in] 结构体 PicStreamUploadParam（见 6.1.3）的大小

Return Values

0: 成功。

<0: 操作失败，通过 NetClient_GetLastError 获得错误码信息

4. 设置人脸检测图片流上传参数

```
int __stdcall NetClient_VCASetConfig(
```

```
    int          _iLogonID,
```

```
    int          _iVCACmdID,
```

```
    void*        _iChannel,
```

```
    int          _lpCmdBuf,
```

```
    int          _iCmdBufLen
```

```
);
```

Parameters

_iLogonID

[in] NetClient_Logon_V4 的返回值。

_iVCACmdID

[in] VCA_CMD_PICSTREAM_UPLOADPARAM

_iChannel

[in] 通道号。取值范围：[0, 通道数-1]，通道数由 NetClient_GetChannelNum 参数输出返回。

_lpCmdBuf

[in] 结构体 PicStreamUploadParam（见 6.1.3）指针

_iCmdBufLen

[in] 结构体 PicStreamUploadParam（见 6.1.3）的大小

Return Values

0: 成功。

<0: 操作失败，通过 NetClient_GetLastError 获得错误码信息

5.5.4 开启/停止人脸图片流

1. 开始接收网络图片流

```
int __stdcall NetClient_StartRecvNetPicStream(
```

```
    int          _iLogonID,
```

```
NetPicPara* _ptPara,
int _iBufLen,
int* _puiRecvID
```

);

Parameters

_iLogonID

[in] 登录 ID

_ptPara

[in] 结构体 NetPicPara（见 6.1.4） 指针

_iBufLen

[in] 结构体 NetPicPara（见 6.1.4）大小

_puiRecvID

[out] 接收的图片流 ID

Return Values

0: 成功。

<0: 操作失败，通过 NetClient_GetLastError 获得错误码信息。

2.停止接收网络图片流

```
int __stdcall NetClient_StopRecvNetPicStream(
```

```
    unsigned int _uiRecvID,
```

);

Parameters

_uiRecvID

[in] 图片流 ID

Return Values

0: 成功。

<0: 操作失败，通过 GetLastError 获得错误码信息。

5.6 人脸识别

5.6.1 人脸库查询

1.查询人脸库信息

```
int __stdcall NetClient_FaceConfig(
```

```
    int _iLogonId,
```

```
    int _iCmdId,
```

```
    int _iChanNo,
```

```
    void* _lpIn,
```

```
    int _iInLen,
```

```
    void* _lpOut,
```

```
    int _iOutLen
```

);

Parameters

_iLogonId

[in] NetClient_Logon_V4 的返回值。

_iCmdId

[in] 使用 FACE_CMD_LIB_QUERY

_iChanNo

[in] 通道号。取值范围：[0, 通道数-1]，通道数由 NetClient_GetChannelNum 参数输出返回。

_lpIn

[in] 结构体 FaceLibQuery（见 6.1.35）类型指针

_iInLen

[in] 结构体 FaceLibQuery（见 6.1.35）大小

_lpOut

[out] 结构体 FaceLibQueryResult（见 6.1.12）类型指针

_iOutLen

[in] 结构体 FaceLibQueryResult（见 6.1.12）大小

Return Values

=0: 成功。

<0: 操作失败，通过 NetClient_GetLastError 获得错误码信息

5.6.2 人脸库添加

```
int _stdcall NetClient_FaceConfig(  
    int        _iLogonId,  
    int        _iCmdId,  
    int        _iChanNo,  
    void*      _lpIn,  
    int        _iInLen,  
    void*      _lpOut,  
    int        _iOutLen  
);
```

Parameters

_iLogonId

[in] NetClient_Logon_V4 的返回值。

_iCmdId

[in] 使用 FACE_CMD_LIB_EDIT

_iChanNo

[in] 通道号。取值范围：[0, 通道数-1]，通道数由 NetClient_GetChannelNum 参数输出返回。

_lpIn

[in] 结构体 FaceLibEdit（见 6.1.36）类型指针

_iInLen

[in] 结构体 FaceLibEdit（见 6.1.36）大小

_lpOut

[out] 结构体 FaceReply（见 6.1.13）类型指针

_iOutLen

[in] 结构体 FaceReply（见 6.1.13）大小

Return Values

=0: 成功。

<0: 操作失败，通过 NetClient_GetLastError 获得错误码信息。

5.6.3 人脸库修改

```
int _stdcall NetClient_FaceConfig(  
    int          _iLogonId,  
    int          _iCmdId,  
    int          _iChanNo,  
    void*        _lpIn,  
    int          _iInLen,  
    void*        _lpOut,  
    int          _iOutLen  
);
```

Parameters

_iLogonId

[in] NetClient_Logon_V4 的返回值。

_iCmdId

[in] 使用 FACE_CMD_LIB_EDIT

_iChanNo

[in] 通道号。取值范围：[0, 通道数-1]，通道数由 NetClient_GetChannelNum 参数输出返回。

_lpIn

[in] 结构体 FaceLibEdit（见 6.1.36）类型指针

_iInLen

[in] 结构体 FaceLibEdit（见 6.1.36）大小

_lpOut

[out] 结构体 FaceReply（见 6.1.13）类型指针

_iOutLen

[in] 结构体 FaceReply（见 6.1.13）大小

Return Values

=0: 成功。

<0: 操作失败，通过 NetClient_GetLastError 获得错误码信息。

5.6.4 人脸库删除

```
int _stdcall NetClient_FaceConfig(  
    int          _iLogonId,  
    int          _iCmdId,  
    int          _iChanNo,  
    void*        _lpIn,  
    int          _iInLen,  
    void*        _lpOut,  
    int          _iOutLen  
);
```

Parameters

_iLogonId

[in] NetClient_Logon_V4 的返回值。

_iCmdId

[in] 使用 FACE_CMD_LIB_DELETE

_iChanNo

[in] 通道号。取值范围：[0, 通道数-1]，通道数由 NetClient_GetChannelNum 参数输出返回。

_lpIn

[in] 结构体 FaceLibDelete（见 6.1.14）类型指针

_iInLen

[in] 结构体 FaceLibDelete（见 6.1.14）大小

_lpOut

[out] 结构体 FaceReply（见 6.1.13）类型指针

_iOutLen

[in] 结构体 FaceReply（见 6.1.13）大小

Return Values

=0: 成功。

<0: 操作失败，通过 NetClient_GetLastError 获得错误码信息。

5.6.5 人脸底图查询

```
int _stdcall NetClient_FaceConfig(  
    int          _iLogonId,  
    int          _iCmdId,  
    int          _iChanNo,  
    void*        _lpIn,  
    int          _iInLen,  
    void*        _lpOut,  
    int          _iOutLen  
);
```

Parameters

_iLogonId

[in] NetClient_Logon_V4 的返回值。

_iCmdId

[in] 使用 FACE_CMD_QUERY

_iChanNo

[in] 通道号。取值范围：[0, 通道数-1]，通道数由 NetClient_GetChannelNum 参数输出返回。

_lpIn

[in] 结构体 FaceQuery（见 6.1.15）类型指针

_iInLen

[in] 结构体 FaceQuery（见 6.1.15）大小

_lpOut

[out] 结构体 FaceQueryResult（见 6.1.16）类型指针

_iOutLen

[in] 结构体 FaceQueryResult（见 6.1.16）大小

Return Values

=0: 成功。

<0: 操作失败，通过 NetClient_GetLastError 获得错误码信息

5.6.6 人脸底图添加

```
int _stdcall NetClient_FaceConfig(  
    int          _iLogonId,  
    int          _iCmdId,  
    int          _iChanNo,  
    void*        _lpIn,  
    int          _iInLen,  
    void*        _lpOut,  
    int          _iOutLen  
);
```

Parameters

_iLogonId

[in] NetClient_Logon_V4 的返回值。

_iCmdId

[in] 使用 FACE_CMD_EDIT

_iChanNo

[in] 通道号。取值范围：[0, 通道数-1]，通道数由 NetClient_GetChannelNum 参数输出返回。

_lpIn

[in] 结构体 FaceEdit 类型指针

_iInLen

[in] 结构体 FaceEdit 大小

_lpOut

[out] 结构体 FaceReply 类型指针

_iOutLen

[in] 结构体 FaceReply 大小

Return Values

=0: 成功。

<0: 操作失败，通过 NetClient_GetLastError 获得错误码信息。

[注]: Ipc 添加人脸底图时，或者添加人脸底图需要建模时，需要暂停智能分析，参考 5.5.1

5.6.7 人脸底图修改

```
int _stdcall NetClient_FaceConfig(  
    int          _iLogonId,  
    int          _iCmdId,  
    int          _iChanNo,  
    void*        _lpIn,  
    int          _iInLen,  
    void*        _lpOut,  
    int          _iOutLen  
);
```

Parameters

_iLogonId

[in] NetClient_Logon_V4 的返回值。

_iCmdId

[in] 使用 FACE_CMD_EDIT

_iChanNo

[in] 通道号。取值范围：[0, 通道数-1]，通道数由 NetClient_GetChannelNum 参数输出返回。

_lpIn

[in] 结构体 FaceEdit（6.1.37）类型指针

_iInLen

[in] 结构体 FaceEdit（6.1.37）大小

_lpOut

[out] 结构体 FaceReply（6.1.13）类型指针

_iOutLen

[in] 结构体 FaceReply（6.1.13）大小

Return Values

=0: 成功。

<0: 操作失败，通过 NetClient_GetLastError 获得错误码信息。

5.6.8 人脸底图删除

```
int __stdcall NetClient_FaceConfig(  
    int          _iLogonId,  
    int          _iCmdId,  
    int          _iChanNo,  
    void*        _lpIn,  
    int          _iInLen,  
    void*        _lpOut,  
    int          _iOutLen  
);
```

Parameters

_iLogonId

[in] NetClient_Logon_V4 的返回值。

_iCmdId

[in] 使用 FACE_CMD_DELETE

_iChanNo

[in] 通道号。取值范围：[0, 通道数-1]，通道数由 NetClient_GetChannelNum 参数输出返回。

_lpIn

[in] 结构体 FaceDelete (6.1.18) 类型指针

_iInLen

[in] 结构体 FaceDelete (6.1.18) 大小

_lpOut

[out] 结构体 FaceReply (6.1.13) 类型指针

_iOutLen

[in] 结构体 FaceReply (6.1.13) 大小

Return Values

=0: 成功。

<0: 操作失败，通过 NetClient_GetLastError 获得错误码信息。

5.6.9 开启/关闭人脸识别算法

1. 获取人脸检测算法参数

```
int __stdcall NetClient_GetDevConfig(  
    int          _iLogonID,  
    int          _iCommand,  
    int          _iChannel,  
    void*        _lpInBuffer,  
    int          _iInBufferSize,
```

```
int*      _lpBytesReturned,
);
```

Parameters*_iLogonID*

[in] NetClient_Logon_V4 的返回值。

_iCommand

[in] 使用 NET_CLIENT_ANYSCENE

_iChannel

[in] 通道号。取值范围：[0, 通道数-1]，通道数由 NetClient_GetChannelNum 参数输出返回。如果命令不需要通道号，改参数无效，置为 0x7FFFFFFF 即可。

_lpInBuffer

[in] 结构体 AnyScene（6.1.1）指针

_iInBufferSize

[in] 结构体 AnyScene（6.1.1）大小

_lpBytesReturned

[in] 实际收到的数据长度指针，不能为 NULL。

Return Values

0: 成功。

<0: 操作失败，通过 NetClient_GetLastError 获得错误码信息。

2. 设置人脸检测算法相关参数

```
int __stdcall NetClient_SetDevConfig(
```

```
int      _iLogonID,
```

```
int      _iCommand,
```

```
int      _iChannel,
```

```
int*     _lpInBuffer,
```

```
int      _iInBufferSize,
```

```
);
```

Parameters*_iLogonID*

[in] NetClient_Logon_V4 的返回值。

_iCommand

[in] 使用 NET_CLIENT_ANYSCENE

_iChannel

[in] 通道号。取值范围：[0, 通道数-1]，通道数由 NetClient_GetChannelNum 参数输出返回。如果命令不需要通道号，该参数无效，置为 0x7FFFFFFF 即可。

_lpInBuffer

[in] 结构体 AnyScene（6.1.1）指针

_iInBufferSize

[in] 结构体 AnyScene（6.1.1）大小

Return Values

0: 成功。

<0: 操作失败，通过 NetClient_GetLastError 获得错误码信息。

5.6.10 设置/获取人脸识别参数

1. 获取人脸检测算法参数

```
int __stdcall NetClient_GetDevConfig(  
    int        _iLogonID,  
    int        _iCommand,  
    int        _iChannel,  
    void*      _lpInBuffer,  
    int        _iInBufferSize,  
    int*       _lpBytesReturned,  
);
```

Parameters

_iLogonID

[in] NetClient_Logon_V4 的返回值。

_iCommand

[in] 使用 NET_CLIENT_FACE_DETECT_ARITHMETIC

_iChannel

[in] 通道号。取值范围：[0, 通道数-1]，通道数由 NetClient_GetChannelNum 参数输出返回。
如果命令不需要通道号，改参数无效，置为 0x7FFFFFFF 即可。

_lpInBuffer

[in] 结构体 FaceDetectArithmetic (6.1.2) 指针

_iInBufferSize

[in] 结构体 FaceDetectArithmetic (6.1.2) 大小

_lpBytesReturned

[in] 实际收到的数据长度指针，不能为 NULL。

Return Values

0: 成功。

<0: 操作失败，通过 NetClient_GetLastError 获得错误码信息。

2. 设置人脸检测算法参数

```
int __stdcall NetClient_SetDevConfig(  
    int        _iLogonID,  
    int        _iCommand,  
    int        _iChannel,  
    int*       _lpInBuffer,  
    int        _iInBufferSize,  
);
```

Parameters

_iLogonID

[in] NetClient_Logon_V4 的返回值。

_iCommand

[in] 使用 NET_CLIENT_FACE_DETECT_ARITHMETIC

_iChannel

[in] 通道号。取值范围：[0, 通道数-1]，通道数由 NetClient_GetChannelNum 参数输出返回。如果命令不需要通道号，该参数无效，置为 0X7FFFFFFF 即可。

_lpInBuffer

[in] 结构体 FaceDetectArithmetic (6.1.2) 指针

_iInBufferSize

[in] 结构体 FaceDetectArithmetic (6.1.2) 大小

Return Values

0: 成功。

<0: 操作失败，通过 NetClient_GetLastError 获得错误码信息。

[注]: 若要使用人脸识别功能，结构体 FaceDetectArithmetic 中的 iDentification 成员需要设置为 1。

3. 获取人脸识别图片流上传参数

```
int __stdcall NetClient_VCAGetConfig(
```

```
    int          _iLogonID,  
    int          _iVCACmdID,  
    void*        _iChannel,  
    int          _lpCmdBuf,  
    int          _iCmdBufLen
```

```
);
```

Parameters

_iLogonID

[in] NetClient_Logon_V4 的返回值。

_iVCACmdID

[in] VCA_CMD_PICSTREAM_UPLOADPARAM

[in] 通道号。取值范围：[0, 通道数-1]，通道数由 NetClient_GetChannelNum 参数输出返回。

_lpCmdBuf

[out] 结构体 PicStreamUploadParam (6.1.3) 指针

_iCmdBufLen

[in] 结构体 PicStreamUploadParam (6.1.3) 的大小

Return Values

0: 成功。

<0: 操作失败，通过 NetClient_GetLastError 获得错误码信息

4. 设置人脸识别图片流上传参数

```
int __stdcall NetClient_VCASetConfig(
```

```
    int          _iLogonID,  
    int          _iVCACmdID,  
    void*        _iChannel,  
    int          _lpCmdBuf,  
    int          _iCmdBufLen
```

```
);
```

Parameters

_iLogonID

[in] NetClient_Logon_V4 的返回值。

_iVCACmdID

[in] VCA_CMD_PICSTREAM_UPLOADPARAM

_iChannel

[in] 通道号。取值范围：[0, 通道数-1]，通道数由 NetClient_GetChannelNum 参数输出返回。

_lpCmdBuf

[in] 结构体 PicStreamUploadParam (6.1.3) 指针

_iCmdBufLen

[in] 结构体 PicStreamUploadParam (6.1.3) 的大小

Return Values

0: 成功。

<0: 操作失败，通过 NetClient_GetLastError 获得错误码信息

5.6.13 开启/停止人脸图片流

开始接收网络图片流

```
int __stdcall NetClient_StartRecvNetPicStream(
    int          _iLogonID,
    NetPicPara*  _ptPara,
    int          _iBufLen,
    int*         _puiRecvID
);
```

Parameters

_iLogonID

[in] 登录 ID

_ptPara

[in] 结构体 NetPicPara (6.1.4) 指针

_iBufLen

[in] 结构体 NetPicPara (6.1.4) 大小

_puiRecvID

[out] 接收的图片流 ID

Return Values

0: 成功。

<0: 操作失败，通过 NetClient_GetLastError 获得错误码信息。

2.停止接收网络图片流

```
int __stdcall NetClient_StopRecvNetPicStream(
    unsigned int  _uiRecvID,
);
```

Parameters

_uiRecvID

[in] 图片流 ID

Return Values

0: 成功。

<0: 操作失败，通过 `GetLastError` 获得错误码信息。

5.6.11 设置/获取人脸布放使能

1. 获取人脸布放使能

```
int __stdcall NetClient_GetAlarmConfig(  
    int          _iLogonID,  
    int          _iChannelNo,  
    int          _iAlarmType,  
    int          _iCmd,  
    void*        _pvCmdBuf  
);
```

Parameters

_iLogonID

[in] `NetClient_Logon_V4` 的返回值。

_iChannelNo

[in] 通道号。取值范围：[0, 通道数-1]，通道数由参数输出返回。

_iAlarmType

[in] 报警类型：20-IPC 人脸识别；21，NVR 本地智能分析

_iCmd

[in] 使用 `CMD_GET_ALARMSCH_ENABLE`

_pvCmdBuf

[out] 结构体 `TAlarmScheEnableParam`(6.1.19) 指针

Return Values

0: 成功。

<0: 操作失败，通过 `NetClient_GetLastError` 获得错误码信息。

2. 设置人脸布放使能

```
int __stdcall NetClient_SetAlarmConfig(  
    int          _iLogonID,  
    int          _iChannelNo,  
    int          _iAlarmType,  
    int          _iCmd,  
    void*        _pvCmdBuf  
);
```

Parameters

_iLogonID

[in] `NetClient_Logon` 的返回值。

_iChannelNo

[in] 通道号。取值范围：[0, 通道数-1]，通道数由 `NetClient_GetChannelNum` 参数输出返回。

_iAlarmType

[in] 使用 ALARM_TYPE_NVR_VCA

_iCmd

[in] 使用 CMD_ALARMSCH_ENABLE_EX

_pvCmdBuf

[in] 结构体 TAlarmScheEnableParam(6.1.19) 指针

Return Values

0: 成功。

<0: 操作失败，通过 NetClient_GetLastError 获得错误码信息。

5.6.12 设置/获取人脸布放模板

1. 获取人脸布放模板

int __stdcall NetClient_GetAlarmConfig(

int *_iLogonID*,

int *_iChannelNo*,

int *_iAlarmType*,

int *_iCmd*,

void* *_pvCmdBuf*

);

Parameters

_iLogonID

[in] NetClient_Logon_V4 的返回值。

_iChannelNo

[in] 通道号。取值范围：[0, 通道数-1]，通道数由参数输出返回。

_iAlarmType

[in] 使用 ALARM_TYPE_FACE_IDENT

_iCmd

[in] 使用 CMD_GET_ALARMSCHEDULE

_pvCmdBuf

[out] 结构体 TAlarmScheduleParam(6.1.20) 指针

Return Values

0: 成功。

<0: 操作失败，通过 NetClient_GetLastError 获得错误码信息。

2. 设置人脸布放模板

int __stdcall NetClient_SetAlarmConfig(

int *_iLogonID*,

int *_iChannelNo*,

int *_iAlarmType*,


```

    int          _iCmd,
    void*        _pvCmdBuf
);

```

Parameters*_iLogonID*

[in] NetClient_Logon_V4 的返回值。

_iChannelNo

[in] 通道号。取值范围：[0, 通道数-1]，通道数由 NetClient_GetChannelNum 参数输出返回。

_iAlarmType

[in] 使用 ALARM_TYPE_FACE_IDENT

_iCmd

[in] 使用 CMD_SET_ALARMSCHEDULE

_pvCmdBuf

[in] 结构体 TAlarmScheduleParam (6. 1. 20) 指针

Return Values

0: 成功。

<0: 操作失败，通过 NetClient_GetLastError 获得错误码信息。

5.6.13 设置/获取人脸报警参数

1. 获取人脸报警参数

```

int __stdcall NetClient_SetAlarmConfig(
    int          _iLogonID,
    int          _iChannelNo,
    int          _iAlarmType,
    int          _iCmd,
    void*        _pvCmdBuf
);

```

Parameters*_iLogonID*

[in] NetClient_Logon_V4 的返回值。

_iChannelNo

[in] 通道号。取值范围：[0, 通道数-1]，通道数由 NetClient_GetChannelNum 参数输出返回。

_iAlarmType

[in] 20-IPC 人脸识别报警；21-NVR 本地智能分析

_iCmd

[in] 使用 CMD_ALARM_FACE_PARAM

_pvCmdBuf

[in] 结构体 FaceAlarmParam (6. 1. 7) 指针

Return Values

0: 成功。

<0: 操作失败，通过 NetClient_GetLastError 获得错误码信息。

2. 获取人脸报警参数

```
int _stdcall NetClient_FaceConfig(
    int          _iLogonId,
    int          _iCmdId,
    int          _iChanNo,
    void*        _lpIn,
    int          _iInLen,
    void*        _lpOut,
    int          _iOutLen
);
```

Parameters

_iLogonId

[in] NetClient_Logon_V4 的返回值。

_iCmdId

[in] 使用 FACE_CMD_ALARM_PARAM

_iChanNo

[in] 通道号。取值范围: [0, 通道数-1], 通道数由 NetClient_GetChannelNum 参数输出返回。

_lpIn

[in] 结构体 FaceAlarmParamIn (6.1.9) 类型指针。

_iInLen

[in] 结构体 FaceAlarmParamIn (6.1.9) 大小

_lpOut

[out] 结构体 FaceAlarmParam (6.1.7) 类型指针。

_iOutLen

[in] 结构体 FaceAlarmParam (6.1.7) 大小

Return Values

=0: 成功。

<0: 操作失败, 通过 GetLastError 获得错误码信息

5.7 人脸检索

5.7.1 按人脸底图检索

1. 抠图

```
int _stdcall NetClient_FaceConfig(
    int          _iLogonId,
    int          _iCmdId,
    int          _iChanNo,
    void*        _lpIn,
    int          _iInLen,
    void*        _lpOut,
```

```

    int          _iOutLen
);
Parameters
    _iLogonId
[in] NetClient_Logon_V4 的返回值。
    _iCmdId
[in] 使用 FACE_CMD_CUT_EX
    _iChanNo
[in] 通道号，取值范围：[0, 通道数-1]，通道数由 NetClient_GetChannelNum 参数输出返回。
    _lpIn
[in] 结构体 FaceCutEx (6. 1. 38) 类型指针。
    _iInLen
[in] 结构体 FaceCutEx (6. 1. 38) 大小
    _lpOut
[out] 结构体 FaceCutQueryResult (6. 1. 39) 类型指针。
    _iOutLen
[in] 结构体 FaceCutQueryResult (6. 1. 39) 大小

```

2. 检索抠图结果

```

int _stdcall NetClient_FaceConfig(
    int          _iLogonId,
    int          _iCmdId,
    int          _iChanNo,
    void*        _lpIn,
    int          _iInLen,
    void*        _lpOut,
    int          _iOutLen
);
Parameters
    _iLogonId
[in] NetClient_Logon_V4 的返回值。
    _iCmdId
[in] 使用 FACE_CMD_SEARCH
    _iChanNo
[in] 通道号。取值范围：[0, 通道数-1]，通道数由 NetClient_GetChannelNum 参数输出返回。
    _lpIn
[in] 结构体 FaceSearch (6. 1. 40) 类型指针。
    _iInLen
[in] 结构体 FaceSearch (6. 1. 40) 大小
    _lpOut
[out] 结构体 FaceQueryResult (6. 1. 16) 类型指针。
    _iOutLen
[in] 结构体 FaceQueryResult (6. 1. 16) 大小

```

5.7.2 按抓拍图检索

1. 抠图

```
int _stdcall NetClient_FaceConfig(  
    int          _iLogonId,  
    int          _iCmdId,  
    int          _iChanNo,  
    void*        _lpIn,  
    int          _iInLen,  
    void*        _lpOut,  
    int          _iOutLen  
);
```

Parameters

_iLogonId

[in] NetClient_Logon_V4 的返回值。

_iCmdId

[in] 使用 FACE_CMD_CUT_EX

_iChanNo

[in] 通道号。取值范围：[0, 通道数-1]，通道数由 NetClient_GetChannelNum 参数输出返回。

_lpIn

[in] 结构体 FaceCutEx (6. 1. 38) 类型指针。

_iInLen

[in] 结构体 FaceCutEx (6. 1. 38) 大小

_lpOut

[out] 结构体 FaceCutQueryResult (6. 1. 39) 类型指针。

_iOutLen

[in] 结构体 FaceCutQueryResult (6. 1. 39) 大小

2. 开始搜索

```
int _stdcall NetClient_FaceConfig(  
    int          _iLogonId,  
    int          _iCmdId,  
    int          _iChanNo,  
    void*        _lpIn,  
    int          _iInLen,  
    void*        _lpOut,  
    int          _iOutLen  
);
```

Parameters

_iLogonId

[in] NetClient_Logon_V4 的返回值

_iCmdId

[in] 使用 FACE_CMD_SEARCH_SNAP

_iChanNo

[in] 通道号。取值范围：[0, 通道数-1]，通道数由 NetClient_GetChannelNum 参数输出返回

_lpIn

[in] 结构体 FaceSearchSnap (6. 1. 41) 类型指针

_iInLen

[in] 结构体 FaceSearchSnap (6. 1. 41) 大小

_lpOut

[out] NULL。

_iOutLen

[in] 0

3. 查询搜索进度

```
int _stdcall NetClient_FaceConfig(
```

```
    int          _iLogonId,
```

```
    int          _iCmdId,
```

```
    int          _iChanNo,
```

```
    void*        _lpIn,
```

```
    int          _iInLen,
```

```
    void*        _lpOut,
```

```
    int          _iOutLen
```

```
);
```

Parameters

_iLogonId

[in] NetClient_Logon_V4 的返回值。

_iCmdId

[in] 使用 FACE_CMD_SEARCH_SNAP_PROCESS

_iChanNo

[in] 通道号。取值范围：[0, 通道数-1]，通道数由 NetClient_GetChannelNum 参数输出返回。

_lpIn

[in] 结构体 FaceSearchSnapProcess (6. 1. 43) 类型指针。

_iInLen

[in] 结构体 FaceSearchSnapProcess (6. 1. 43) 大小

_lpOut

[out] 结构体 FaceReply (6. 1. 13) 类型指针。

_iOutLen

[in] 结构体 FaceReply (6. 1. 13) 大小

4. 获取搜索结果

```
int _stdcall NetClient_FaceConfig(
```

```
    int          _iLogonId,
```

```
    int          _iCmdId,
```

```
    int          _iChanNo,
```

```
    void*        _lpIn,
```

```

    int          _iInLen,
    void*         _lpOut,
    int           _iOutLen

```

```
);
```

Parameters

_iLogonId

[in] NetClient_Logon_V4 的返回值。

_iCmdId

[in] 使用 FACE_CMD_SEARCH_SNAP_RESULT

_iChanNo

[in] 通道号。取值范围：[0, 通道数-1]，通道数由 NetClient_GetChannelNum 参数输出返回。

_lpIn

[in] 结构体 FaceSearchSnapQuery (6. 1. 43) 类型指针。

_iInLen

[in] 结构体 FaceSearchSnapQuery (6. 1. 43) 大小

_lpOut

[out] 结构体 FaceSearchSnapResult (6. 1. 13) 类型指针。

_iOutLen

[in] 结构体 FaceSearchSnapResult (6. 1. 13) 大小

5.7.3 按事件检索

```

int __stdcall NetClient_Query_V5(
    int          _iLogonId,
    int          _iCmdId,
    int          _iChanNo,
    void*         _lpIn,
    int          _iInLen,
    void*         _lpOut,
    int          _iOutLen);

```

Parameters

_iLogonId

[in] NetClient_Logon_V4 的返回值。

_iCmdId

[in] 使用 CMD_NETFILE_QUERY_VCA

_iChanNo

[in] 通道号。取值范围：[0, 通道数-1]，通道数由 NetClient_GetChannelNum 参数输出返回。

_lpIn

[in] 结构体 NetFileQueryVca (6. 1. 47) 类型指针。

[注]: 成员 cQueryCondition[0] 需赋为 1

_iInLen

[in] 结构体 NetFileQueryVca (6. 1. 47) 大小

_lpOut

[out] 结构体 NetFileQueryVcaResult (6. 1. 48)类型指针。

_iOutLen

[in] 结构体 NetFileQueryVcaResult (6. 1. 48) 大小

5.7.4 按特征检索

```
int __stdcall NetClient_Query_V5(
    int          _iLogonId,
    int          _iCmdId,
    int          _iChanNo,
    void*        _lpIn,
    int          _iInLen,
    void*        _lpOut,
    int          _iOutLen);
```

Parameters

_iLogonId

[in] NetClient_Logon_V4 的返回值。

_iCmdId

[in] 使用 CMD_NETFILE_QUERY_VCA

_iChanNo

[in] 通道号。取值范围：[0, 通道数-1]，通道数由 NetClient_GetChannelNum 参数输出返回。

_lpIn

[in] 结构体 NetFileQueryVca (6. 1. 47) 类型指针。

[注]: 成员 cQueryCondition[0] 需赋为 0

_iInLen

[in] 结构体 NetFileQueryVca (6. 1. 47) 大小

_lpOut

[out] 结构体 NetFileQueryVcaResult (6. 1. 48)类型指针。

_iOutLen

[in] 结构体 NetFileQueryVcaResult (6. 1. 48) 大小

5.8 用户数据

5.8.1 获得当前播放实时流的用户数据

```
int __stdcall NetClient_GetUserDataInfo(
    unsigned int  _ulConID,
    int          _iFlag,
    void*        _pBuffer,
    int          _iSize)
```

);

Parameters

_ulConID

[in] NetClient_Logon 或 NetClient_Logon_V4 的返回值。

_iFlag

[in] 用户数据标志: GET_USERDATA_INFO_MIN~GET_USERDATA_INFO_MAX。

_pBuffer

[in] 输入数据缓冲区指针。

_iSize

[in] 输入数据的缓冲区长度（以字节为单位）。

Return Values

0: 成功。

<0: 操作失败，通过 NetClient_GetLastError 获得错误码信息。

5.9 获取错误码

获取网络库 SDK 错误码。

```
unsigned long _stdcall NetClient_GetLastError();
```

Return Values

>=0: 成功，返回错误码，详见第 7 章

5.10 数字通道

1. 获取通道属性

```
int _stdcall NetClient_GetChannelProperty(
```

```
    int        _iLogonID,
```

```
    int        _iChannel,
```

```
    int        _iCmd,
```

```
    void*      _lpBuf,
```

```
    int        _iSize
```

```
);
```

Parameters

_iLogonID

[in] NetClient_Logon_v4 的返回值。

_iChannel

[in] 通道号。取值范围: [0, 通道数-1]，通道数由 NetClient_GetChannelNum 参数输出返回。

_iCmd

[in] 命令 ID，

GENERAL_CMD_CHANNEL_TYPE，取值为 1，标识设备的某个通道的通道类型；

GENERAL_CMD_CHANATYPE_LIST，取值为 2，标识通道可以支持编辑的通道类型。

_lpBuf

[out] 数据缓冲区指针，

当命令为 GENERAL_CMD_CHANNEL_TYPE 时:如果获取通道为模拟通道或数字通道

_lpBuf 返回为 int 型，

CHANNEL_TYPE_MAINSTREAM--主码流；

CHANNEL_TYPE_SUBSTREAM--副码流；

CHANNEL_TYPE_DIGITAL--数字通道； CHANNEL_TYPE_COMBINE--合成通道；

当命令为 GENERAL_CMD_CHANATYPE_LIST 时,_lpBuf 返回为 int 型(按位表示):

1--可以编辑为数字通道；

2--可以编辑为本地模拟通道；

4--可以编辑为虚拟合成通道按位表示。

_iSize

[in] 数据缓冲区大小。

Return Values

0: 成功

<0: 操作失败

Remarks

对于主码流，_iChannel 为通道号；对于副码流，_iChannel 为通道数+相应的通道号。

2.设置数字通道信息

int __stdcall NetClient_SetDigitalChannelConfig(

int _iLogonID,

int _iChannel,

int _iCmd,

void* _lpCmdBuf,

int _iBufSize

);

Parameters

_iLogonID

[in] NetClient_Logon_V4 的返回值。

_iChannel

[in] 视频通道号，取值范围：[0, 通道数-1]， 通道数由 NetClient_GetChannelNum 参数输出返回。

_iCmd

[in] 命令 DC_CMD_SET_ALL

_lpCmdBuf

[in] 使用 DigitalChnPara（见 6.1.52）结构体

_iBufSize

[in] 结构体 DigitalChnPara（见 6.1.52）大小

Return Values

0: 成功。

<0: 操作失败。

参数改变消息: PARA_DIGITALCHANNEL

3. 获取数字通道参数

```
int __stdcall NetClient_GetDigitalChannelConfig(
    int          _iLogonID,
    int          _iChannel,
    int          _iCmd,
    void*        _lpCmdBuf,
    int          _iBufSize
);
```

Parameters

_iLogonID

[in] NetClient_Logon_V4 的返回值。

_iChannel

[in] 视频通道号, 取值范围: [0, 通道数-1], 通道数由 NetClient_GetChannelNum 参数输出返回。

_iCmd

[in] 命令 DC_CMD_GET_ALL

_lpCmdBuf

[out] 使用 DigitalChnPara (见 6.1.52) 结构体

_iBufSize

[in] 结构体 DigitalChnPara (见 6.1.52) 大小

Return Values

0: 成功。

<0: 操作失败。

第 6 章 结构体说明

6.1 结构体

6.1.1. 场景信息结构体

```
typedef struct tagAnyScene
{
    int      iBufSize;
    int      iSceneID;
```

```
char    cSceneName[LEN_32];
int     iArithmetic;
int     iDevType;
int     iArithmeticEx;
} AnyScene,*pAnyScene;
```

Members

iBufSize

[in]结构体大小

iSceneID

[in]场景号

cSceneName[LEN_32]

[in]场景名称

iArithmetic

[in]

bit0: 行为分析算法;

bit1: 跟踪算法;

bit2: 人脸检测算法;

bit3: 人数统计算法;

bit4: 视频诊断算法;

bit5: 车牌识别算法;

bit6: 音频异常算法;

bit7: 离岗算法;

bit8: 人群聚集算法;

bit9: 违章停车算法;

bit10: 打架算法;

bit11: 证人保护算法;

bit12: 人脸检测算法 (ST 专用);

bit13: 翻窗检测算法;

bit14: 车位看守算法;

bit18: 安全帽检测算法;

bit19: 颜色跟踪算法;

bit20: 结构化算法;

bit21: 单人询问/无人看管;

bit22: 攀高;

bit23: 新离岗;

bit24: 人数异常;

bit25: 人员起身;

bit26: 离床;

bit27: 静止检测;

bit28: 睡岗;

bit29: 摔倒;

bit30: 新打架;

bit31: 肢体接触;

iDevType :

设备类型，0-IPC，1-NVR

iArithmeticEx :

[in]启用扩展算法类型，按位使能，每一位代表一种算法，0-不启用，1-启用。

bit0: 人形检测算法;

6.1.2 人脸检测参数

```
typedef struct tagFaceDetectArithmetic {  
    int                iBufSize;  
    int                iSceneID;  
    int                iMaxSize;  
    int                iMinSize;  
    int                iLevel;  
    int                iSensitiv;  
    int                iPicScale;  
    int                iSnapEnable;  
    int                iSnapSpace;  
    int                iSnapTimes;  
    int                iPointNum;  
    POINT              ptArea[MAX_FACE_DETECT_AREA_COUNT];  
    int                iDisplayTarget;  
    int                iExposureBright;  
    int                iDisplayRule;  
    int                iMinSizeEx;  
    int                iMaxSizeEx;  
    int                iPushMode;  
    int                iPushLevel;  
    int                iSnapMode;  
    int                iSnapLevel;  
    int                iIdentification;  
    int                iDevType;  
    int                iQpvalueBig;  
    int                iQpvalueSmall;  
    int                iAlgSnapMode;  
    int                iPlateMinSize;  
}
```

}FaceDetectArithmetic, *pFaceDetectArithmetic;

Members

iBufSize :

[in]结构体大小

iSceneID :

[in]场景号：支持最大 16 个场景，编号 0~15

iMaxSize :

[in]最大人脸尺寸：0~100 图像宽度百分比，100 表示整个屏幕的宽度。

iMinSize :

[in]最小人脸尺寸：0~100 图像宽度百分比，100 表示整个屏幕的宽度。

iLevel :

[in]算法运行级别：0~5

iSensitiv :

[in]灵敏度：0~5

iPicScale :

[in]图像缩放比例：1~10

iSnapEnable :

[in]人脸抓拍使能：1-使能，0-不使能

iSnapSpace :

[in]抓拍间隔：间隔帧数

iSnapTimes :

[in]抓拍次数：1~10

iPointNum :

[in]多边形区域顶点个数：3~32

ptArea :

[in]区域参数：视频对应分辨率下坐标

iDisplayTarget :

[in]目标框是否显示：0-不显示 1-显示

iExposureBright :

[in]曝光亮度，范围 0~255。

iDisplayRule :

[in]规则框是否显示：0-不显示 1-显示。

iMinSizeEx :

[in]最小人脸尺寸，图像宽度万分比，范围 1~10000，10000 表示整个屏幕的宽度，此字段为 0 时不生效。

iMaxSizeEx :

[in]最大人脸尺寸，图像宽度万分比，范围 1~10000，10000 表示整个屏幕的宽度，此字段为 0 时不生效。

iPushMode :

[in]推图策略：0-预留，1-最快，2-最优，3-自定义，4-定时，5-门禁。

iPushLevel :

[in]推图等级

当 *iPushMode* 为 3 时，0-预留，1-快，2-中，3-慢。

当 *iPushMode* 为 4 时，表示定时时间。

iSnapMode :

[in]抓拍策略：0-预留，1-全抓模式，2-高质量模式，3-自定义模式。

iSnapLevel :

[in]抓拍等级，当 *iSnapMode* 为 3 时有效，范围[0-100]。

iIdentification :

[in]人脸识别算法开关，0-不支持，1-关闭，2-开启。

iDevType :

[in]设备类型，0-IPC，1-NVR。

iQpvalueBig :

[in]定制专用字段，用户无需关注。（背景图质量，范围 1~100，0 表示不使用此项设置。）

iQpvalueSmall :

[in]定制专用字段，用户无需关注。（人脸小图质量，范围 1~100，0 表示不使用此项设置。）

iAlgSnapMode :

[in]定制专用字段，用户无需关注。（算法抓拍模式，0-人脸，1-车辆，2-混合。）

iPlateMinSize :

[in]定制专用字段，用户无需关注。（图像宽度万分比，范围 1~10000，10000 表示整个屏幕的宽度，此字段为 0 时表示不处理。）

6.1.3 人脸检测图片流上传参数

```
typedef struct tagPicStreamUploadParam{
    int          iSize;
    int          iSceneId;
    int          iRuleNo;
    int          iPicType;
    int          iSnapEnable;
    int          iIsOsd;
    int          iQpvalue;
    int          iFaceFrameEnable;
}PicStreamUploadParam,*pPicStreamUploadParam;
```

Members

iSize

[in]结构体大小

iSceneId

[in]场景编号, 0~15

iRuleNo

[in]规则 ID, 0~7

iPicType

[in]需要设置的图片类型,0 为背景大图,1 为人脸小图

iSnapEnable

[in]发送使能,0 为不上传, 1 为上传

iIsOsd

[in]背景大图是否叠加信息,0 为不叠加, 1 为叠加 ,注:人脸小图不支持此项

iQpvalue

[in]抓拍图片质量,1-100

iFaceFrameEnable

[in]背景大图是否叠加人脸框,0 为不叠加, 1 为叠加,注:人脸小图不支持此项

6.1.4 图片流相关结构体

```
typedef struct tagNetPicPara {
    int                iStructLen;
    int                iChannelNo;
    NET_PICSTREAM_NOTIFY cbkPicStreamNotify;
    void*              pvUser;
    int                iPicType;
}
```

}NetPicPara, *pNetPicPara;

Members

iStructLen

[in]结构体大小

iChannelNo

[in]通道号

cbkPicStreamNotify

[in]回调函数

pvUser

[in]用户数据

iPicType

[in]客户端请求图片流类型：按位使能，bit0-人脸，bit1-行人，bit2-车牌，bit3-机动车，bit4-非机动车；该字段不发或发 0，表示由 server 自适应发送，基于当前的配置发送。[结构化专用字段，此类型与当前业务不匹配时，创建图片流失败]

CallBack function

```
typedef int (__stdcall *NET_PICSTREAM_NOTIFY)(
    unsigned int    _uiRecvID,
    long            _lCommand,
    void*           _pvBuf,
    int             _iBufLen,
    void*           _pvUser
);
```

CallBack function parameters

_uiRecvID

[in] 图片流 ID。

_lCommand

[in] 使用 NET_PICSTREAM_CMD_FACE

_pvBuf

[in] 结构体 FacePicStream 指针

_iBufLen

[in] 结构体 FacePicStream 大小

_pvUser

[in] 用户数据。

6.1.5 智能分析暂停结果结构体

```
typedef struct tagVCASuspendResult{  
    int          iBufSize;  
    int          iStatus;  
    int          iResult;  
    int          iDevType;  
} VCASuspendResult, *pVCASuspendResult;
```

Members

iBufSize

[in]结构体大小

iStatus

[out]状态 0-暂停智能分析, 1,-开启智能分析

iResult

[out]暂停结果, 1 成功; 2 失败

iDevType

[out]设备类型, 0-IPC, 1-NVR

[注]: 以后 sdk 变为阻塞接口, 不需要再查询结果

6.1.6 暂停智能分析结构体

```
typedef struct tagVCASuspend{  
    int          iStatus;  
    int          iDevType;  
} VCASuspend, *pVCASuspend;
```

Members

iStatus

[in]状态 0-暂停智能分析, 1,-开启智能分析

iDevType

[in]设备类型, 0-IPC, 1-NVR

6.1.7 人脸报警信息结构体

```
typedef struct tagFaceAlarmParam{  
    int          iSize;  
    int          iChanNo;  
    int          iAlarmType;  
    int          iParam1;  
    int          iParam2;
```

```

    int      iParam3;
    char      cLibkey[LEN_64];
    int      iRecognition;
    int      iSimilar;
    int      iDevType;
    int      iEnable;
    char      cLibUUID[LEN_64];
    int      iLibEnable;

```

```
}FaceAlarmParam,*pFaceAlarmParam;
```

Members

iSize

[in]结构体大小。

iChanNo

[in]通道号

iAlarmType

[in]

iAlarmType 宏定义	取值	说明
ALARM_TYPE_FACE_IDENT	20	IPC 人脸识别
ALARM_TYPE_NVR_VCA	21	NVR 本地人脸识别

iParam1

[in]参数 1, 当 iAlarmType=21 时, iParam1 表示 NVR 本地人脸识别的子报警类型, 详细信息见下表:

iParam1 宏定义	取值	说明
FACE_ALARM_RECOGNITON_DISCERN	1	比对报警
FACE_ALARM_RECOGNITON_STRANGER	2	陌生人报警
FACE_ALARM_TYPE_BLACKLIST	3	频次报警
FACE_ALARM_TYPE_WHITELIST	4	滞留报警

iParam2

[in]当 iAlarmType=21, iParam1=3, 4 时, iParam2 有效, 代表时间

iParam3

[in]当 iAlarmType=21, iParam2=3 时, iParam3 有效, 代表频次

cLibkey[LEN_64]

[in]表示人脸库键值

iRecognition

[in]是否上传识别信息, 0-不支持, 1-不上传, 2-上传

iSimilar

[in]相似度 0--100

iDevType

[in]设备类型, 0-IPC, 1-NVR

iEnable

[in]人脸识别子报警类型的报警使能（比对报警-陌生人报警-频次报警-滞留报警），0-不使能, 1-使能

cLibUUID[LEN_64]

[in]人脸库的唯一标识, 若为空 cLibkey 为人脸库唯一标识

iLibEnable

[in]人脸库关联报警使能，0-不使能，1-使能

6.1.8 人脸信息结构体

```
typedef struct tagFaceInfo
{
    int            iSize;
    int            iLibKey;
    int            iFaceKey;
    int            iCheckCode;
    int            iFileType;
    int            iModeling;
    char           cName[LEN_64];
    int            iSex;
    char           cBirthTime[LEN_16];
    int            iNation;
    int            iPlace;
    int            iCertType;
    char           cCertNum[LEN_64];
    int            iOptType;
    char           cLibUUID[LEN_UUID];
    char           cFaceUUID[LEN_UUID];
    char           cLibVersion[LEN_64];
    char           cVerifyCode[LEN_64];
    int            iTaskId;
    char           cFileName[LEN_256];
    int            iSimilar;
}FaceInfo,*pFaceInfo;
```

Members

iSize

[out]结构体大小

iLibKey

[out]人脸库 key 值

iFaceKey

[out]图片 key 值

iCheckCode

[out]文件校验码

iFileType

[out]文件扩展类型，0-jpg，1-png

iModeling

[out]建模状态，0-建模成功，1-建模失败，2-未建模

cName[LEN_64]

```

[out]姓名
iSex
[out]性别
cBirthTime[LEN_16]
[out]出生日期
iNation
[out]民族
iPlace
[out]籍贯
iCertType
[out]证件类型, 0-未知, 1-身份证, 2-军官证
cCertNum[LEN_64]
[out]证件号
iOptType
[out]操作类型, 1=添加, 2=修改, 3=复制, 4=迁移
cLibUUID[LEN_UUID]
[out]人脸库在平台的 ID
cFaceUUID[LEN_UUID]
[out]图片在平台对应 ID
cLibVersion[LEN_64]
[out]人脸库版本号
cVerifyCode[LEN_64]
[out]图片校验码
iTaskId
[out]事务 ID, 以图搜底图有效
cFileName[LEN_256]
[out]图片名称
iSimilar
[out]相似度

```

6.1.9 人脸报警入参结构体

```

typedef struct tagFaceAlarmParamIn
{
    int            iSize;
    int            iType;
}FaceAlarmParamIn, *pFaceAlarmParamIn;
Members
iSize
[in]结构体大小
iType
[in]NVR 本地智能分析类型 1-人脸识别-比对 2-人脸识别-陌生人 3-人脸识别-频

```

6.1.10 登陆参数结构体

```
typedef struct tagLogonPara
{
    int      iSize;
    char     cProxy[LEN_32];
    char     cNvsIP[LEN_32];
    char     cNvsName[LEN_32];
    char     cUserName[LEN_16];
    char     cUserPwd[LEN_16];
    char     cProductID[LEN_32];
    int      iNvsPort;
    char     cCharSet[LEN_32];
    char     cAccontName[LEN_16];
    char     cAccontPasswd[LEN_16];
} LogonPara, *pLogonPara;
```

Members

iSize

[in]结构体大小。

cProxy

[in]连接视频所经的上一级代理 IP 地址，不超过 32 个字符，包括‘\0’。

cNvsIP

[in]IP 地址，不超过 32 个字符，包括‘\0’。

cNvsName

[in]Nvs name 域名解析时使用。

cUserName

[in]登录 Nvs 用户名，不超过 16 个字符，包括‘\0’。

cUserPwd

[in]登录 Nvs 密码，不超过 16 个字符，包括‘\0’。

cProductID

[in]产品 ID，不超过 32 个字符，包括‘\0’。

iNvsPort

[in]该 Nvs 服务器所使用的通讯端口，若不指定则使用系统默认的端口(3000)。

cCharSet

[in]保留参数，暂时无用。

cAccontName

[in]保留参数，暂时无用。

cAccontPasswd

[in]保留参数，暂时无用。

6.1.11 人脸库信息结构体

```
typedef struct tagFaceLibInfo{
    int      iSize;
    int      iLibKey;
    char      cName[LEN_64];
    int      iThreshold;
    char      cExtrInfo[LEN_64];
    int      iAlarmType;
    int      iOptType;
    char      cLibUUID[LEN_UUID];
    char      cLibVersion[LEN_64];
}FaceLibInfo,*pFaceLibInfo;
```

Members

iSize

[in]结构体大小。

iLibKey

[in]人脸库键值（由设备生成）。

cName

人脸库名称。

iThreshold

人脸库阈值，0~100。

cExtrInfo

附加信息。

iAlarmType

是否上传识别信息。

0-不上传。

1-上传。

iOptType

操作指令。

1-添加人脸库。

2-修改人脸库。

cLibUUID

人脸库的唯一标识，若为空 *iLibKey* 为人脸库唯一标识（由上层生成维护，若上层不维护此项传空即可）。

cLibVersion

人脸库版本号

6.1.12 人脸库查询结果

```
typedef struct tagFaceLibQueryResult
{
```

```

    int                iSize;
    int                iChanNo;
    int                iTTotal;
    int                iPageNo;
    int                iIndex;
    int                iPageCount;
    FaceLibInfo        tFaceLib;
}FaceLibQueryResult,*pFaceLibQueryResult;
Members
iSize
[in]结构体大小
iChanNo
[out]通道号
iTTotal
[out]总条数
iPageNo
[out]页码
iIndex
[out]页码内序号
iPageCount
[out]每页个数，每页最大查询个
tFaceLib
[out]人脸库信息

```

6.1.13 人脸通用回复结构体

```

typedef struct tagFaceReply{
    int                iSize;
    int                iLibKey;
    int                iFaceKey;
    int                iOptType;
    int                iResult;
    char                cLibUUID[LEN_UUID];
    char                cFaceUUID[LEN_UUID];
    int                iDelLibProgress;
}FaceReply,*pFaceReply;
Members
iSize
[in]结构体大小。
iLibKey
[out]人脸库键值（由设备生成）。
iFaceKey
[out]人脸底图键值（由设备生成）。

```

[out]iOptType

操作指令。

- 0-添加人脸底图。
- 1-修改人脸底图。
- 2-删除人脸底图。
- 3-添加人脸库。
- 4-修改人脸库。
- 5-删除人脸库。
- 6-人脸底图建模。
- 7-查询人脸库。
- 8-查询人脸底图。
- 9-查询人脸属性。
- 10-人脸属性提取。
- 11-扣人脸。
- 12-以图搜底图。
- 13-同步人脸库。
- 14-查询同步人脸库状态。
- 15-以图搜抓拍图。
- 16-人脸库拷贝迁移。
- 17-人脸库解锁密码校验。

iResult

[out]操作结果。

- 0-成功。
- 1-系统准备好。
- 2-失败。
- 3-超出最大范围。
- 4-系统正在删除。
- 5-正在抠图。
- 6-系统正在执行以图搜抓拍图操作。
- 7-密码错误。

cLibUUID

[out]人脸库的唯一标识，若为空 iLibKey 为人脸库唯一标识（由上层生成维护，若上层不维护此项传空即可）。

cFaceUUID

[out]人脸底图的唯一标识，若为空 iFaceKey 为底图唯一标识（由上层生成维护，若上层不维护此项传空即可）。

iDelLibProgress

[out]进度。

当 iResult=4 时有效，表示人脸库删除进度，取值范围 0-100

当 iResult=6 时有效，表示以图搜抓拍图搜索进度，取值范围 0-100

6.1.14 人脸检测库删除结构体

```
typedef struct tagFaceLibDelete{
    int      iSize;
    int      iChanNo;
    int      iLibKey;
    char      cLibUUID[LEN_UUID];
}FaceLibDelete,*pFaceLibDelete;
```

Members

iSize

[in]结构体大小。

iChanNo

[in]通道号。

iLibKey

[in]人脸库键值（由设备生成）。

cLibUUID

[in]人脸库的唯一标识，若为空 *iLibKey* 为人脸库唯一标识（由上层生成维护，若上层不维护此项传空即可）。

6.1.15 查询人脸信息结构体

```
typedef struct tagFaceQuery
{
    int      iSize;
    int      iChanNo;
    int      iPageNo;
    int      iPageCount;
    int      iLibKey;
    int      iModeling;
    char      cName[LEN_64];
    int      iSex;
    char      cBirthStart[LEN_16];
    char      cBirthEnd[LEN_16];
    int      iNation;
    int      iPlace;
    int      iCertType;
    char      cCertNum[LEN_64];
    char      cLibUUID[LEN_UUID];
}FaceQuery,*pFaceQuery;
```

Members

iSize

[in]结构体大小

iChanNo
[in]通道号
iPageNo
[in]页码
iPageCount
[in]当前页条数
iLibKey
[in]人脸库 key 值
iModeling
[in]建模状态
cName[LEN_64]
[in]姓名
iSex
[in]性别
cBirthStart[LEN_16]
[in]出生日期
cBirthEnd[LEN_16]
[in]出生日期
iNation
[in]民族
iPlace
[in]籍贯
iCertType
[in]证件类型
cCertNum[LEN_64]
[in]证件号
cLibUUID[LEN_UUID]
[in]人脸库在平台的 ID

6.1.16 人脸底图查询结果

```
typedef struct tagFaceQueryResult{  
    int          iSize;  
    int          iChanNo;  
    int          iTotal;  
    int          iPageNo;  
    int          iPageCount;  
    int          iIndex;  
    FaceInfo     tFace;  
}FaceQueryResult,*pFaceQueryResult;  
Members  
iSize  
[in]结构体大小。
```

iChanNo

[out]通道号。

iTotal

[out]人脸底图总个数。

iPageNo

[out]人脸底图当前页码编号。

iPageCount

[out]人脸底图当前页总个数。

iIndex

[out]人脸底图信息在当前页面中索引。

tFace

[out]人脸底图信息。

6.1.17 人脸底图信息结构体

```
typedef struct tagFaceInfo{
    int          iSize;
    int          iLibKey;
    int          iFaceKey;
    int          iCheckCode;
    int          iFileType;
    int          iModeling;
    char         cName[LEN_64];
    int          iSex;
    char         cBirthTime[LEN_16];
    int          iNation;
    int          iPlace;
    int          iCertType;
    char         cCertNum[LEN_64];
    int          iOptType;
    char         cLibUUID[LEN_UUID];
    char         cFaceUUID[LEN_UUID];
    char         cLibVersion[LEN_64];
    char         cVerifyCode[LEN_64];
    int          iTaskId;
    char         cFileName[[LEN_256]];
    int          iSimilar;
}FaceInfo,*pFaceInfo;
```

Members

iSize

结构体大小。

iLibKey

人脸库键值。iLibKey=0，表示添加；iLibKey>0，表示修改。（由设备生成）

iFaceKey

人脸底图键值（由设备生成）。

iCheckCode

文件校验码，添加时有效。

iFileType

文件类型，添加时有效，0-jpg，1-png。

iModeling

是否建模，添加时有效，0-不建模，1-建模。

cName

人脸底图名称。

iSex

人脸底图性别，0-未知，1-男，2-女。

cBirthTime

人脸底图出生日期，格式为：YYYY-MM-DD。

iNation

人脸底图民族，按国家标准代码赋值，0 表示未知。

iPlace

人脸底图籍贯，高 16 位表示省，低 16 位表示市，分别按国家行政区划代码赋值，0 表示未知。

iCertType

人脸底图证件类型，0-未知，1-身份证，2-军官证。

cCertNum

人脸底图证件号码。

iOptType

操作指令。

1-添加人脸底图。

2-修改人脸底图。

cLibUUID

人脸库的唯一标识，若为空 iLibKey 为人脸库唯一标识（由上层生成维护，若上层不维护此项传空即可）。

cFaceUUID

人脸底图的唯一标识，若为空 iFaceKey 为底图唯一标识（由上层生成维护，若上层不维护此项传空即可）。

cLibVersion

人脸库版本号。

cVerifyCode

图片校验码，图片文件的 MD5。

iTaskId

事务 ID,以图搜底图有效。

cFileName

图片名称。

iSimilar

相似度，0-100（以图搜底图中的相似度）。

6.1.18 删除人脸结构体

```
typedef struct tagFaceDelete
{
    int            iSize;
    int            iChanNo;
    int            iLibKey;
    int            iFaceKey;
    char           cLibUUID[LEN_UUID];
    char           cFaceUUID[LEN_UUID];
}FaceDelete,*pFaceDelete;
```

Members

iSize

[in]结构体大小

iChanNo

[in]通道号

iLibKey

[in]人脸库 key 值

iFaceKey

[in]图片 key 值

cLibUUID[LEN_UUID]

[in]人脸库在平台的 ID

cFaceUUID[LEN_UUID]

[in]图片在平台对应 ID

6.1.19 报警模板使能结构体

```
typedef struct __tagAlarmScheEnableParam{
    int          iBuffSize;
    int          iEnable;
    int          iParam1;
    int          iParam2;
    int          iParam3;
    void*        pvReserved;
    char         iSceneID;
    char         cLibUUID[LEN_UUID];
}TAlarmScheEnableParam, *PTAlarmScheEnableParam;
```

Members

iBuffSize

[in]该结构体的大小（sizeof）。

iEnable

[in]使能，0--不使能；1--使能。

iParam1

[in]参数 1：

当 NetClient_SetAlarmConfig 接口中_iAlarmType=4（智能分析）时，表示规则 ID。

当 NetClient_SetAlarmConfig 接口中_iAlarmType=20（人脸识别）时，表示人脸库键值。

当 NetClient_SetAlarmConfig 接口中_iAlarmType=21（NVR 本地人脸识别）时，表示人脸库键值。

iParam2

[in]参数 2：

当 NetClient_SetAlarmConfig 接口中_iAlarmType=4（智能分析）时，表示事件类型（0：单绊线 1：双绊线 2：周界检测 3：徘徊 4：停车 5：奔跑 6：区域内人员密度 7：被盗物 8：遗弃物 9：人脸识别 10：视频诊断）。

当 NetClient_SetAlarmConfig 接口中_iAlarmType=20（人脸识别）时，表示人脸识别报警子类型（0：比对报警，1：陌生人报警）。

当 NetClient_SetAlarmConfig 接口中_iAlarmType=21（NVR 本地人脸识别）时，表示 NVR 本地人脸识别报警子类型（1：人脸识别-比对报警，2：人脸识别-陌生人报警，3：人脸识别-频次报警，4：人脸识别-滞留报警）。

iParam3

[in]参数 3，保留。

pvReserved

预留参数。

iSceneID

[in]场景 ID

cLibUUID

[in]人脸库的唯一标识，若为空人脸库键值为人脸库唯一标识（由上层生成维护，若上层不维护此项传空即可）。

6.1.20 报警联动参数结构体

```
typedef struct __tagAlarmScheduleParam{
    int                iBuffSize;
    int                iWeekday;
    int                iParam1;
    int                iParam2;
    NVS_SCHEDULETIME  timeSeg[MAX_DAYS][MAX_TIMESEGMENT];
    void*              pvReserved;
    char               iSceneID;
    char               cLibUUID[LEN_UUID];
}TAlarmScheduleParam, *PTAlarmScheduleParam;
```

Members

iBuffSize

[in]该结构体的大小（sizeof）。

iWeekday

[in]星期日到星期六（0-6）

iParam1

[in]参数 1:

当 NetClient_SetAlarmConfig 接口中_iAlarmType=4（智能分析）时，表示规则 ID。

当 NetClient_SetAlarmConfig 接口中_iAlarmType=20（人脸识别）时，表示人脸库键值。

当 NetClient_SetAlarmConfig 接口中_iAlarmType=21（NVR 本地人脸识别）时，表示人脸库键值。

iParam2

[in]参数 2:

当 NetClient_SetAlarmConfig 接口中_iAlarmType=4（智能分析）时，表示事件类型（0：单绊线 1：双绊线 2：周界检测 3：徘徊 4：停车 5：奔跑 6：区域内人员密度 7：被盗物 8：遗弃物 9：人脸识别 10：视频诊断）。

当 NetClient_SetAlarmConfig 接口中_iAlarmType=20（人脸识别）时，表示人脸识别报警子类型（0：比对报警，1：陌生人报警）。

当 NetClient_SetAlarmConfig 接口中_iAlarmType=21（NVR 本地人脸识别）时，表示 NVR 本地人脸识别报警子类型（1：人脸识别-比对报警，2：人脸识别-陌生人报警，3：人脸识别-频次报警，4：人脸识别-滞留报警）。

timeSeg[MAX_DAYS][MAX_TIMESEGMENT]

时间段

pvReserved

预留参数

iSceneID

[in]场景 ID

cLibUUID

[in]人脸库的唯一标识，若为空人脸库键值为人脸库唯一标识（由上层生成维护，若上层不维护此项传空即可）。

6.1.21 时间模板结构体

```
typedef struct {  
    unsigned short      iStartHour;  
    unsigned short      iStartMin;  
    unsigned short      iStopHour;  
    unsigned short      iStopMin;  
    unsigned short      iRecordMode;  
} NVS_SCHEDTIME,*PNVS_SCHEDTIME;
```

Members

iStartHour

开始时间，时：0-23。

iStartMin

开始时间，分：0-59。

iStopHour

结束时间，时：0-23。

iStopMin

结束时间，分：0-59。

iRecordMode

使能，位模式：bit0---定时录像；bit1---端口；bit2---移动侦测；bit3---视频丢失

6.1.22 人脸图片流相关结构体

```
typedef struct tagFacePicData{  
    int      iFaceId;  
    int      iDrop;  
    int      iFaceLevel;  
    RECT      tFaceRect;  
    int      iWidth;  
    int      iHeight;  
    int      iFaceAttrCount;  
    int      iFaceAttrStructSize;  
    FaceAttribute* ptFaceAttr[LEN_256];  
    int      iDataLen;  
    char*      pcPicData;  
    unsigned long long ullPts;  
    int      iAlramType;  
    int      iSimilativity;  
    int      iLibKey;  
    int      iFaceKey;
```



```

    int            iNegPicLen;
    char*          pcNegPicData;
    int            iNegPicType;
    int            iSex;
    int            iNation;
    int            iPlace;
    int            iCertType;
    char           cCertNum[LEN_64];
    char           cBirthTime[LEN_16];
    char           cName[LEN_64];
    char           cLibName[LEN_64];
    char           cLibUUID[LEN_64];
    char           cFaceUUID[LEN_64];
    char           cVerifyCode[LEN_36];
    int            iFeatureLen;
    char*          pcFeatureData;
    char           cFaceJpegName[LEN_64];
    char           cJpegName[LEN_64];
} FacePicData, *pFacePicData;

```

Members*iFaceId*

[out]人脸侦测号

iDrop

[out]人脸侦测结束：0 未结束，1 结束

iFaceLevel

[out]识别到的人脸质量

tFaceRect

[out]人脸坐标

iWidth

[out]人脸宽度

iHeight

[out]人脸高度

iFaceAttrCount

[out]人脸属性个数

iFaceAttrStructSize

[out]结构体 FaceAttribute 的大小

ptFaceAttr

[out]面部属性，最对支持 256 种（详见 6.1.53）。

iDataLen

[out]人脸图片数据长度

pcPicData

[out]人脸图片数据

ullPts

[out]数据生成的绝对时间

iAlramType

[out]0-人脸检测，1-比对报警（有底图），2-陌生人报警，3-频次报警，4-滞留报警

iSimilality

[out]相似度 0~100

iLibKey

[out]相匹配的人脸库键值

iFaceKey

[out]相匹配的人脸底图键值

iNegPicLen

[out]人脸底图长度

pcNegPicData

[out]人脸底图数据

iNegPicType

[out]人脸底图类型，0-jpg，1-png

iSex

[out]人脸底图性别，0-女，1-男

iNation

[out]人脸底图民族，国家标准民族代码， 0-未知

iPlace

[out]人脸底图籍贯，高 16 位表示省，低 16 位表示市，分别按国家行政区划代码赋值，0 未知

iCertType

[out]人脸底图证件类型，0-未知，1-身份证，2-军官证

cCertNum

[out]人脸底图证件号码

cBirthTime

[out]人脸底图出生日期，格式为：YYYY-MM-DD

cName

[out]人脸底图姓名

cLibName

[out]人脸所属库名称

cLibUUID[LEN_64]

[out]人脸库的唯一标识，若为空 iLibKey 为人脸库唯一标识

cFaceUUID[LEN_64]

[out]人脸底图的唯一标识，若为空 iFaceKey 为底图唯一标识

cVerifyCode[LEN_36]

[out]图片校验码，图片文件的 MD5

iFeatureLen

[out]人脸特征值数据长度

pcFeatureData

[out]特征值数据

cFaceJpegName[LEN_64]

[out]背景图（大图）文件名

cJpegName[LEN_64]

[out]特写图（小图）文件名

6.1.23 图片信息相关结构体

```
typedef struct tagPicData{  
    PicTime          tPicTime;  
    int              iDataLen;  
    char*            pcPicData;  
} PicData, *pPicData;
```

Members

tPicTime

[out]每张图片的抓拍时间:年-月-日-星期-时-分-秒-毫秒

iDataLen

[out]图片长度

pcPicData

[out]图片数据

6.1.24 时间模板结构体

```
typedef struct {  
    unsigned short          iStartHour;  
    unsigned short          iStartMin;  
    unsigned short          iStopHour;  
    unsigned short          iStopMin;  
    unsigned short          iRecordMode;  
} NVS_SCHEDULETIME, *PNVS_SCHEDULETIME;
```

Members

iStartHour

开始时间，时：0-23。

iStartMin

开始时间，分：0-59。

iStopHour

结束时间，时：0-23。

iStopMin

结束时间，分：0-59。

iRecordMode

使能，位模式：bit0---定时录像；bit1---端口；bit2---移动侦测；bit3---视频丢失

6.1.25 连接视频时需要传递给开发包的数据结构

```
typedef struct {
    int                m_iServerID;
    int                m_iChannelNo;
    char               m_cNetFile[255];
    char               m_cRemoteIP[16];
    int                m_iNetMode;
    int                m_iTimeout;
    int                m_iTTL;
    int                m_iBufferCount;
    int                m_iDelayNum;
    int                m_iDelayTime;
    int                m_iStreamNO;
    int                m_iFlag;
    int                m_iPosition;
    int                m_iSpeed;
}
```

} CLIENTINFO;

Members

m_iServerID

[in]NVS ID, NetClient_Logon 的返回值。

m_iChannelNo

[in]要连接的远程主机的视频通道号（从 0 开始），取值范围：[0, 通道数-1]，据设备类型而定，通道数由 NetClient_GetChannelNum 参数输出返回。

m_cNetFile

[in]播放网络上的文件,暂时不用。

m_cRemoteIP

[in]远程主机的 IP 地址。

m_iNetMode

[in]选择网络模式：1--TCP；2--UDP；3--多播。

m_iTimeout

[in]数据接收的超时时间。

m_iTTL

[in]多播时的 TTL 值。

m_iBufferCount

[in]缓冲个数。

m_iDelayNum

[in]第几个缓冲区块被填充后开始调用播放线程。

m_iDelayTime

[in]延时时间（秒）,保留。

m_iStreamNO

[in]码流类型，0：主码流；1：副码流；2：双向对讲；3：文件下载；4：图片抓拍；5：模拟抓拍流；6：轨迹流；7：纯音频通道。

m_iFlag

[in]0：首次请求该录像文件；1：操作录像文件。

m_iPosition

[in]0~100：定位文件播放位置；-1：不进行定位。

m_iSpeed

[in]取值为 1，2，4，8，控制文件播放速度。

6.1.26 前端录像文件查询条件结构体

```
typedef struct {
    int                iType;
    int                iChannel;
    NVS_FILE_TIME      struStartTime;
    NVS_FILE_TIME      struStopTime;
    int                iPageSize;
    int                iPageNo;
    int                iFiletype;
    int                iDevType;
} NVS_FILE_QUERY/PNVS_FILE_QUERY;
```

Members

iType

[in]录像类型：

0xFF----全部；

1----手动录像；

2----定时录像；

3----报警录像。

iChannel

[in]录像通道：0xFF-全部通道；0~channel-指定的通道号。

struStartTime

[in]文件的开始时间。

struStopTime

[in]文件的结束时间。

iPageSize

[in]每次查询返回记录的条数，取值范围：[1，20]。

iPageNo

[in]从第几页开始查。

iFiletype

[in]文件类型：

0----所有；

1----音视频文件；

2----图片。

iDevType

[in]设备类型：

0---摄像;
1---网络视频服务器;
2---网络摄像机;
0xff---全部。

6.1.27 录像时间属性结构体

```
typedef struct {  
    unsigned short      iYear;  
    unsigned short      iMonth;  
    unsigned short      iDay;  
    unsigned short      iHour;  
    unsigned short      iMinute;  
    unsigned short      iSecond;  
} NVS_FILE_TIME,*PNVS_FILE_TIME;
```

Members

iYear

年

iMonth

月

iDay

日

iHour

时

iMinute

分

iSecond

秒

6.1.28 录像文件属性结构体

```
typedef struct {  
    int      iType;  
    int      iChannel;  
    char      cFileName[250];  
    NVS_FILE_TIME      struStartTime;  
    NVS_FILE_TIME      struStoptime;  
    int      iFileSize;  
} NVS_FILE_DATA,*PNVS_FILE_DATA;
```

Members

iType

[out]录像类型：

1----手动录像；

2----定时录像；

3----报警录像。

iChannel

[out]录像通道，0~channel 为指定的通道号

cFileName

[out]文件名。

struStartTime

[out]文件的开始时间。

struStoptime

[out]文件的结束时间。

iFileSize

[out]文件的大小。

6.1.29 录像文件查询结构体

```
typedef struct tagNETFILE_QUERY_V5{
    int                iBufSize;
    int                iQueryChannelNo;
    int                iStreamNo;
    int                iType;
    NVS_FILE_TIME      tStartTime;
    NVS_FILE_TIME      tStopTime;
    int                iPageSize;
    int                iPageNo;
    int                iFiletype;
    int                iDevType;
    int                cOtherQuery[LEN_64 + 1];
    int                iTriggerType;
    int                iTrigger;
    int                iQueryType;
    int                iQueryCondition;
    char                cField[QUERY_MSG_COUNT][QUERY_MSG_LEN];
    int                iQueryChannelCount;
    int                iBufferSize;
    QueryFileChannel*  ptChannelList;
    char                cLaneNo[LEN_64 + 1];
    char                cVehicleType[LEN_64 + 1];
    int                iFileAttr;
    int                iQueryTypeValuey[LEN_6];
}NETFILE_QUERY_V5, *PNETFILE_QUERY_V5;
```

Members

iBufSize

[in]结构体大小。

iQueryChannelNo

[in]通道号：0x7FFFFFFF-全部通道；0~channel-指定的通道号。

iStreamNo

[in]码流号

0---主码流；

2---副码流。

iType

[in]文件类型：

0---所有；

1---音视频文件；

2---图片。

tStartTime

[in]文件的开始时间。

tStopTime

[in]文件的结束时间。

iPageSize

[in]每次查询返回记录的条数，取值范围：[1，20]。

iPageNo

[in]从第几页开始查。

iFiletype

[in]文件类型：

0---所有；

1---音视频文件；

2---图片。

iDevType

[in]设备类型：

0---摄像；

1---网络视频服务器；

2---网络摄像机；

0xff---全部。

cOtherQuery

[in]其他查询信息条件。如：叠加字符。

iTriggerType

[in]报警类型 3--端口报警；4--移动报警；5--视频丢失报警；0x7FFFFFFF--无效。

iTrigger

[in]通道号。(当 iTriggerType=3 时，改参数为端口号)。

iQueryType

[in]查询类型，0-基本查询，1-ATM 查询，2-ITS 查询。

iQueryCondition

[in]预留字段

iQueryChannelCount

[in]多通道查询时的通道个数：QueryChannelCount 为 0 时，查询 iQueryChannelNo，和 iStreamNo 中的录像，ptChannelList 不生效。

QueryChannelCount 大于 0 是表示查询的通道个数，使用 ptChannelList 中的通道列表查询。

iBufferSize

[in]ptChannelList 中的缓冲区大小。

ptChannelList

[in]查询的通道列表

cLaneNo

[in]车道号

cVehicleType

[in]车辆类型

iFileAttr

[in]文件属性，0：本地存储；10000：前端存储。

iQueryTypeValue

[in]扩展交通违法查询条件

6.1.30 主动模式登陆参数

```
typedef struct tagLogonActiveServer
```

```
{
```

```
    int        iSize;
```

```
    char        cUserName[LEN_16];
```

```
    char        cUserPwd[LEN_16];
```

```
    char        cProductID[LEN_32];
```

```
} LogonActiveServer;
```

Members

iSize

[in]结构体大小。

cUserName

[in]设备登录用户名，默认“admin”。

cUserPwd

[in]设备登录密码，默认“1111”。

cProductID

[in]设备出厂 ID。

6.1.31 录像文件属性扩展结构体

```
typedef struct{
```

```
    int                iSize;
```

```
    NVS_FILE_DATA      tFileData;
```

```
    int                iLocked;
```

```
}NVS_FILE_DATA_EX,*PNVS_FILE_DATA;
```

Members*iSize*

[out]结构体大小

tFileData

[out]文件基本信息

iLocked

[out]加解锁状态，0--未加锁，1--已加锁。

6.1.32 回放结构体

```

struct PlayerParam {
    int                iSize;
    char               strFileName[LEN_128];
    int                iLogonID;
    int                iChannNo;
    NVS_FILE_TIME      tBeginTime;
    NVS_FILE_TIME      tEndTime;
    int                iPlayerType;
    char               cLocalFilename[LEN_256];
    int                iPosition;
    int                iSpeed;
    int                iIFrame;
    int                iReqMode;
    int                iRemoteFileLen;
    int                iVodTransEnable;
    int                iVodTransVideoSize;
    int                iVodTransFrameRate;
    int                iVodTransStreamRate;
    RawFrameNotifyInfo tRawNotifyInfo;
}PlayerParam;

```

Members*iSize*

[in]结构体大小。

strFileName[LEN_128]

[in]前端录像文件名。

iLogonID

[in]设备登录 ID。

iChannNo

[in]设备通道。

tBeginTime

[in]录像开始时间。

tEndTime

[in]录像结束时间。

iPlayerType

[in]播放类型。

cLocalFilename

[in]本地录像文件名。

iPosition

[in]文件定位时,按百分比 0~100;断点续传时, 请求的文件指针偏移量。

iSpeed

[in]播放速度 1,2,4,8。

iIFrame

[in]I 帧帧率 1 直播 I 帧, 2 全部播放。

iReqMode

[in]需求数据的模式 1,帧模式;0,流模式。

iRemoteFileLen

[in]前端文件大小, 如果本地文件名不为空, 此参数置为空。

iVodTransEnable

[in]使能。

iVodTransVideoSize

[in]视频分辨率。

iVodTransFrameRate

[in]帧率。

iVodTransStreamRate

[in]码率。

tRawNotifyInfo

[in]原始流数据参数, 见 6.1.33

6.1.33 原始流数据结构体

```
struct _tagRawFrameNotifyInfo {
    RAWFRAME_NOTIFY      pcbkRawFrameNotify;
    void*                 pUserData;
    int                   iForbidDecodeEnable;
};
```

Members

pcbkRawFrameNotify

原始流数据参数。

pUserData

用户数据。

iForbidDecodeEnable

是否需要播放 0, 播放; 1. 不需要解码播放

CallBack function

```
typedef void (_stdcall *RAWFRAME_NOTIFY)(
```

```
    unsigned int          _ulID,  
    unsigned char*       _ucData,  
    int                  _iLen,  
    RAWFRAME_INFO*      _pRawFrameInfo,  
    void*                _iUser  
);  
CallBack function parameters  
_ulID  
[in] 连接标识，在 NetClient_StartRecv_V4 中返回的索引 ID 号。  
_ucData  
[in] 接收的数据。  
_iSize  
[in] 数据大小。  
_pRawFrameInfo  
[in] 音视频数据信息结构体（见 6.1.34）  
_iUser  
[in] 用户数据。
```

6.1.34 音视频数据信息

```
typedef struct {  
    unsigned int          nWidth;  
    unsigned int          nHeight;  
    unsigned int          nStamp;  
    unsigned int          nType;  
    unsigned int          nEnCoder;  
    unsigned int          nFrameRate;  
    unsigned int          nAbsStamp;  
    unsigned char         ucBitsPerSample;  
    unsigned int          uiSamplesPerSec;  
} RAWFRAME_INFO;  
Members  
nWidth  
[out]视频宽度，音频帧该成员为 0。  
nHeight  
[out]视频高度，音频帧该成员为 0。  
nStamp  
[out]时间戳(ms)。  
nType  
[out]  
RAWFRAMETYPE，类型定义 typedef int RAWFRAMETYPE。具体取值如下表：
```

宏定义	取值	含义
-----	----	----

VI_FRAME	0	I 帧
VP_FRAME	1	P 帧
AUDIO_FRAME	5	音频帧

nEnCoder

[out]编码类型

视频具体取值如下表所示：

宏定义	取值	含义
RAW_VIDEO_H264	1	H.264 编码
RAW_VIDEO_MPEG4	2	MPEG4 编码
RAW_VIDEO_MJPEG	41	MJPEG 编码
RAW_VIDEO_H265	23	H.265 编码

音频具体取值如下表所示：

宏定义	取值	含义
RAW_AUDIO_G711_A	0x01	G711A 编码
RAW_AUDIO_G711_U	0x02	G711U 编码
RAW_AUDIO_ADPCM_A	0x03	ADPCM 编码
RAW_AUDIO_AAC	0x16	AAC 编码

nFrameRate

[out]帧率。

nAbsStamp

[out]绝对时间。

ucBitsPerSample

[out]采样位，为 8/16/24,默认为 16。

uiSamplesPerSec

[out]采样率，默认为 8000。

6.1.35 人脸库查询结构体

```
typedef struct tagFaceLibQuery
{
    int          iSize;
    int          iChanNo;
    int          iPageNo;
    int          iPageCount;
}FaceLibQuery, *pFaceLibQuery;
```

Members

iSize

[in]结构体大小

iChanNo

[in]通道号

iPageNo

[in]页码

iPageCount

[in]当前页查询个数

6.1.36 人脸库修改结构体

```
typedef struct tagFaceLibEdit
{
    int            iSize;
    int            iChanNo;
    FaceLibInfo    tFaceLib;
}FaceLibEdit,*pFaceLibEdit;
```

Members

iSize

[in]结构体大小

iChanNo

[in]通道号

tFaceLib

[in]人脸库信息，见 6.1.35

6.1.37 人脸底图修改结构体

```
typedef struct tagFaceEdit
{
    int            iSize;
    int            iChanNo;
    char           cFacePic[LEN_256];
    FaceInfo       tFace;
}FaceEdit,*pFaceEdit;
```

Members

iSize

[in]结构体大小

iChanNo

[in]通道号

cFacePic[LEN_256]

[in]人脸底图名称

tFace

[in]人脸属性图信息，见 6.1.8

6.1.38 抠图结构体

```
typedef struct tagFaceCutEx
{
    int            iSize;
    int            iChanNo;
    char           cCheckCode[LEN_64];
    int            iPicType;
    char           cPicPath[LEN_256];
    int            iPageNo;
    int            iPageCount;
}FaceCutEx,*pFaceCutEx;
```

Members

iSize

[in]结构体大小

iChanNo

[in]通道号，从 0 开始

cCheckCode[LEN_64]

[in]文件校验码，用来校验人脸图片 MD5

iPicType

[in]文件扩展类型：0-jpg，1-png

cPicPath[LEN_256]

[in]需要抠图的图片路径

iPageNo

[in]页编号

iPageCount

[in]页大小

6.1.39 抠图结果结构体

```
typedef struct tagFaceCutQueryResult
{
    int            iSize;
    int            iChanNo;
    int            iTaskId;
    int            iTotal;
    int            iPageNo;
    int            iIndex;
    int            iPageCount;
    char           cFileName[LEN_256];
}FaceCutQueryResult,*pFaceCutQueryResult;
```

Members

iSize
[in]结构体大小
iChanNo
[out]通道号
iTaskId
[out]事务 ID
iTotal
[out]扣出人脸个数
iPageNo
[out]页编号
iIndex
[out]当前第几条，最大值 20
iPageCount
[out]当前页条数
cFileName[LEN_256]
[out]图片名称，图片下载走之前的图片下载协议

6.1.40 抠图结果查询结构体

```
typedef struct tagFaceSearch
{
    int                iSize;
    int                iChanNo;
    int                iTaskId;
    char               cLibKey[LEN_64];
    char               cPicName[LEN_256];
    int                iSimilar;
    int                iPageNo;
    int                iPageCount;
    int                iLibKey;
}FaceSearch,*pFaceSearch;
```

Members

iSize
[in]结构体大小
iChanNo
[in]通道号
iTaskId
[in]事物 Id
cLibKey[LEN_64]
[in]人脸库 id
cPicName[LEN_256]
[in]图片名称
iSimilar

[in]相似度
 iPageNo
 [in]页编号
 iPageCount
 [in]页大小
 iLibKey
 [in]人脸库 Id

6.1.41 搜索条件结构体

```
typedef struct tagFaceSearchSnap
{
    int                iSize;
    int                iChanCount;
    int                iChanSize;
    QueryChanNo*       pChanList;
    NVS_FILE_TIME      tBegTime;
    NVS_FILE_TIME      tEndTime;
    char                cPicturePath[LEN_256];
    int                iSimilarity;
    int                iSortMode;
    int                iTaskId;
}FaceSearchSnap, *pFaceSearchSnap;
```

Members

iSize
 [in]结构体大小
 iChanCount
 [in]通道个数
 iChanSize
 [in]结构体 QueryChanNo（见 6.1.42）大小
 pChanList
 [in]通道列表
 tBegTime
 [in]开始时间
 tEndTime
 [in]结束时间
 cPicturePath[LEN_256]
 [in]图片名称
 iSimilarity
 [in]相似度，0-100
 iSortMode
 [in]排序方式 0-按时间排序 1-按相似度排序
 iTaskId

[in]事物 Id

6.1.42 通道结构体

```
typedef struct tagQueryChanNo
{
    int          iChanNo;
    int          iStream;
}QueryChanNo, *PQueryChanNo;
```

Members

iChanNo

通道号

iStream

码流类型

6.1.43 查询搜索进度结构体

```
typedef struct tagFaceSearchSnapProcess
{
    int      iSize;
    int      iTaskId;
} FaceSearchSnapProcess, *pFaceSearchSnapProcess;
```

Members

iSize

[in]结构体大小

iTaskId

[in]事物 ID

6.1.44 搜索结果查询结构体

```
typedef struct tagFaceSearchSnapQuery
{
    int      iSize;
    int      iTaskId;
    int      iPageSize;
    int      iPageNo;
}FaceSearchSnapQuery, *pFaceSearchSnapQuery;
```

Members

iSize

[in]结构体大小
iTaskId
[in]事物 ID
iPageSize
[in]页大小
iPageNo
[in]页编号

6.1.45 搜索结果结构体

```
typedef struct tagFaceSearchSnapResult{  
    int                iSize;  
    int                iChanNo;  
    int                iTotat;  
    int                iCurPageCount;  
    int                iItemIndex;  
    NVS_FILE_TIME      tBegTime;  
    NVS_FILE_TIME      tEndTime;  
    int                iAge;  
    int                iSex;  
    int                iNation;  
    int                iWearGlasses;  
    int                iWearMask;  
    int                iSimilarity;  
    VcaFileAttr        tPicSnap;  
    VcaFileAttr        tPicNeg;  
}FaceSearchSnapResult, *pFaceSearchSnapResult;
```

Members

iSize
[in]结构体大小
iChanNo
[out]通道号
iTotal
[out]查询结果总数
iCurPageCount
[out]当前分页内结果总数
iItemIndex
[out]当前分页内序号
tBegTime
[out]开始时间
tEndTime
[out]结束时间
iAge

[out]年龄
iSex
[out]性别，1-男，2-女，3-未知
iNation
[out]民族，1-汉族，2-少数民族
iWearGlasses
[out]戴眼镜 0-预留，1-佩戴，2-未佩戴
iWearMask
[out]戴口罩 0-预留，1-佩戴，2-未佩戴
iSimilarity
[out]相似度
tPicSnap
[out]人脸大图属性（见 6.1.46）
tPicNeg
[out]人脸小图属性（见 6.1.46）

6.1.46 智能分析文件属性结构体

```
typedef struct tagVcaFileAttr
{
    int             iFileIndex;
    char            cFileName[LEN_256];
    int             iFileSize;
    int             iFileType;
    int             iReserve;
    char            cReserve[LEN_64];
} VcaFileAttr, *pVcaFileAttr;
```

Members

iFileIndex
[out]文件序号
cFileName[LEN_256]
[out]文件名称
iFileSize
[out]文件大小
iFileType
[out]文件类型， 1-小图，2-大图
iReserve
保留
cReserve[LEN_64]
保留

6.1.47 查询智能分析文件结构体

```
typedef struct tagNetFileQueryVca
{
    int                iSize;
    int                iChanCount;
    int                iChanSize;
    QueryChanNo*       pChanList;
    int                iVcaCount;
    int                iVcaList[MAX_QUERY_LIST_COUNT];
    NVS_FILE_TIME      tBegTime;
    NVS_FILE_TIME      tEndTime;
    int                iPageCount;
    int                iPageNo;
    int                iFileType;
    int                iConditionCount;
    char               cQueryCondition[MAX_QUERY_VCA_CONDITION][LEN_256];
}NetFileQueryVca, *pNetFileQueryVca;
```

Members

iSize

[in]结构体大小

iChanCount

[in]通道个数

iChanSize

[in]结构体 QueryChanNo（见 6.1.42）大小

pChanList

[in]通道列表

iVcaCount

[in]智能分析算法个数

iVcaList[MAX_QUERY_LIST_COUNT]

[in]智能分析算法列表

tBegTime

[in]开始时间

tEndTime

[in]结束时间

iPageCount

[in]页大小

iPageNo

[in]页编号

iFileType

[in]文件类型， 0：全部， 1：录像， 2：图片

iConditionCount

[in]查询条件个数，最大 32， 当值为 0 时，表示查询全部

cQueryCondition[MAX_QUERY_VCA_CONDITION][LEN_256]

当智能分析类型为 9 时:

cQueryCondition[0]: 表示检索类型 0-按特征检索, 1-按事件检索, 2-按图片检索;

cQueryCondition[1]: cQueryCondition[0]为 0 时 表示年龄, 1-少年, 2-青年, 3-中年, 4-老年;
cQueryCondition[0]为 1 时 1-人脸检测 2-人脸比对 3-陌生人 4-频次 5-时长;

cQueryCondition[2]: cQueryCondition[0]为 0 时 表示性别, 1-男, 2-女, 3-未知;

cQueryCondition[0]为 2 时 表示操作 1-查询 2-获取结果;

cQueryCondition[3]: cQueryCondition[0]为 0 时 表示民族, 1-汉族, 2-少数民族;

cQueryCondition[0]为 2 时 表示图片类型 1-指定文件 2-从人脸库中选择文件;

cQueryCondition[4]: cQueryCondition[0]为 0 时 表示姓名;

cQueryCondition[0]为 2, cQueryCondition[2]为 1 时 cQueryCondition[3]为 1 时 表示图片名称;

cQueryCondition[0]为 2, cQueryCondition[2]为 1 时 cQueryCondition[3]为 2 时 人脸库图片 Id;

cQueryCondition[5]: cQueryCondition[0]为 0 时, 表示戴眼镜 0-预留, 1-佩戴, 2-未佩戴

cQueryCondition[0]为 2, cQueryCondition[2]为 1 时 表示相似度

cQueryCondition[6]: cQueryCondition[0]为 0 时, 表示戴口罩 0-预留, 1-佩戴, 2-未佩戴

cQueryCondition[0]为 2, cQueryCondition[2]为 1 时 表示排序方式 0-按时间排序 1-按相似度排序

6.1.48 智能分析文件查询结果结构体

```
typedef struct tagNetFileQueryVcaResult
```

```
{
```

```
    int                iSize;
```

```
    int                iChanNo;
```

```
    int                iFileAttrCount;
```

```
    VcaFileAttr        tFileAttr[MAX_VCA_FILE_COUNT];
```

```
    int                iTotal;
```

```
    int                iCurPageCount;
```

```
    int                iItemIndex;
```

```
    int                iFileType;
```

```
    int                iVcaType;
```

```
    NVS_FILE_TIME      tBegTime;
```

```
    NVS_FILE_TIME      tEndTime;
```

```
    int                iExAttrCount;
```

```
    char                cExAttr[MAX_VCA_ATTR_COUNT][LEN_256];
```

```
}NetFileQueryVcaResult, *pNetFileQueryVcaResult;
```

Members

iSize

```

[in]结构体大小
iChanNo
[out]通道号
iFileAttrCount
[out]文件属性个数
tFileAttr[MAX_VCA_FILE_COUNT]
[out]文件属性
iTotal
[out]查询结果总数
iCurPageCount
[out]当前页总数
iItemIndex
[out]当前分页内序号
iFileType
[out]文件类型，1：录像，2：图片
iVcaType
[out]智能分析类型，此处为 9：人脸识别
tBegTime
[out]开始时间
tEndTime
[out]结束时间
iExAttrCount
[out]扩展属性字段个数
cExAttr[MAX_VCA_ATTR_COUNT][LEN_256]
cExAttr[0]: iVcaType=9 时，表示年龄值
cExAttr[1]: iVcaType=9 时，表示性别，1-男，2-女，3-未知
cExAttr[2]: iVcaType=9 时，表示民族，1-汉族，2-少数民族
cExAttr[3]: iVcaType=9 时，表示戴眼镜 0-预留，1-佩戴，2-未佩戴
cExAttr[4]: iVcaType=9 时，表示戴口罩 0-预留，1-佩戴，2-未佩戴
cExAttr[5]: iVcaType=9 时，表示相似度
cExAttr[6]: iVcaType=9 时，表示 cLibUUID 人脸库在平台的 ID
cExAttr[7]: iVcaType=9 时，表示 cFaceUUID 图片在平台对应 ID
cExAttr[8]: iVcaType=9 时，表示姓名 最大 64 个字节

```

6.1.49 设备在线状态结构体

```

typedef struct tagDsmOnline
{
    int          Size;
    char         cProductID[LEN_32];
    int          iOnline;
}

```

```
} DsmOnline;
```

Members

iSize

[in]结构体大小。

cProxy

[in]设备出厂 ID。

iOnline

[out]在线状态：0，不在线；1，在线。

6.1.50 报警回调结构体

```
typedef struct tagFaceAlarmNotify
{
    int                iSize;
    int                iChanNo;
    int                iState;
    int                iType;
    int                iFaceKey;
    int                iTrackId;
    PicTime            tCapTime;
}FaceAlarmNotify,*pFaceAlarmNotify;
```

Members

iSize

[in]结构体大小。

iChanNo

通道号

iState

1，报警；0，消警

iType

报警类型，人脸识别使用 ALARM_TYPE_FACE_IDENT=20

iFaceKey

iType=21 时，人脸识别底图的数据库 id

iTrackId

iType=21 时，人脸轨迹 id

PicTime

iType=20 时，代表人脸图片抓拍时间

6.1.51 图片抓拍时间结构体

```
typedef struct tagPicTime
{
```



```
    unsigned int uiYear;
    unsigned int uiMonth;
    unsigned int uiDay;
    unsigned int uiWeek;
    unsigned int uiHour;
    unsigned int uiMinute;
    unsigned int uiSecondsr;
    unsigned int uiMilliseconds;
} PicTime, *pPicTime;
uiYear
年
uiMonth
月
uiDay
日
uiWeek
星期 0-6
uiHour
时
uiMinute
分
uiSecondsr
秒
uiMilliseconds
毫秒
```

6.1.52 数字通道结构体

```
struct DigitalChnPara{
    int          iSize;
    int          iChannel;
    int          iEnable;
    int          iConnectMode;
    int          iDevChannel;
    int          iDevPort;
    int          iStreamType;
    int          iNetMode;
    int          iPtzAddr;
    char         cIP[33];
    char         cProxyIP[33];
    char         cPtzProtocolName[33];
    char         cUserName[17];
```

```

char        cPassword[17];
char        cEncryptKey[17];
int         iServerType;
char        cRTSPUrl[256];
char        cIPv6[64];
char        cMucIp[64];
int         iMucPort;
int         iActivation;
int         iSynchro;
char        cIpcMac[32];

```

```
}DigitalChnPara, *pDigitalChnPara;
```

Members

iSize

结构体大小

iChannel

数字通道号，取值范围据设备类型而定，登录时会向客户端发送数字通道数。

iEnable

使能，0--禁用该数字通道；1--启用该数字通道，默认为启用。

iConnectMode

连接方式，0--IP；1--域名；2--主动模式，默认为 IP。

iDevChannel

前端设备通道号，取值范围已前端设备能力而定，默认为 0。

iDevPort

前端设备端口号，取值范围：[81, 65535]，默认为 3000。

iStreamType

码流类型，0--主码流；1--副码流，默认为主码流。

iNetMode

网络类型，1--TCP；2--UDP；3--组播，暂定为 TCP,不可更改。

iPtzAddr

PTZ 地址，取值范围：[0, 256]，为空则使用默认值。

cIP[33]

连接参数，iConnectMode=0 时，IP 地址；iConnectMode=1 时，域名；iConnectMode=2 时，设备 ID，默认为空，至多 32 个字符。

cProxyIP[33]

代理 IP，iConnectMode=2 时，该字段无效，默认为空，至多 32 个字符。

cPtzProtocolName[33]

PTZ 协议，登录设备时会向客户端发送设备支持的 PTZ 协议列表，为空则使用默认值，至多 32 个字符。

cUserName[17]

用户名，登录前端设备的用户名，不可为空，至多 16 个字符。

cPassword[17]

密码，登录前端设备的密码，不可为空，至多 16 个字符。

cEncryptKey[17]

视频加密密码，前端设备视频加密的密码，为空则表示不加密，至多 16 个字符。

iServerType

前端设备所采用的网络协议(Miracle 新增字段) 0--私有协议，1--Onvif 协议，2--Push 流，3--RTSP 协议。

cRTSPUrl[256]

rtsp URL 连接地址

cIPv6[64]

IPV6 地址

cMucIp[64]

多播 IP 地址

iMucPort

多播端口号，80~65535

iActivation

激活状态，0：未激活 1：已激活

iSynchro

是否同步邮件或手机号到前端，0：不同步 1：同步

cIpcMac[32]

前端设备 Mac 地址，此字段是设备上报和回复时使用，客户端 SDK 发送时该字段发空即可

6.1.53 人脸属性结构体

```
struct tagFaceAttribute {
```

```
    int                iType;
```

```
    int                iValue;
```

```
}FaceAttribute;
```

Members

iType

人脸属性类型，最对支持 256 种，目前支持的人脸属性类型如下表：

iValue

人脸属性值，目前支持的属性类型所对应的属性值如下表（其他未列出的属性值取值范围 [0,100]，比如年龄属性值是 30 就代表 30 岁）：

iType 类型取值	说明	iValue 属性取值
0	年龄	30 就代表 30 岁
1	性别	0-女，1-男
2	口罩	0-无，1-有
3	胡子	0-无，1-有
4	睁眼	0-无，1-有
5	张嘴	0-无，1-有
6	眼镜	0-无，1-普通，2-太阳镜
7	种族	0-黄种人，1-黑种人，2-白种人，3-维族人
8	表情	0-生气，1-平静，2-厌恶，3-

		困惑，4-高兴，5-悲伤，6-害怕，7-惊讶，8-斜视，9-尖叫
9	微笑	0-无，1-有
10	颜值	取值范围[0,100]
11	民族	0-汉族，1-少数民族。(该字段为人脸算法计算的结果；注意区别于 FacePicData 里面 iNation 的民族字段，iNation 为用户添加人脸底图时所选择的民族)
17	目标关联 ID	与之相匹配的人形 ID (iFaceId)
18	目标类型	0-人脸，1-机动车，2-非机动车，3-人形
19	背包	0-无，1-有
20	运动方向	0-向上，1-向下，2-向左，3-向右
21	上身颜色	0-未知，1-白色，2-灰色，3-棕色，4-红色，5-蓝色，6-黄色，7-绿色，8-粉色，9-橙色，10-青色，11-紫色，12-浅蓝，13-黑色，14-彩色
22	下身颜色	0-未知，1-白色，2-灰色，3-棕色，4-红色，5-蓝色，6-黄色，7-绿色，8-粉色，9-橙色，10-青色，11-紫色，12-浅蓝，13-黑色，14-彩色
23	头发	0-短发，1-长发，2-秃头，3-其他
24	长袖	0-无，1-有
25	速度	0-静止，1-运动
26	戴帽子	0-无，1-有
27	长裤	0-无，1-有
28	短裙	0-无，1-有
29	骑车	0-无，1-有
30	频次报警次数/滞留报警时长（根据报警类型区分）	频次报警次数以实际取值为准（>0 有效）；滞留报警时长以实际取值为准，时长单位：秒（>0 有效）
31	场景（人脸点名球专用）	取值范围[0,65535]
32	民族概率	取值范围[0,100]，大于 50 为少数民族
33	人脸属性可信度	0-低，1-高

34	发色	0-未知, 1-白色, 2-灰色, 3-棕色, 4-红色, 5-蓝色, 6-黄色, 7-绿色, 8-粉色, 9-橙色, 10-青色, 11-紫色, 12-浅蓝, 13-黑色, 14-彩色
35	帽子颜色	0-未知, 1-白色, 2-灰色, 3-棕色, 4-红色, 5-蓝色, 6-黄色, 7-绿色, 8-粉色, 9-橙色, 10-青色, 11-紫色, 12-浅蓝, 13-黑色, 14-彩色
36	口罩颜色	0-未知, 1-白色, 2-灰色, 3-棕色, 4-红色, 5-蓝色, 6-黄色, 7-绿色, 8-粉色, 9-橙色, 10-青色, 11-紫色, 12-浅蓝, 13-黑色, 14-彩色
37	鞋子款式	0-皮鞋, 1-靴子, 2-休闲鞋, 3-凉鞋
38	鞋子颜色	0-未知, 1-白色, 2-灰色, 3-棕色, 4-红色, 5-蓝色, 6-黄色, 7-绿色, 8-粉色, 9-橙色, 10-青色, 11-紫色, 12-浅蓝, 13-黑色, 14-彩色
39	箱包类型	0-无, 1-单肩, 2-双肩, 3-手拎, 4-腰包, 5-拉杆
40	箱包颜色	0-未知, 1-白色, 2-灰色, 3-棕色, 4-红色, 5-蓝色, 6-黄色, 7-绿色, 8-粉色, 9-橙色, 10-青色, 11-紫色, 12-浅蓝, 13-黑色, 14-彩色
41	雨伞	0-未打伞, 1-打伞
42	雨伞颜色	0-未知, 1-白色, 2-灰色, 3-棕色, 4-红色, 5-蓝色, 6-黄色, 7-绿色, 8-粉色, 9-橙色, 10-青色, 11-紫色, 12-浅蓝, 13-黑色, 14-彩色
43	上衣款式	0-长款大衣, 1-夹克及牛仔服, 2-T 恤, 3-运动服, 4-羽绒服, 5-衬衫, 6-连衣裙, 7-西装
44	上衣模式	0-纯色, 1-条纹, 2-图案, 3-拼接, 4-格子
45	裤子类型	0-短裤, 1-长裤, 2-裙子
46	裤子模式	0-纯色, 1-条纹, 2-图案, 3-拼接, 4-格子

47	胸前抱东西	0-有，1-无
48	人形角度	0-正面，1-侧面，2-背面
49	推图是否轨迹结束	0 预留；1 轨迹结束；2 轨迹未结束
50	是否为活体	0：预留；1：活体；2：非活体

第 7 章 错误代码及说明

类型	数值	含义
ERROR_NO_DEV	USER_ERROR+0x01	没有找到设备
ERROR_UNLOGON	USER_ERROR+0x02	未登陆
ERROR_PARAM_OVER	USER_ERROR+0x03	参数越界
ERROR_REOPERATION	USER_ERROR+0x04	重复操作
ERROR_WSASTARTUP	USER_ERROR+0x05	WSAStartup 执行失败
ERROR_CREATEMSG	USER_ERROR+0x06	创建消息循环失败
ERROR_NOSUPPORTRECORD	USER_ERROR+0x07	不支持前端录像
ERROR_INITCHANNELNET	USER_ERROR+0x08	通道网络创建失败
ERROR_CREATEDDRAW	USER_ERROR+0x09	无法创建更多的 DirectDraw
ERROR_NO_CHANNEL	USER_ERROR+0x0A	通道没有创建
ERROR_NO_FUN	USER_ERROR+0x0B	无此功能
ERROR_PARAM_INVALID	USER_ERROR+0x0C	参数无效
ERROR_DEV_FULL	USER_ERROR+0x0D	设备列表已满
ERROR_LOGON	USER_ERROR+0x0E	设备已经登录，正在登录
ERROR_CREATECPTHREAD	USER_ERROR+0x0F	创建 CPU 检测线程失败
ERROR_CREATEPLAYER	USER_ERROR+0x10	创建 Player 失败
ERROR_NOAUTHORITY	USER_ERROR+0x11	权限不足
ERROR_NOTALK	USER_ERROR+0x12	未对讲
ERROR_NOCALLBACK	USER_ERROR+0x13	没有设备回调函数
ERROR_CREATEFILE	USER_ERROR+0x14	创建文件失败
ERROR_NORECORD	USER_ERROR+0x15	不是从当前 Player 发起的录像
ERROR_NOPLAYER	USER_ERROR+0x16	没有对应 Player
ERROR_INITCHANNEL	USER_ERROR+0x17	通道没有初始化
ERROR_NOPLAYING	USER_ERROR+0x18	Player 没有播放
ERROR_PARAM_LONG	USER_ERROR+0x19	字符串参数长度过长
ERROR_INVALID_FILE	USER_ERROR+0x1A	文件不符合要求
ERROR_USER_FULL	USER_ERROR+0x1B	用户列表已满
ERROR_USER_DEL	USER_ERROR+0x1C	当前用户无法删除
ERROR_CARD_LOAD	USER_ERROR+0x1D	加载卡 dll 失败

ERROR_CARD_CORE	USER_ERROR+0x1F	加载卡内核失败
ERROR_CARD_COREFILE	USER_ERROR+0x1F	加载卡内核文件失败
ERROR_CARD_INIT	USER_ERROR+0x20	卡初始化失败
ERROR_CARD_COREREAD	USER_ERROR+0x21	读取卡内核文件失败
ERROR_CHARACTER_LOAD	USER_ERROR+0x22	加载字库失败
ERROR_NOCARD	USER_ERROR+0x23	卡未初始化
ERROR_SHOW_MODE	USER_ERROR+0x24	显示模式未设置
ERROR_FUN_LOAD	USER_ERROR+0x25	函数未加载
ERROR_CREATE_DOWNLOAD	USER_ERROR+0x26	没有更多的下载通道可用
ERROR_PROXY_DELACT	USER_ERROR+0x27	没有更多的下载通道可用
ERROR_PROXY_HASCONNECT	USER_ERROR+0x28	还有连接
ERROR_PROXY_NOPROXY	USER_ERROR+0x29	代理没有启动
ERROR_PROXY_IDENTITY	USER_ERROR+0x2A	代理没有启动
ERROR_CONNECT_MAX	USER_ERROR+0x2B	连接已经到达最大
ERROR_NO_DISK	USER_ERROR+0x2C	没有挂接存储设备
ERROR_NO_DRAW	USER_ERROR+0x2D	没有足够的 Draw
ERROR_INIT_DATA_CHAN	USER_ERROR+0x2E	初始化数据通道失败
ERROR_SOCKET_CLOSED	USER_ERROR+0x2F	socket 关闭
ERROR_SOCKET_SEND_DATA	USER_ERROR+0x30	socket 发送数据失败
ERROR_DO_UPGRADING	USER_ERROR+0x31	正在升级
ERROR_ALREADY_INTERTALK	USER_ERROR+0x32	正在进行双向对讲
ERROR_FUNCTION_NOT_SUPPORTED	USER_ERROR+0x33	功能不被此设备所支持
ERROR_DISPLAY_ON_OTHER_WND	USER_ERROR+0x34	功能不被此设备所支持
ERROR_BUFFER_TOO_SMALL	USER_ERROR+0x35	缓冲区太小
ERROR_NVR_NOT_SUPPORT	USER_ERROR+0x36	此设备不支持 NVR 相关功能
ERROR_SOCKET_NULL	socket 未创建或已释放	socket 未创建或已释放
ERROR_EVENT_NOT_ENABLE	USER_ERROR+0x38	本规则中此事件没有使能
ERROR_RULE_NOT_ENALE	USER_ERROR+0x39	此规则没有使能
ERROR_INIT_CHAN	USER_ERROR+0x3A	初始化通道失败
ERROR_PROLEN_INVALID	USER_ERROR+0x3B	协议长度不合法
ERROR_UNSUPPORT_CHARSET	USER_ERROR+0x3C	不支持的字符集
ERROR_CHARTRANS_FAILED	USER_ERROR+0x3D	字符转码失败
ERROR_WAITREPLY_FAILED	USER_ERROR+0x3E	等待设备或目录服务器回码失败
ERROR_NOTFOUND_PLAYERID	USER_ERROR+0x3F	SDK 内部未找到合法的 playerID
ERROR_UNSUPPORT_STREAM	USER_ERROR+0x40	不支持码流类型
ERROR_INVALID_DECLIBIDX	USER_ERROR+0x41	解码库索引不合法

ERROR_COMMANDCHAN_EMPTY	USER_ERROR+0x50	CommanChan 空
-------------------------	-----------------	--------------

第 8 章 SDK 返回值及说明

类型	数值	含义
RET_NVD_ISVIEW	1	网络视频解码器 SDK 接口返回值，存在正在连接的 NVS
RET_NOTHING	((void)0)	
RET_SUCCESS	0	成功
RET_FAILED	-1	失败
RET_MALLOC_FALIED	-2	申请内存失败
RET_INVALIDPARA	-3	传入非法参数
RET_FAILED_UPGRADE	-4	网络视频服务器 SDK 接口返回值，升级失败
RET_APPLYMEMORY_FAILED	-5	内存申请失败
RET_INVALIDFILE	-7	文件格式非法
RET_NOTLOGON	-8	没有登录，或登陆句柄已失效
RET_BUFFER_FULL	-11	缓冲区满，数据没有放入缓冲区
RET_BUFFER_WILL_BE_FULL	-18	即将满，降低送入数据的频率
RET_BUFFER_WILL_BE_EMPTY	-19	即将空，提高送入数据的频率
RET_BUFFER_IS_OK	-20	正常可以放数据
RET_INVALID_ARRAY_INDEX	-21	不合法的数组下标
RET_BUILD_PROTOCOL	-24	组协议错误
RET_SEND_PROTOCOL	-25	发送协议错误
RET_INVALID_FILEHEADER	-27	无效文件头
RET_LIGHTMODE_NOTSUPPORT	-28	Sdk 不支持轻量级获取参数
RET_LOAD_OSCORE	-29	Sdk 加载 OsCore.dll 失败
RET_INVALID_OPT_TYPE	-30	非法的操作类型
RET_GETDATA_END	-50	获取数据正常结束，读到文件尾或者播放停止位置
RET_NVD_ERR_SYSTEM	-100	系统错误
RET_INVALID_PARA	-101	非法参数
RET_DEVICE_NOT_LOGON	-102	设备未登录
RET_MEMORY_OVER	-103	内存不足
RET_ERR_AUTHORITY	-104	权限不足
RET_SEND_LIST_FULL	-105	发送列表满了
RET_INVALID_MEMORY	-110	非法内存
RET_SYSTEM_CALL_FAILED	-200	调用系统函数失败
RET_SDK_CALL_FAILED	-201	调用 SDK 接口失败
RET_INNER_CALL_FAILED	-202	调用内部函数失败
RET_DRAW_CREATE_FAILED	-203	创建 Draw 失败

RET_NOSUPPORT_PARATYPE	-204	轻量级获取参数不支持的参数类型
RET_LIGHTLOGON_NETERROR	-205	轻量级发协议网络错误
RET_LIGHTLOGON_GET_TIME_OUT	-206	轻量级发协议等待设备回复超时
RET_DEV_NOT_SUPPORT	-207	设备不支持当前参数获取方式
RET_GETREGDEVCOUNT_TIMEOUT	-208	带目录的主动模式获取注册设备总数超时
RET_ERR_RECREATE_DECODER	-209	需要重新创建解码器
RET_FACE_CUT_QUERY_TIMEOUT	-210	查询人脸抠图结果超时返回
RET_SYNCLOGON_TIMEOUT	-300	同步登录超时，网络正常，但对端无任何响应
RET_SYNCLOGON_USERNAME_ERROR	-301	同步登录失败，用户名不存在，默认用户名：admin
RET_SYNCLOGON_USRPWD_ERROR	-302	同步登录失败，密码错误，默认密码：，如果用户修改过密码请输入正确密码。
RET_SYNCLOGON_PWDERRTIMES_OVERRUN	-303	同步登录失败，密码错误次数超限，账号已锁定
RET_SYNCLOGON_NET_ERROR	-304	同步登录失败，网络连接错误
RET_SYNCLOGON_PORT_ERROR	-305	同步登录失败，通信端口输入有误，默认传入端口
RET_SYNCLOGON_UNKNOW_ERROR	-306	同步登录失败，错误未知
RET_SYNCREALPLAY_TIMEOUT	-307	同步连接视频超时，对端没有发送视频头协议
RET_SYNCREALPLAY_FAIL	-308	同步连接视频时播放视频失败
RET_SYNCSUSPENDVCA_CONFIGING	-309	同步暂停智能分析失败，智能分析参数正被其他客户端设置
RET_SYNCSUSPENDVCA_FAIL	-310	同步暂停智能分析失败，设备未回码
RET_SYNCOPENVCA_CONFIGING	-311	同步开启智能分析失败，智能分析参数正被其他客户端设置
RET_SYNCOPENVCA_FAIL	-312	同步开启智能分析失败，设备未回码
RET_SYNCOPTVCA_TIMEOUE	-313	同步暂停/开启智能分析超时